UNIVERSITY OF COPENHAGEN
DEPARTMENT OF COMPUTER SCIENCE



Master's thesis

Aditya Fadhillah

Configurable Interactive Web Interface for the Fabrication Experiment Design Tool

Designing and implementing a web interface to support the planning and execution of fabrication experiments.

Date: 1. June 2026

Advisor: Valkyrie Savage

Table of Contents

1 Introduction **6**

2 Related Works **7**

2.1 Programmatic Interfaces 8

2.2 Tools for Experimental Design and Planning 8

2.3 Declarative Interaction and Reactive Execution Models 9

2.4 Block-Based Programming 10

2.5 Gap 10

3 Background **11**

3.1 Design of Experiments (DOE) 11

3.2 Fabrication Experiment Design Tool (FEDT) 12

3.3 Blockly 13

4 Abstraction Model **13**

4.1 Core Experimental Concepts 14

4.2 Visual Representation and API Translation 14

4.3 Declarative DOE Structure 15

4.4 Three-Section Scaffold 15

4.5 Generated Representations 16

5 System Design and Implementation **16**

5.1 Formative Paper Prototype 16

5.2 Architecture 17

5.3 Block Type System 18

5.4 Code Generation 19

5.5 AST-Based Static Analysis 23

5.6 Modes of Operation and SSE Streaming 25

5.7 Data Panel and File Management 26
5.8 Saved Experiments and Workspace Persistence 27

6 Validation and Results **28**

6.1 Validation Methodology 28
6.1.1 Code Compilation Verification 29
6.1.2 Flowchart Structure Comparison 30
6.1.3 CSV Output Verification 30
6.2 Numerical Validation Results 30
6.2.1 Code Compilation Success Rate 30
6.2.2 FEDT-Specific Pattern Preservation 30
6.2.3 Flowchart Structure Comparison 31
6.2.4 CSV Output Verification 31
6.2.5 Design Table Preview Accuracy 32
6.3 Standard Factorial Designs 32
6.3.1 Complex Multi-Factor Experiments 35
6.4 Measurement Abstractions and Device Extensions 36
6.4.1 Standard Measurement Devices 37
6.4.2 Manual Intervention and Dynamic Files 38
6.4.3 Textile Fabrication and Class-Based Iteration 41
6.5 Discovered Limitations 45
6.5.1 Static Analysis and Preview Boundaries 46
6.5.2 Large Design Spaces 46
6.5.3 List Indexing in Generated Code 46
6.5.4 Adaptive Termination and Runtime-Determined Sample Counts 47
6.5.5 Runtime Assignment and Conditional Branching 49
6.5.6 Systematic Counterbalancing 49
6.6 Validation Summary 50

7 Discussion **53**

7.1 Interpretation of Validation Results . . . 53

7.2 Dual Contribution Revisited . . . 54

7.3 Design Implications from Implementation Experience . . . 54

7.3.1 Scaffold over Feature Parity . . . 54

7.3.2 Design Tables as Machine-Readable Specifications . . . 55

7.3.3 Instruction Blocks as Controlled Escape Hatches . . . 55

7.3.4 Device Extensibility and Maintenance Cost . . . 55

7.3.5 Trade-offs Between Visual and Textual Programming . . . 56

7.4 Implications for Fabrication Experiment Design . . . 56

7.4.1 Design Space Visibility . . . 56

7.4.2 Reproducibility Through Structured Representations . . . 56

7.4.3 Boundaries of Declarative Experiment Design . . . 57

7.5 Generalizability Beyond FEDT . . . 57

7.6 Limitations and Threats to Validity . . . 57

7.6.1 Self-Evaluation Approach . . . 57

7.6.2 Coverage-Based Rather Than User Study Evaluation . . . 58

7.6.3 Static Analysis Boundaries . . . 58

7.7 Future Work Directions . . . 59

8 Conclusion **59**

Acknowledgements **61**

A User Walkthrough: Creating a Laser Cutting Experiment **64**

A.1 Building the Experiment . . . 64

A.2 Execution and Results . . . 69

A.3 Formative Paper Prototype and Early Layout Sketches . . . 72

Abstract

In fabrication experiments, researchers must keep track of design parameters, machine settings, fabrication steps, and measurements if the experiment is to be repeatable. The Fabrication Experiment Design Tool (FEDT) supports this through executable Python specifications, but its command-line and code-centric workflow can be difficult for users who are less comfortable with programming. This thesis presents a web-based block interface that uses Blockly to author FEDT experiments through visual blocks. The interface is organized around Design of Experiments principles and a three-section scaffold: Define Factors, Fabricate, and Measure. From this structure, the system generates executable FEDT Python code, supports Evaluate and Execute modes, and provides planning previews through design tables, measurement tables, and flowcharts.

The system was validated by implementing 22 fabrication experiments from FEDT demonstration examples and published HCI fabrication papers. For the implemented cases, the generated programs ran in the FEDT environment without manual edits, preserved key FEDT patterns such as `Parallel`, `Series`, and `BatchMeasurements`, and produced corresponding flowchart structures and CSV outputs for supported declarative experiments. The results indicate that the interface works well for factorial experiment designs, device-specific fabrication abstractions, and systematic measurement workflows. Its main limitations concern adaptive termination, runtime-dependent branching, counterbalanced assignment, large design spaces, and incomplete generator coverage for more complex value expressions. The evaluation therefore shows which FEDT experiment patterns the interface can represent and run. It does not evaluate usability, and it does not claim full FEDT feature parity.

1 Introduction

Digital fabrication allows researchers to prototype physical artifacts and test variations in material, geometry, and machine settings through controlled experiments. However, running these experiments remains challenging: researchers must coordinate geometry generation, machine configurations, fabrication sequences, and measurement protocols while maintaining reproducibility and documentation. The Fabrication Experiment Design Tool (FEDT) [16] addresses this challenge by treating experiments as executable programs, capturing the workflow from design through measurement in structured Python code.

While FEDT supports precise and reproducible descriptions of fabrication experiments, its command-line interface and Python workflow can create barriers for students, non-technical collaborators, and researchers who primarily work with visual design tools. This creates the central tension addressed in this thesis: structured experiment specifications can make procedures explicit and reduce ambiguity, but a programming-based interface may limit who can comfortably author and inspect them.

This thesis explores whether a block-based web interface can provide an alternative entry point to FEDT while preserving the precision of executable experiment specifications. The approach combines two ideas. First, block-based visual programming environments such as Scratch [9] and Blockly [13] make program structure visible and manipulable through connected blocks rather than typed syntax. Blockly is especially relevant here because it is not a complete end-user language by itself; applications built with Blockly must define their block vocabulary, code generator, and execution model. Second, Design of Experiments (DOE) [1] methodology provides a planning structure in which factors and levels are declared before fabrication begins. Together, these ideas led to the interface structure used in this thesis: users first define what varies, then what is fabricated, and finally what is measured.

The central research question is:

Can visual programming provide an alternative entry point to FEDT while maintaining inspectable experiment structure and executable code generation for supported fabrication patterns?

To answer this question, this thesis presents a web-based block interface for FEDT experiment design. The system uses Blockly as the editing workspace and translates structured blocks into FEDT Python code. The interface is organized around a three-section scaffold. This scaffold makes the experiment structure inspectable while constraining code generation to supported FEDT patterns. The system also provides design tables, measurement tables, and flowcharts, allowing the planned experiment structure to be checked before execution.

Scope of this thesis. The main evaluation in this thesis is technical and coverage-based rather than a formal usability study. The thesis evaluates whether the proposed interface can represent, translate, preview, and execute a broad set of FEDT experiment structures, especially declarative factorial designs where factors and levels are known before execution. A small formative paper prototype informed the

interface design, but it is not treated as evidence of usability, learnability, or error reduction. The thesis therefore does not claim to prove that the interface is easier to learn or more usable than FEDT's Python workflow. Instead, it establishes the system design, implementation feasibility, translation correctness, and practical boundaries of the approach. Formal usability evaluation with novice and expert users is left as future work.

The contributions of this work are:

1. A block-based web interface for FEDT. The thesis presents a web application that allows users to construct FEDT experiments through visual blocks for factors, fabrication steps, and measurements. The interface separates experiment logic from execution configuration and exposes planning previews before fabrication.

2. A dual-perspective framing for evaluating the interface. The thesis evaluates the system from both a visual representation perspective and an API translation perspective. The first concerns whether experiment structure becomes inspectable through blocks, tables, and flowcharts. The second concerns whether these visual configurations generate FEDT Python code that runs in the FEDT environment and preserves the intended structure for supported patterns.

3. A coverage-based validation across FEDT experiments. The system is validated by implementing 22 fabrication experiments from the FEDT demonstration examples and published HCI fabrication papers [17, 8, 5, 23, 24, 22, 20, 7, 4, 21, 18, 6]. This evaluation identifies supported patterns, such as declarative factorial designs and systematic measurement workflows, as well as boundaries involving adaptive termination, runtime-dependent branching, counterbalancing, and incomplete generator coverage for some value expressions.

Project repository. The source code for the web interface, backend, and supporting materials is available at: `https://github.com/AditFadhillah/Thesis_Submission`

The thesis is organized as follows. Section 2 reviews related work on fabrication programming systems, experimental design tools, declarative interfaces, and block-based programming. Section 3 introduces DOE, FEDT, and Blockly as the technical background. Section 4 presents the abstraction model behind the interface. Section 5 describes the system design and implementation. Section 6 presents the validation and results. Section 7 discusses implications, limitations, and future work. Section 8 concludes the thesis.

2 Related Works

This chapter reviews work in two areas: programmatic systems for specifying and running fabrication workflows and interface approaches that support planning, inspection, and execution.

FEDT is the immediate technical point of reference because it provides both a representation of fabrication experiments and an execution model that combines computation with human and machine actions. The interface presented in this thesis builds on that foundation by examining how FEDT-like experiment structures can be made inspectable through a block interface.

2.1 Programmatic Interfaces

Several recent fabrication systems treat fabrication workflows as programs that can be inspected, edited, executed, and shared. FEDT frames HCI fabrication experiments as orchestrated workflows spanning CAD/CAM configuration, fabrication actions, post-processing, and measurement. It provides an embedded Python DSL for specifying these workflows while producing execution-time artifacts such as flowcharts and measurement support files. This makes FEDT an important foundation for this thesis, but its code-centric interface also motivates the question of how such precision can be made more approachable in practice. [16]

Other work has used computational notebooks to combine fabrication control with documentation. Imprimer reframes hobbyist CNC milling as a literate programming activity in which documentation, interactive visualizations, and machine-control code are combined in a notebook environment. The system emphasizes rapid iteration, live feedback, and learnable abstractions, such as hiding or revealing code, alongside situated previews, such as augmented reality projections onto the machine. [11]

Tandem extends the notebook-as-program framing toward reproducibility of complex digital fabrication workflows. It treats a workflow as a multimodal program that can connect to CAD and CAM tools, coordinate augmented reality overlays for manual interventions, and directly control machines. Tandem also introduces assertions as a way to encode preconditions and common pitfalls related to physical-digital mismatches. [12]

These systems are relevant because they treat fabrication workflows as executable or inspectable artifacts, rather than only as written descriptions. This thesis takes up a related interface problem: how to show program structure, parameters, and execution state in a way that users can inspect while planning an experiment.

2.2 Tools for Experimental Design and Planning

Fabrication experiments are often bench workflows rather than human-subjects protocols, but they still involve core experimental-design problems: choosing factors and values, structuring repetitions, deciding which steps must be ordered, and ensuring that assignments or orderings reflect intended constraints. PLanet addresses a related problem in HCI by formalizing experimental assignment procedures as a grammar of composable operators. Users express assignment requirements, and the system compiles them into constraint-satisfaction problems over matrices to generate viable experimental plans. Its contribution is

not only plan generation, but also making design assumptions explicit and supporting exploration of alternatives. [2]

The HCI User Studies Toolkit illustrates how web-based tooling can scaffold planning work for novice researchers by bundling method selection guidance, study design planning, informed-consent generation, scheduling support, and participation confirmation. Although it targets user studies rather than fabrication workflows, its motivation is relevant here: structured interfaces can make protocol steps, assumptions, and required information more explicit for users who are still learning how to plan experiments. [19]

Two distinctions from this work are useful for this thesis. First, design-time support, which helps users reason about a plan before running it, differs from execution-time support, which helps users carry out the plan. Second, planning tools are valuable when they reveal assumptions, constraints, and trade-offs. The web interface developed in this thesis uses design tables, flowcharts, and measurement previews to make these aspects of a fabrication experiment visible before and after execution.

2.3 Declarative Interaction and Reactive Execution Models

Since the interface must keep blocks, generated code, tables, and flowcharts consistent, related work on declarative interactive systems is relevant. Reactive Vega and Vega-Lite are influential examples from interactive visualization. Reactive Vega describes a dataflow architecture derived from a declarative specification, where input data, scene graph elements, and interaction events are treated as first-class streaming data sources. Updates then propagate through the generated dataflow graph. [15]

Vega-Lite provides a higher-level grammar for interactive graphics. Interaction is expressed through composable selections, which can parameterize visual encodings, scale domains, and filtering behavior. The Vega-Lite compiler then generates lower-level Vega specifications with the required dataflow and event-handling logic. [14] Although Vega and Vega-Lite target visualization rather than fabrication, they are relevant here because they show how a declarative specification can be compiled into coordinated runtime behavior while keeping multiple views linked through structured state.

In the context of the FEDT interface, similar ideas support traceability and consistency. The interface needs to keep the Blockly workspace, generated FEDT Python code, design tables, measurement tables, and flowcharts aligned with the same underlying experiment specification. The architecture therefore keeps visual edits linked to a structured experiment state, which is used to generate inspectable outputs before execution.

2.4 Block-Based Programming

A key interface challenge for FEDT-like systems is that the underlying representation is programmatic. Block-based programming environments offer lessons for making program structure more visible and approachable. Scratch, for example, is a visual programming environment designed to support self-directed learning through tinkering, personally meaningful projects, and peer collaboration. Its design includes a single-window layout, direct manipulation of command blocks, and immediate feedback during script execution [9].

Prior work on Scratch also provides concepts for thinking about how people work in such environments. Brennan and Resnick propose a framework of computational concepts, practices, and perspectives, emphasizing that learning is not only about syntax but also about iterative work, debugging, and reuse or remix [3]. These ideas are relevant to this thesis because the interface aims to make experiment structure visible and inspectable rather than requiring users to understand the generated Python code first.

Blockly-specific design guidance further emphasizes that Blockly is a library for constructing block-based programming environments rather than a complete language or application. Pasternak et al. [13] argue that developers must define the vocabulary, API mappings, and runtime behavior of their block language according to the intended audience and application context. They also caution that exposing too many blocks can overwhelm users and recommend consistency in wording, visual design, input order, and defaults.

The goal is not to expose FEDT's Python API as a one-to-one collection of visual blocks, since that would reproduce much of the complexity of the code-based workflow. Instead, the interface defines a smaller experiment-oriented vocabulary around factors, fabrication steps, measurements, and execution support. The resulting block language is therefore used as a design layer: it preserves the structures needed for FEDT code generation while presenting experiment structure through concepts that can be inspected visually.

2.5 Gap

Prior systems provide either powerful programmatic ways to specify and run fabrication workflows, such as DSLs and notebook-style environments, or planning support through web-based guidance and formal design exploration. What is less explored is how a DSL such as FEDT can be made more inspectable for lab users who are less comfortable with command-line workflows while still preserving its executable Python representation.

Simply turning each FEDT function into a block would not solve the problem. A direct block-for-block copy of FEDT would preserve much of the same complexity that makes the original workflow difficult for new users. The more important design challenge is to provide abstractions aligned with how

people describe experiments: what is being varied, what artifact is being produced, what gets repeated, what is measured, and which steps belong to the experiment logic rather than auxiliary setup.

This thesis addresses that gap with a web-based block interface layered on FEDT. The interface provides mechanisms for defining factors and levels, inspecting run structure and experiment graphs, previewing generated design and measurement tables, and distinguishing experiment logic from execution-support details such as machine configuration. The intended contribution is a design and implementation that preserves FEDT's underlying representation while presenting experiment structure through representations that are more directly inspectable than raw script alone.

3 Background

3.1 Design of Experiments (DOE)

Design of Experiments (DOE) is a methodology for systematically studying how input factors influence selected response variables. Instead of varying one parameter at a time, DOE supports experiments where several factors are varied together, making it possible to study both individual factor effects and interactions between factors [10, 1].

Changing one variable at a time can be inefficient and may miss interactions between factors. An interaction occurs when the effect of one factor depends on the level of another factor [1]. In fabrication experiments, this matters because the effect of a material, geometry parameter, or machine setting may depend on another variable.

A common DOE pattern is the full factorial design, where all combinations of factor levels are tested. If k factors are considered and each has two levels, the design contains 2^k experimental runs [10]. More generally, the number of runs is the product of the number of levels for each factor. This structure makes the planned experiment explicit before execution begins, because the factor definitions can be expanded into a design matrix where each row represents one run.

DOE also follows a broader experimental workflow: defining the research problem, establishing objectives, selecting factors and responses, choosing the design structure, executing the planned runs, recording measurements, analyzing results, interpreting findings, and validating conclusions [10]. For this thesis, the most relevant aspect is the declarative structure of DOE. Factors and levels are declared upfront, fabrication is applied across the planned combinations, and measurements are collected systematically for each run.

The web interface uses this DOE structure as the basis for its three main sections: Define Factors, Fabricate, and Measure. Factors and levels are declared first, fabrication steps are applied across the resulting design matrix, and measurements are associated with each experimental run. This makes ex-

perimental scope visible and enables automatic generation of design tables. It also imposes constraints that differ from FEDT's more flexible imperative programming model, a trade-off examined in the abstraction model, validation, and discussion sections.

3.2 Fabrication Experiment Design Tool (FEDT)

The Fabrication Experiment Design Tool (FEDT) is the technical foundation of this thesis. FEDT addresses a common problem in HCI fabrication research: replication can be difficult when procedural details are spread across notes, scripts, machine settings, and lab practice. FEDT responds by representing fabrication experiments as executable Python workflows that can be inspected, executed, and shared [16].

FEDT is implemented as an embedded domain-specific language (DSL) in Python for orchestrating experiments across the interactive object fabrication workflow. This includes design, configuration, toolpath generation, fabrication, post-processing, and user interaction, together with the measurement and data-capture steps needed for experimental evaluation. Its design is guided by four goals: *precision*, so procedures are unambiguous; *comprehensiveness*, so workflows that mix human and machine actions can be represented; *extensibility*, so new machines and software tools can be integrated; and *analysability*, so experiment structure can be inspected, compared, and communicated [16].

In FEDT, an experiment is written as a Python function instrumented by the FEDT runtime. As the function executes, FEDT records events such as instructions, loop structure, object creation, fabrication steps, and measurements. It can then produce flowcharts, machine files, measurement templates, and CSV key files that connect objects and measurements to experimental conditions [16]. This allows one experiment definition to function both as an executable procedure and as a communication artifact for reporting or sharing methods.

A central distinction for this thesis is the difference between planning and execution. In the FEDT paper, this planning mode is called *Design mode*; in this thesis, the corresponding web-interface mode is called *Evaluate mode*. In this mode, device execution is disabled, but the program still runs far enough to generate a flowchart and compute derived information such as object and measurement counts. However, the FEDT paper describes flowcharts and counts rather than a complete tabular preview of the factorial design space. In *Execute mode*, FEDT performs file generation, prompts users for required manual actions, and invokes machine or tool executors where available. After execution, FEDT can export CSV files that record experimental conditions and collected measurements [16].

FEDT also provides experiment-oriented control flow and measurement abstractions. It distinguishes loops whose iterations may occur in any order, using `Parallel`, from loops that must be executed sequentially, using `Series`. It supports randomization through `shuffle`, open-ended iteration through `while` loops, step-by-step measurement entry, and batch measurement collection through generated CSV tem-

plates. These constructs are important for this thesis because the web interface must decide which FEDT patterns can be represented directly as blocks, which require simplified representations, and which fall outside the current scaffold.

FEDT was evaluated by modeling fabrication experiments from published papers and discussing these models with original authors. This process surfaced missing details and incorrect assumptions that were easy to overlook in prose, and authors described value in using executable specifications as a contract for collaborators and students [16]. At the same time, FEDT's interaction paradigm is primarily code- and command-line-centric. This motivates the central question of this thesis: how FEDT-like experiment structures can be exposed through a web-based block interface while preserving executable FEDT Python code.

This thesis builds on FEDT by treating it as the underlying representation and execution engine. Rather than translating FEDT's Python API one-to-one into visual blocks, the interface introduces a higher-level abstraction organized around core experimental questions: what factors are varied, what artifact is produced, what steps are repeated, and what is measured?

3.3 Blockly

Blockly is an open-source JavaScript library for adding block-based visual programming to applications. It provides a block editor and a code-generation framework, but it does not provide a complete end-user language or runtime by itself. Developers using Blockly must therefore define the block vocabulary, decide how blocks map to application concepts, and implement how the generated code is executed [13].

This matters here because the thesis does not use Blockly as a generic editor. The web interface is not simply a generic Blockly editor placed on top of FEDT; it defines a domain-specific block language for fabrication experiments. The block vocabulary, connection constraints, and code generator are designed around experimental concepts such as factors, fabrication steps, and measurements rather than around a direct one-to-one copy of FEDT's Python API.

4 Abstraction Model

The web interface is not a one-to-one visual translation of FEDT's Python API or command-line workflow. Instead, it organizes FEDT experiments around four questions: what varies, what is fabricated, what is measured, and how the procedure is ordered. The goal is to support structured fabrication experiments that can be inspected in the interface and translated into executable FEDT Python code.

This abstraction prioritizes experimental concepts over full FEDT feature parity. Factors, runs, fabrication steps, sequencing, and measurements form the main structure of the workspace, while machine settings, file handling, slicer parameters, and device-specific options remain nested inside relevant blocks.

This separates experimental logic from supporting configuration and avoids turning the interface into a block-for-block copy of FEDT's Python API.

4.1 Core Experimental Concepts

The interface is organized around a small set of experimental concepts.

Factor: A variable that is intentionally changed to study its effect on an outcome, such as material, laser power, infill density, geometry size, wall thickness, or post-processing method.

Level: One possible value of a factor. For example, a material factor may have the levels wood and acrylic, while a rectangle-length factor may have levels of 50 mm, 75 mm, and 100 mm.

Artifact design: The digital or procedural description of what will be fabricated or tested. In FEDT, this may refer to a geometry file, generated shape, modified model, or other structured fabrication artifact.

Run: One concrete execution of the experiment under a specific combination of factor levels and procedural steps. Each run corresponds to a row in the design table and, when measurements are collected per run, to a row in the measurement table.

Experiment graph: A procedural representation of the experiment showing sequence, repetition, branching, and dependencies between steps. In the web interface, this appears as a FEDT-generated flowchart.

4.2 Visual Representation and API Translation

The system can be evaluated from two sides: what the interface makes visible and what code it generates.

Visual Representation Perspective: The interface represents experiment structure through blocks and generated previews. Blocks show the experiment as connected elements in the workspace, while previews expose the planned design space, expected measurement schema, and procedural flow. The question from this perspective is whether the experiment structure is inspectable without first reading the generated Python code.

API Translation Perspective: The system also acts as a translation layer from visual block configurations to executable FEDT Python code. Blocks map to FEDT constructs or helper code, preserving patterns such as `Parallel`/`Series` iteration semantics and `BatchMeasurements` accumulation. From this perspective, the generated program must be syntactically valid, run in the FEDT environment, and preserve the intended experiment structure for supported patterns.

Both perspectives are needed. Blocks and previews are only useful if they correspond to the generated FEDT program, and valid code generation alone would not make the workflow inspectable. The abstraction model therefore has to support both visible experiment structure and consistent translation to

FEDT's execution model.

4.3 Declarative DOE Structure

The interface adopts Design of Experiments (DOE) as its organizing structure. Factors and levels are declared before execution, allowing the system to enumerate the planned design space and present it as a design table. This planning-first structure lets users check the run count, factor combinations, and measurement columns before using material, machine time, or manual effort.

This declarative structure differs from FEDT's more general imperative Python model. FEDT can express factorial experiments through nested loops, but it can also express adaptive procedures where iteration counts, branches, or assignments are determined during execution. The block interface prioritizes experiments whose design space is known before execution because these are the cases where static previews and structured code generation are most reliable.

The same choice creates a boundary. Adaptive termination, runtime-determined sample counts, complex branching, and counterbalanced assignment schemes are possible in FEDT's Python DSL, but they do not fit naturally into the DOE-based scaffold. The interface therefore does not expose lower-level programming features such as arbitrary file I/O, dynamic imports, custom Python functions, or highly imperative control flow. Adding these features would make the block language closer to general Python and weaken the experiment-focused structure. This trade-off between declarative clarity and imperative expressiveness is examined in the validation and discussion chapters.

4.4 Three-Section Scaffold

The Blockly workspace is organized into three sections that mirror the DOE workflow:

1. **Define Factors:** Experimental variables are declared with their names, levels, and iteration semantics (`Parallel` or `Series`). Nested factors create multi-factor designs and define the combinations that will be executed.
2. **Fabricate:** Fabrication steps specify how artifacts are produced, including geometry definitions, machine configuration, and fabrication operations. These blocks can reference the factor variables defined in the first section.
3. **Measure:** Measurement operations specify what data is collected for each fabricated object. Measurements can be collected during execution or exported as batch-entry templates.

The scaffold asks users to specify the experiment in order: first the design space, then the fabrication steps, and finally the measurements. This order also simplifies code generation, because factor variables are introduced before they are referenced by fabrication and measurement blocks.

4.5 Generated Representations

The visual representation perspective is supported by three generated representations: design tables, measurement tables, and flowcharts.

Design Table. The design table displays the factorial expansion of the declared factors. Each row corresponds to one planned run, and each column corresponds to a factor or derived parameter. This preview helps reveal configuration errors such as missing factor levels, unexpected run counts, or incorrect parameter bindings before fabrication begins.

Measurement Table. The measurement table shows the expected data schema for the experiment. Each row corresponds to an experimental object or run, and each column corresponds to a measurement type. In Evaluate mode, the table is shown as an empty template. In Execute mode, measurement values are collected during execution or entered later through batch measurement files.

Flowchart. The flowchart shows the procedural structure generated by FEDT, including sequence, repetition, branching, and dependencies between experimental steps. In Evaluate mode, it supports planning. In Execute mode, it helps users follow progress through fabrication and measurement steps.

Each representation shows a different part of the experiment: the design table shows factor combinations, the measurement table shows the expected data columns, and the flowchart shows the execution order. The implementation of these representations, including AST-based table extraction, is described in Section 5.5.

5 System Design and Implementation

This section describes how the abstraction model is implemented in the web interface through Blockly blocks, generated FEDT Python code, AST-based static analysis, and runtime integration with FEDT. The system uses a local client-server architecture: the browser frontend provides the Blockly workspace, while the backend runs generated experiments using the unmodified FEDT library. The frontend is implemented in React with TypeScript and Google Blockly[1]. The backend is implemented with FastAPI, exposes FEDT operations through REST endpoints, streams interactive execution events through Server-Sent Events (SSE), and extracts table previews from generated FEDT Python code. Figure 7 shows the full interface.

5.1 Formative Paper Prototype

Before implementing the Blockly interface, a low-fidelity paper prototype was used as formative design input to explore which FEDT concepts should be exposed as visual blocks and how users might interpret

[1] `https://developers.google.com/blockly`

a guided experiment-building workflow. The prototype task asked participants to construct a material breakage experiment by creating a data collection variable, loading a geometry file, iterating over three materials, laser cutting each object, giving manual instructions, and recording a breaking-weight measurement. This task was chosen because it combines recurring FEDT concepts that later became central to the interface: factor iteration, geometry loading, fabrication, human-in-the-loop instructions, and measurement collection.

The sessions influenced the block vocabulary and the structure of the interface. In particular, the task supported the separation between Define Factors, Fabricate, and Measure; the manual instruction steps motivated explicit instruction blocks and execution prompts; and the need to inspect whether a workflow was valid motivated feedback mechanisms such as generated code, flowcharts, and table previews. Early sketches of the paper prototype and web layout are included in Appendix A.3.

Because the paper-prototype sessions were exploratory, small-scale, and conducted before the final implementation, they are not used as evidence that the completed system is easier to learn or less error-prone than FEDT's command-line workflow. Instead, they are reported as formative design work that informed the block vocabulary, three-section scaffold, instruction prompts, and preview mechanisms.

5.2 Architecture

The system is organized into three layers:

1. **Blockly frontend** (React + TypeScript, served by Vite). Users construct experiments by connecting blocks in the Blockly workspace. The workspace state is stored as XML, Blockly's native workspace format, and translated to FEDT Python code by a custom TypeScript generator.
2. **FastAPI backend** (Python, served by Uvicorn). The backend receives generated code, writes it to a temporary file, and dynamically loads it as a Python module using `importlib`. The backend provides both synchronous endpoints for simple execution and an interactive endpoint that runs experiments in a background thread to support real-time SSE streaming for both Execute and Evaluate modes. The backend also manages geometry file uploads, saved experiment storage, generated output files, and the SSE endpoint for interactive execution. In addition, the backend contains an AST analyzer that parses generated FEDT Python code and returns design and measurement table structures as JSON. Because the backend executes generated Python code directly, the system is intended as a local, trusted research tool rather than as a public multi-user service.
3. **FEDT library** (Python). The FEDT codebase is used unmodified. The backend sets the `control.MODE` global variable to select Evaluate or Execute mode before the generated module is run. During execution, calls from the generated code enter the FEDT core, which reads the current mode to determine whether device actions should be evaluated or executed. FEDT then produces flowcharts, CSV measurement templates, key files, and console output, which the backend collects

and returns to the frontend.

The frontend and backend communicate through two mechanisms. Non-interactive operations such as compilation, evaluation, code analysis, and file management use HTTP requests with JSON responses. For table previews, the frontend sends generated FEDT Python code to the backend, where the AST analyzer extracts design and measurement structures and returns them as JSON. The frontend then renders these JSON structures as tables in the Data panel. Interactive execution uses an SSE stream that carries execution output, instruction prompts that pause execution until the user responds, measurement events, and completion events with final artifacts.

Figure 1 summarizes the system architecture, including the data flow between layers and the two communication mechanisms.

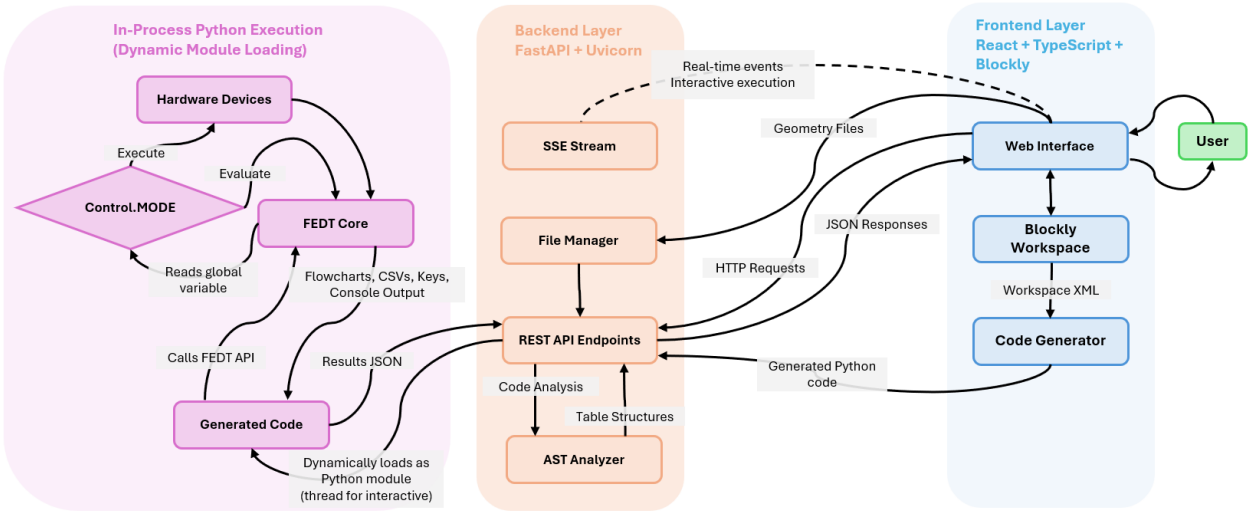


Figure 1: System architecture showing the three-layer design. The **Frontend Layer** (blue) contains the Blockly interface, workspace XML, and code generator. The **Backend Layer** (orange) provides REST endpoints, file management, SSE streaming, and AST-based code analysis. It parses generated FEDT Python code and returns JSON table structures that the frontend renders in the Data panel. The **In-Process Python Execution** layer (purple) runs the generated FEDT code by dynamically loading it as a module within the backend's Python process. FEDT outputs such as flowcharts, CSV files, key files, and console output are returned to the interface, while SSE streams provide real-time updates during interactive execution.

5.3 Block Type System

The Blockly workspace provides custom blocks organized into functional categories in the block palette. Each block type specifies its shape, connection rules, visual styling, and code generation behavior. These definitions, implemented in `fedt_blocks.js`, define how blocks appear and interact in the editor while also providing the metadata needed to map blocks to FEDT Python constructs.

Table 1 lists the available blocks organized by category. As shown in Figure 2 (left), each category uses a distinct color.

The central structural block is `iteration_block`, which implements the three-section scaffold de-

Section	Category	Blocks
Experiment Definition	Experiment Setup	`fedt_experiment`, `iteration_block`, `summarize_function`
	Factors & Nestedness	`define_factor`
Execution Setup	Artifact Design	`geometry_file`, `stl_cube`, `create_rectangle`, `create_circle`, `svg_editor_design`
	Fabrication	`printer_fab`, `laser_fab`, `knitting_machine_knit`, `loom_fab`, `human_post_process`
	Process Instructions	`instruction`, `human_confirm`
	Measurement	`batch_measurements`, `immediate_measurements`, `measure_size`, `measure_custom`, `take_picture`
Programming Utilities	Control Flow	`parallel_loop`, `series_loop`, `while_loop`, `shuffle`, `random_choice`
	Variables	`variables_get`, `variables_set`, `assign_variable`
	Logic	`logic_boolean`, `logic_compare`, `logic_operation`
	Math	`math_number`, `math_arithmetic`, `range_block`, `arange_block`
	Lists	`lists_create_with`, `list_multiply`, `list_concatenate`, `lists_getIndex`
	Text	`text`, `text_join`

Table 1: Block palette organization with representative block names for each category.

scribed in Section 4.4 (see Figure 2, top right). It enforces the DOE-inspired structure by requiring factor definitions before fabrication and measurement steps. Its three connection areas, Define Factors, Fabricate, and Measure, use Blockly connection constraints so that factor, fabrication, and measurement blocks attach only to their corresponding sections.

Device-specific blocks, such as `knitting_machine_knit` and `loom_fab`, encapsulate domain knowledge by exposing relevant parameters as block fields. For example, `loom_type_selector` provides a dropdown menu with five loom types (SPEERLoom, Jacq3GLoom, AlbaughLoom, AshfordLoom, and TC2Loom), rather than requiring users to remember API names. This makes device options explicit in the interface, but it also means that each supported device needs its own block definitions.

Figures 2–4 show the block palette and representative blocks, including dropdown menus, text fields, and nested value inputs.

5.4 Code Generation

The frontend includes a custom Python code generator, implemented in TypeScript in `generatePython.ts`, that traverses the Blockly workspace and emits FEDT Python code. Instead of using Blockly's standard generator API, it walks the block tree directly by following `getNextBlock()` connections for sequential blocks and `getInputTargetBlock()` for nested blocks. Supported block types are then translated into the corresponding Python statement or expression. This traversal makes it easier to handle the three-

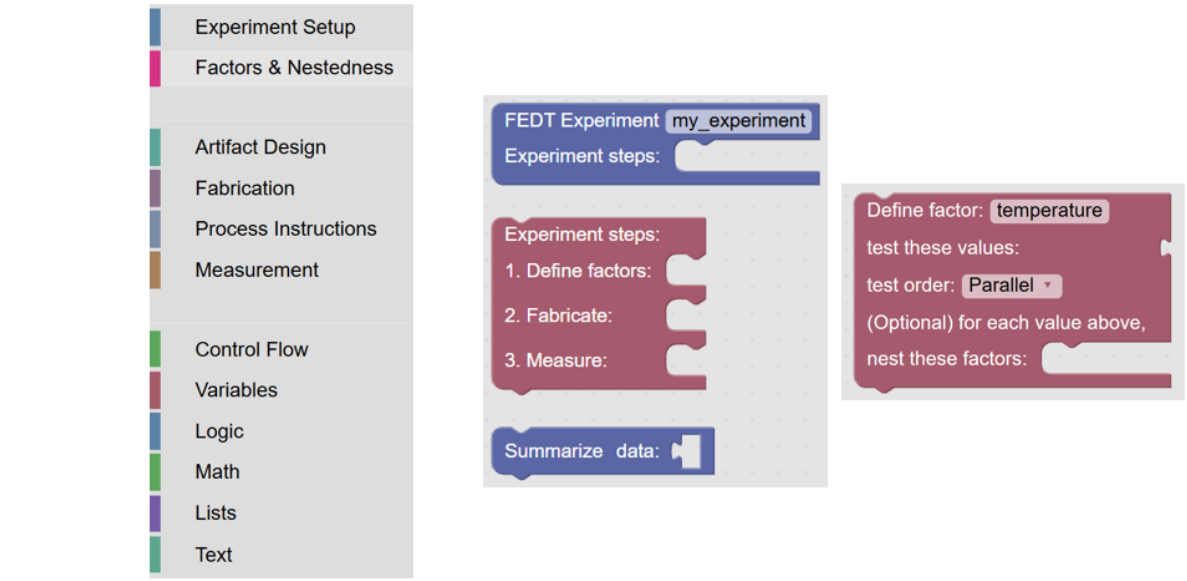


Figure 2: Left: The block palette organized into three sections: Experiment Definition, Execution Setup, and Programming Utilities. **Mid:** Experiment Setup (`iteration_block`) scaffold with its Define Factors, Fabricate, and Measure sections. **Right:** Factors (`define_factor`) block showing factor name, test values, iteration order, and optional nested factors.

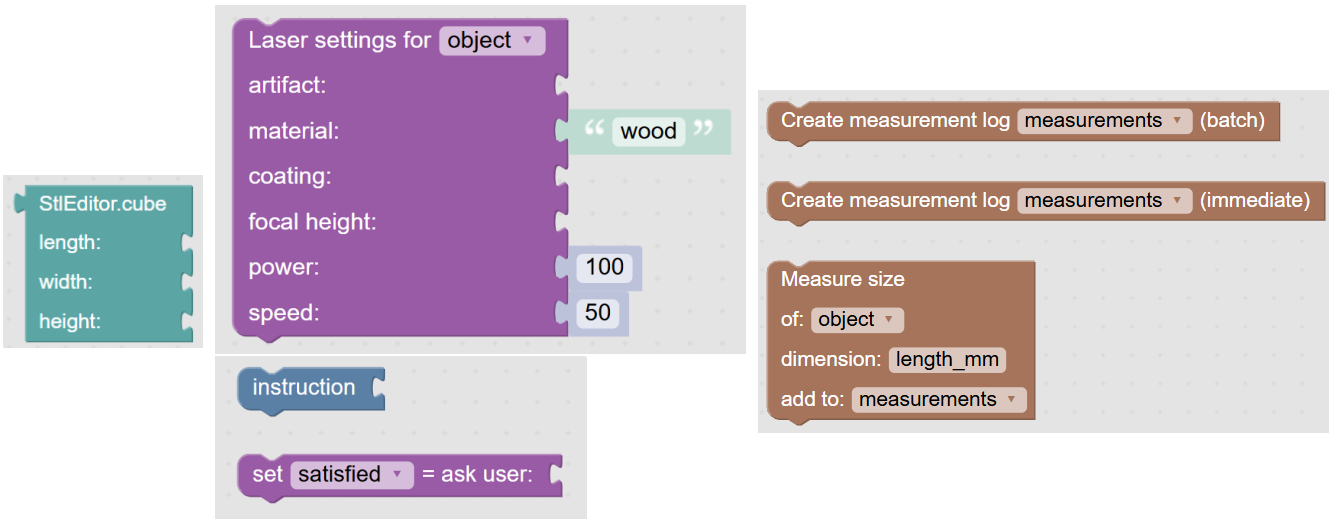


Figure 3: Blocks from the Execution Setup section. **Left to right:** Artifact Design (`stl_cube`) for parametric geometry; Fabrication (`laser_fab`) containing settings for devices; Process Instructions (`instruction`, `human_confirm`) for human interaction workflows; and Measurement (`batch_measurements`, `measure_size`) category for logs and measurements.

section scaffold and to separate statement blocks from value blocks.

The code generator distinguishes between statement blocks (which produce Python statements like `for` loops or function calls) and value blocks (which produce Python expressions like `range(0, 10)` or variable references). The `blockToCode()` function handles statement blocks and indentation, while `valueToCode()` returns Python expressions for value blocks.

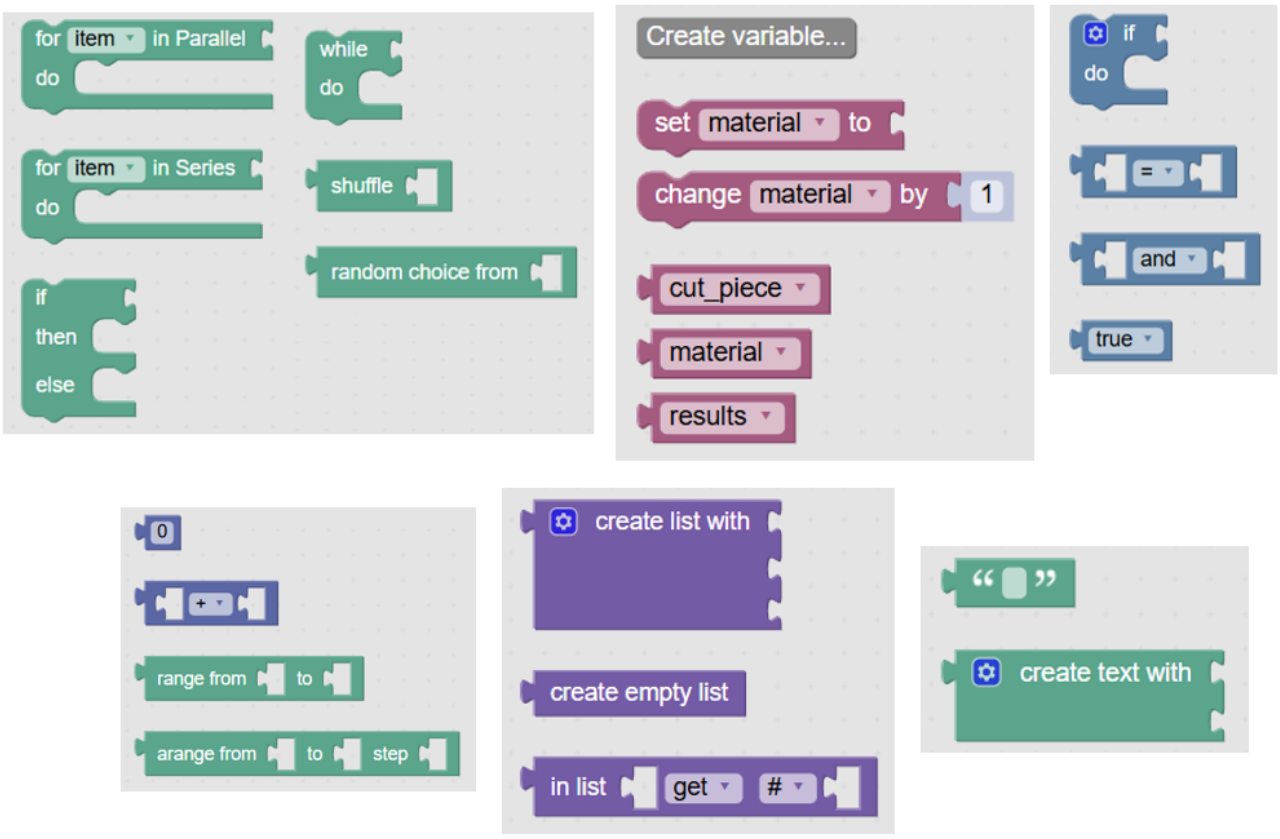


Figure 4: Blocks from the Programming Utilities section. The top row shows control-flow (`parallel_loop`, `series_loop`, `random_choice`), variable (`variables_get`, `variables_set`), and logic (`logic_compare`, `logic_operation`) blocks. The bottom row shows math (`math_number`, `math_arithmetic`, `range_block`), list (`lists_create_with`, `list_multiply`), and text (`text`, `text_join`) blocks.

Generated files follow a fixed structure:

1. Import statements for FEDT modules such as `control`, `iterators`, `measurement`, `flowchart`, `decorator`, and `lib`, together with standard-library helpers such as `random` where required, followed by a helper `summarize()` function.
2. The experiment function decorated with `@fedt_experiment` containing the block-generated logic.
3. A `__main__` guard with commented-out calls to `render_flowchart()` and the experiment function (users can uncomment these for standalone execution).

The generator uses three main patterns.

Factor iteration. The `iteration_block` processes `define_factor` blocks to generate nested `for` loops. For example, a factor named `material` with levels `["PLA", "ABS"]` and parallel iteration generates:

```
for material in Parallel(["PLA", "ABS"]):
    # 2. Fabricate
    # (fabrication blocks here)
    # 3. Measure
    # (measurement blocks here)
```

Each `define_factor` block specifies one experimental factor, its levels, and whether its iteration should use FEDT's `Parallel` or `Series` wrapper. When multiple factors are nested in the scaffold, the generator emits nested `for` loops. With nested factors, the generated experiment becomes full-factorial: the runs are the Cartesian product of the declared factor levels. For example, a two-factor experiment with materials `["PLA", "ABS"]` and infill values `[20, 50, 80]` produces $2 \times 3 = 6$ runs. The same generated loop structure is later used by the AST analyzer to construct the design table preview.

Expression evaluation. The `arange_block` generates Python expressions that include FEDT's `include_last` constant to ensure endpoint inclusion[2]. An arange block with start 0, end 100, and step 10 generates:

```
list(arange(0, 100+include_last, 10))
```

This keeps the generated code consistent with FEDT's endpoint-handling pattern. Because value blocks are processed recursively, an arange block whose end value is computed by a math block can generate expressions such as `50*2+include_last`.

The `random_choice` block is also treated as a value block. It allows a generated expression to select one value from a provided list, so local randomized choices can be embedded inside instructions, parameter assignments, or other generated expressions.

The generator covers the value-block patterns needed by the supported validation cases, including literals, variables, arithmetic expressions, ranges, `arange()`, list construction, list concatenation, list multiplication, and local random choice. However, not every Blockly value block is fully translated. In particular, `lists_getIndex` can appear in the visual workspace, but the current generator does not yet emit a Python indexing expression.

Measurement accumulation. Measurement blocks generate FEDT's `+=` accumulation pattern. A sequence of three `measure_size` blocks targeting the same measurement variable generates:

```
measurements = BatchMeasurements.empty()  # FEDT's initialization for measurement collections
measurements += Calipers.measure_size(obj, "x_axis (mm)")
measurements += Calipers.measure_size(obj, "y_axis (mm)")
measurements += Calipers.measure_size(obj, "z_axis (mm)")
```

The generator reads each block's measurement field to determine which measurement variable it should update, allowing multiple measurement logs to coexist in the same experiment.

[2] This addresses floating-point precision issues where ranges like `arange(0, 1.0, 0.1)` might exclude the endpoint due to rounding errors.

The Code panel shows the generated FEDT Python code next to the workspace, so users can inspect how the blocks are translated. Figure 5 shows the complete generated code for a simple laser cutting experiment.

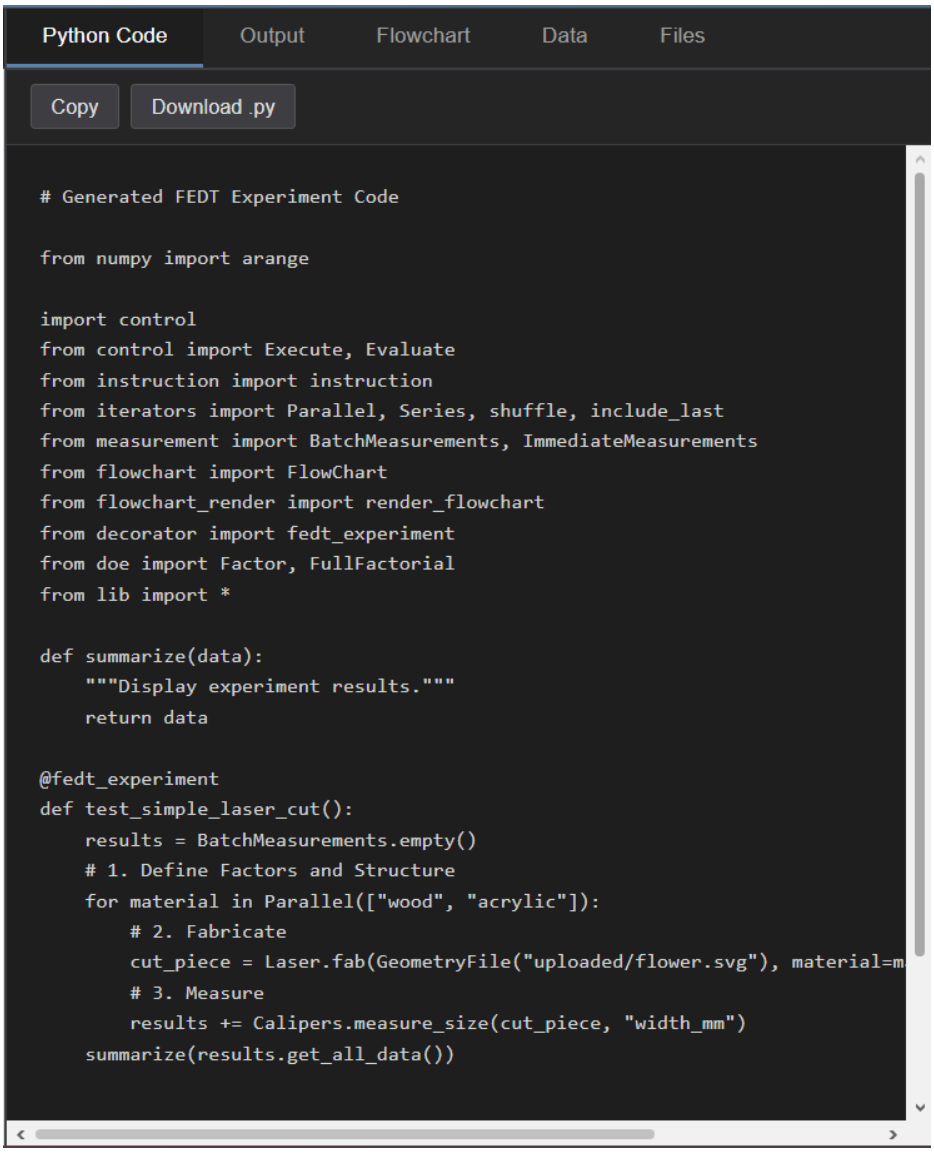


Figure 5: Generated Python code for a simple laser cutting experiment is shown in the Code panel. The code includes FEDT imports, a `summarize()` helper function, the `@fedt_experiment` decorator, a `for` loop iterating over materials (`Parallel(["wood", "acrylic"])`), laser fabrication with `Laser.fab()`, and measurement collection with `Calipers.measure_size()`. The generated code is syntactically complete and can be executed by the backend.

5.5 AST-Based Static Analysis

The backend performs Abstract Syntax Tree (AST) analysis to extract experiment structure from generated Python code without running fabrication steps. This static analysis lets the interface show a design table before hardware execution. In FEDT's original workflow, these tables are produced after execution.

The AST parser (`_extract_design_structure()` and `_extract_measurement_structure()`)

uses Python's built-in `ast` module to parse generated code into an AST, then walks the tree to identify node patterns for factor definitions, factorial designs, loop structures, and measurement operations.

Design table extraction. The parser analyzes nested `for` loops generated by `define_factor` blocks. Each `define_factor` block produces a loop of the form `for variable in Parallel([...])` or `for variable in Series([...])`, and nested factors produce nested loops. The parser extracts the design table structure as follows:

1. Locate `for` loops in the experiment function body. Each loop corresponds to one experimental factor.
2. Extract the loop variable name (e.g., `material`, `infill_density`) which becomes a column name in the design table.
3. Extract the iteration expression inside the `Parallel()` or `Series()` wrapper. This can be a list literal (`["PLA", "ABS"]`), a `range()` call (`range(5)`), an `arange()` expression (`arange(0, 100+include_last, 10)`), or a variable reference to a pre-defined list.
4. Evaluate each iteration expression using the built-in expression evaluator (described below) to produce the list of factor levels.
5. For nested loops, compute the Cartesian product of all factor levels using `itertools.product()`, producing one row per combination with a zero-indexed Label column.

For example, a two-factor experiment with `for material in Parallel(["PLA", "ABS"])` nested with `for infill in Series([20, 50, 80])` produces a design table with columns `Label`, `material`, and `infill`, containing $2 \times 3 = 6$ rows representing all combinations.

Expression evaluation. The AST parser includes an expression evaluator (`_eval_expr_to_number()`) for arithmetic expressions inside factor level definitions, such as `arange(0, 500+include_last, 10)` in the G-ID experiment. The evaluator recursively processes `ast.BinOp` nodes for arithmetic, `ast.Constant` nodes for numeric literals and `ast.Name` nodes for known variable references such as `include_last`.

The evaluator is intentionally limited. It does not handle file I/O, database queries, or custom function calls that require runtime execution. Factor definitions loaded from CSV files, for example through `pd.read_csv()`, therefore cannot be previewed statically.

Measurement table extraction. The parser walks the AST to find measurement variable initializations (`measurements = BatchMeasurements.empty()`) and measurement accumulation operations (`measurements += Calipers.measure_size(...)`). For each `ast.AugAssign` node with a `+=` operator, it extracts the measurement label and unit from the function arguments to construct column headers such as `"x_axis (mm)"`.

The parser computes the number of table rows by analyzing loop dimensions. It locates `for` loops, evaluates their iteration expressions with the same expression evaluator, and multiplies the counts to

determine the total number of runs. For a triple-nested experiment with 8, 6, and 51 loop iterations, the parser computes $8 \times 6 \times 51 = 2,448$ rows.

In Evaluate mode, the measurement table is generated with empty cells, providing a template of the data schema before measurements are collected.

5.6 Modes of Operation and SSE Streaming

The web interface supports two modes that mirror FEDT's distinction between planning and execution.

Evaluate mode. In Evaluate mode, the backend executes the generated code with device execution disabled (`control.MODE = Evaluate()`). The experiment runs far enough to resolve the factor structure and generate a flowchart, but it does not invoke fabrication devices or prompt for manual actions. When using the interactive execution endpoint, the code runs in a background thread to support SSE streaming; when using synchronous endpoints, it runs directly in the request handler.

Evaluate mode uses a hybrid approach: FEDT execution generates the flowchart, while AST analysis extracts the design and measurement table structures. This combines FEDT's existing planning support with the web interface's static previews.

Execute mode. In Execute mode, the backend launches an interactive session using Server-Sent Events (SSE) to stream execution progress to the frontend. SSE provides one-way server-to-client communication over HTTP, allowing the backend to push events without client polling.

When the backend receives an Execute mode request, it:

1. Creates a session object with a thread-safe event queue (`queue.Queue`) for communication between the execution thread and request handlers
2. Spawns a background thread that dynamically loads the generated Python code as a module and executes it with `control.MODE = Execute()`
3. Patches Python's `input()` built-in to intercept FEDT's `instruction()` prompts
4. Opens an SSE stream endpoint that yields events from the queue as they arrive

The patched `input()` function pushes an `input_required` event to the queue and blocks the execution thread until the frontend sends a response. When the user confirms the instruction in the modal dialog, the frontend sends a POST request to `/respond/<session_id>`, which unblocks the execution thread and allows FEDT to continue.

The SSE stream uses the following event types:

- `output`: progress messages from FEDT's console output
- `input_required`: Instruction prompts that require user confirmation
- `measurement_structure`: Design and measurement tables sent early in Execute mode

- `complete`: Final event with flowchart, CSV data, and generated files
- `error`: Execution failures or user abort signals

Using SSE allows the frontend to receive progress updates while the backend continues running the experiment. It also lets execution pause when FEDT asks the user to complete a manual step. Figure 6 shows an instruction prompt during Execute mode.

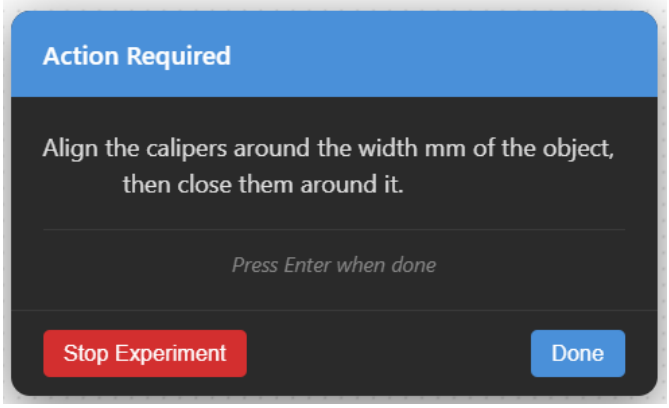


Figure 6: Interactive execution modal during Execute mode. The system streams an `input_required` event via SSE when FEDT calls `input()`, pausing the background execution thread and displaying measurement instructions. The user responds by clicking "Done" (or "Stop Experiment" to abort), which unblocks the thread and allows FEDT to continue.

The execution prompts shown here are part of a longer guided execution workflow. A complete walkthrough of the modal sequence and final measurement table is provided in Appendix A.2.

5.7 Data Panel and File Management

The Data panel displays the experiment design table and the measurement table. Both use a shared, zero-indexed Label column to identify experimental objects consistently with FEDT's object tracking. The design table is read-only, while measurement values can be collected during execution or entered later through batch measurement files.

In Evaluate mode, the Data panel shows the planned experiment structure extracted through static analysis. In Execute mode, it shows the current design structure and the measurement data collected for that run. Figure 7 shows the complete interface in Evaluate mode, with the Blockly workspace on the left and the Data panel on the right.

A step-by-step walkthrough of a user building a similar experiment with the iteration scaffold is provided in Appendix A.1.

File management is divided into two parts:

Geometry files. The Geometry Files panel handles uploads of SVG and STL files referenced by fabrication blocks. Uploaded files are stored in `uploaded_geometry/` and made available to FEDT

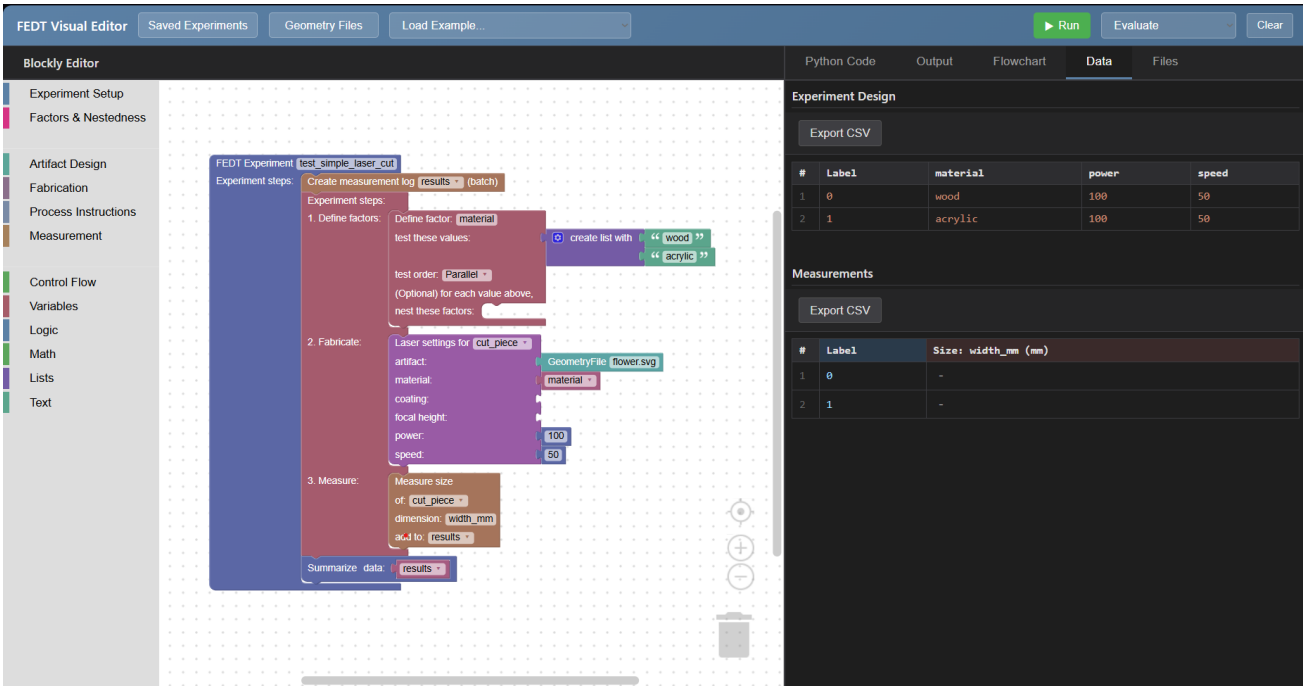


Figure 7: Complete web interface in Evaluate mode for a simple laser cutting experiment. **Left:** Blockly editor with the iteration scaffold, material factor, fabrication block, and measurement block. **Right:** Data panel showing the Experiment Design table and Measurements template generated through AST-based static analysis.

during experiment execution. Geometry blocks such as `geometry_file` and `geometry_file_dynamic` reference these files, and the code generator translates them to `GeometryFile("filename.stl")` calls.

Output files. The Output Files panel displays artifacts produced during evaluation or execution, including flowcharts, measurement CSV files, and key files that map object IDs to experimental conditions. The backend detects newly generated files after each run and parses CSV outputs so that their contents can be shown in the Data panel.

5.8 Saved Experiments and Workspace Persistence

The web interface supports saving and loading experiment configurations. When an experiment is saved, the system stores the Blockly workspace XML, which encodes the block structure, connections, field values, and variable names, together with the experiment name and a timestamp. Saved experiments are stored in the browser's local storage as a JSON array, with each experiment represented as a JSON object containing the XML string and metadata.

The Saved Experiments panel lists saved workspaces with load and delete options. Loading an experiment restores the Blockly workspace by parsing the stored XML and reconstructing the block tree.

Workspace XML preserves information that is not present in the generated Python code, such as block layout and visual connections. The XML is used to reopen and edit the Blockly workspace, while the generated Python code is used for inspection and FEDT execution.

6 Validation and Results

The validation used 22 implemented experiments drawn from FEDT demonstration examples, including `demo_studies.py`, and published HCI fabrication papers. The experiment set was selected as an intentional coverage sample rather than as an exhaustive or statistically representative sample. It was chosen to cover the main patterns relevant to the interface: simple factorial designs, nested designs, parametric geometry, device-specific abstractions, manual steps, and cases that stress the three-section scaffold. For some difficult FEDT examples, only one representative experiment was implemented because additional variants depended on proprietary functions, device-specific APIs, or custom blocks that would mainly test additional engineering work rather than a new structural pattern.

This validation is a technical coverage study. It tests what the interface can represent, what it can translate into executable FEDT Python code, and where the approach breaks down. It does not claim full FEDT API coverage or complete equivalence with every original protocol. Instead, it distinguishes between direct scaffold support, minor structural differences, simplified or manual workarounds, and cases that require imperative blocks outside the scaffold.

A recurring issue is the placement of artifact and geometry definitions. In the three-section scaffold, geometry definitions placed in the Fabricate section are generated inside the active factor-loop structure. Handwritten FEDT code, by contrast, can place such definitions between or outside nested loops. This affects code organization and, in some cases, how directly an original FEDT protocol maps to the scaffold. Table 2 summarizes the implemented experiments and reports how each one fits the scaffold.

6.1 Validation Methodology

The validation used three criteria: whether the experiment structure is visible in blocks, tables, and flowcharts; whether the generated FEDT code runs without manual edits; and whether the generated outputs match the implemented representation. These criteria were interpreted according to the support level in Table 2. For directly supported experiments, the validation checks close correspondence between the original FEDT structure and the Blockly-generated implementation. For workaround cases, it checks whether the generated program faithfully executes the simplified or manual representation chosen for the web interface.

An experiment was considered successfully supported when the generated program ran without manual edits, the flowchart reflected the intended block structure, and the generated CSV outputs matched the expected run count and measurement schema for the implemented representation.

Source	Experiment	F.	Runs	Pattern	Scaffold support
Canonical Experiments (demo_studies.py)					
demo_studies [16]	Print Shrinkage	2	15	3D print, batch measurements	Direct scaffold fit.
demo_studies [16]	Force at Break	2	6	`arange()`, parametric geometry	Supported; geometry placement differs.
demo_studies [16]	Paint Layers	0	variable	Adaptive while loop	Partial/workaround; adaptive termination.
demo_studies [16]	User Assembly Time	3	variable	Shuffle, conditional order	Outside scaffold; raw imperative blocks.
Published Papers: Simpler Experiments					
AirLogic [17]	OR Bend Radius	2	20	Parallel/Series, airflow	Direct scaffold fit; manual compressor prompt.
AirLogic [17]	Gate Types	2	36	Pneumatic widgets, massflow	Direct scaffold fit; setup via instructions.
CircWood [8]	Grain Direction	1	2	Wood orientation, resistance	Direct scaffold fit.
CircWood [8]	Test Materials	2	20	Wood species, coatings	Direct scaffold fit.
CircWood [8]	Focal Height	1	6	Numeric laser focus levels	Direct scaffold fit.
ElectriPop [5]	Physical Inflation	1	2	Inflation timing	Supported with manual setup prompt.
BlowFab [23]	Incision Number	1	4	Manual cutting/inflation	Supported with instruction blocks.
BlowFab [23]	Interplate Distance	1	3	Geometry parameter adjustment	Partial/workaround; no full geometry mutation.
FabHydro [24]	Pneumatic vs. Hydraulic	1	2	Actuation type, displacement	Direct scaffold fit.
FabHydro [24]	Resin Types	1	3	Material, size/time measurements	Direct scaffold fit.
FabHydro [24]	Min Wall Thickness	1	3	Floating `arange()`, pressure	Direct scaffold fit.
Published Papers: Complex Experiments					
Aline [22]	Cross Section Ratios	2	9	Parametric profile geometry	Direct scaffold fit.
Metamaterials [20]	Density & Materials	2	12	Geometry scaling pipeline	Supported with simplified pipeline.
KnitPicking [7]	Texture Comparison	1	10	Custom knitting block	Supported with simplified device wrapper.
G-ID [4]	Materials & Lighting	3	2,448	Triple-nested factorial design	Supported; dynamic prompt works, but indexed configuration parameters are omitted.
SPEERLoom [21]	Weaving Quality	1	5	Class-based loom iteration	Supported with specialized class-reference blocks.
Trilaterate [18]	Placement Response	2	84	Participant/electrode nesting	Supported; includes warmup with `random_choice`; no Latin-square counterbalancing.
DesignMould-Grow [6]	Geometric Features	2	6	Dynamic files, temporal waits	Supported with dynamic files and wait instructions.

Table 2: Overview of 22 implemented FEDT experiments grouped by source and complexity. **F.** reports the number of main experimental factors in the implemented representation, and **Runs** reports the expected number of generated runs where this is fixed. The final column describes how each experiment fits the three-section scaffold. "Direct scaffold fit" means that the experiment maps cleanly to Define Factors, Fabricate, and Measure. "Supported" indicates a minor extension or simplification. "Partial/workaround" marks cases where the main structure is represented but important protocol details remain outside the scaffold.

6.1.1 Code Compilation Verification

Each implemented experiment was built in Blockly, translated to Python, and run in the FEDT environment without manual edits to the generated code. Successful execution indicates that the generated program is syntactically valid, includes the required imports, and uses the tested FEDT APIs in a way

accepted by the runtime.

6.1.2 Flowchart Structure Comparison

FEDT generates a flowchart showing the sequence of fabrication and measurement operations. For directly supported experiments, the comparison checks whether the control-flow structure matches the corresponding handwritten FEDT implementation. For partial or workaround cases, the comparison is used more cautiously: it identifies whether the generated flowchart reflects the intended simplified representation and where it diverges from the original protocol.

6.1.3 CSV Output Verification

During Execute mode, FEDT produces CSV artifacts such as key files that record factor combinations and measurement files that store collected data. The web interface additionally generates design and measurement table previews before execution through AST-based static analysis.

For directly supported experiments, comparing these files checks whether the design space and measurement schema correspond to the original FEDT implementation. For workaround cases, the CSV files verify the generated representation rather than full equivalence to the original protocol.

6.2 Numerical Validation Results

6.2.1 Code Compilation Success Rate

All 22 implemented experiments generated Python programs that ran in the FEDT environment without manual edits. This indicates that, for the implemented experiment set, the code generator produced syntactically valid Python with the required FEDT imports, function decorators, indentation, and expression structure. However, successful execution should not be interpreted as full semantic equivalence with every original FEDT experiment. As shown in Table 2, several experiments are supported through simplifications, manual instruction blocks, or specialized device-specific blocks.

6.2.2 FEDT-Specific Pattern Preservation

In the tested cases, the generated code preserved the main FEDT-specific patterns used in the validation set.

`include_last`: The `arange` block correctly generated endpoint-inclusive expressions such as `arange(0, 100+include_last, 10)`. The resulting design table contained 11 values, from 0 to 100, confirming that the endpoint was included.

`Parallel()` and `Series()`: Factor blocks with parallel iteration generated `Parallel([...])`, while factor blocks with sequential iteration generated `Series([...])`. This preserved FEDT's distinc-

tion between order-independent and sequential execution.

`BatchMeasurements`: Measurement blocks generated the expected += accumulation pattern, tracked which measurement variable each block updated, and called `summarize()` with the appropriate collection method, such as `.get_all_data()` for batch measurements.

6.2.3 Flowchart Structure Comparison

The flowchart comparison focused on node types and nesting structure: Header nodes for loop iterations, fabrication-related instruction nodes, and measurement-related instruction nodes. For directly supported experiments, the validation checked whether the generated flowchart matched the intended FEDT control-flow structure. For workaround cases, the flowchart was used to inspect how the simplified representation differed from the original protocol.

Print Shrinkage: The flowchart contains Header nodes for the factor iterations, Instruction nodes for `Printer.fab()`, and Instruction nodes for `Calipers.measure_size()`. The node placement matches the original experiment structure.

Metamaterials: The flowchart preserves the triple-nested hierarchy of `ruffle_design`, `material`, and `load_direction`, followed by fabrication, post-processing, and compression-measurement nodes.

Force at Break: The flowchart shows Header nodes for rectangle length and material, followed by `Laser.fab()` and `Scale.measure_weight()` nodes. It also makes the geometry-placement difference visible: the Blockly-generated version creates geometry inside both active factor loops, while the original FEDT implementation creates it once per rectangle length before iterating over materials.

6.2.4 CSV Output Verification

CSV output verification focused on artifacts produced during Execute mode, including key files that map experimental objects to factor conditions and measurement CSV files that store collected data. For directly supported scaffold cases, these files preserved the expected factor conditions and measurement columns. For workaround cases, the CSV comparison verifies the simplified representation implemented in the web interface rather than claiming complete equivalence to every detail of the original protocol.

Factor-condition outputs: The execution-time key files contained the expected factor values for the implemented representation. For declarative factorial designs, the number of recorded conditions matched the expected Cartesian product of the declared factors.

Measurement outputs: The measurement CSV files contained the expected column schemas for the generated measurement blocks. For example, the Print Shrinkage experiment generates columns for `x_axis (mm)`, `y_axis (mm)`, and `z_axis (mm)`, matching the three `measure_size` blocks in the Measure section, as shown in Figure 9.

6.2.5 Design Table Preview Accuracy

The AST-based static analysis described in Section 5.5 successfully predicted the table structures for the factor patterns used in the validation set. For supported patterns, the design tables extracted in Evaluate mode matched the corresponding execution-time CSV outputs produced during Execute mode.

Run Count Accuracy: The number of rows in the design table matched the number of run contexts implied by the generated nested loops. For example, the G-ID experiment contains 8 filament values, 6 configuration IDs, and 51 light-intensity values, so the AST analyzer correctly expands the design to $8 \times 6 \times 51 = 2,448$ rows.

Factor Value Accuracy: Each row in the design table corresponds to one combination of factor values from the generated loop structure. For the tested cases, the extracted factor values matched the values used in the generated `Parallel` and `Series` loops.

Measurement Schema Prediction: The measurement table preview contained the expected columns for the generated measurement blocks. The system inferred column names from measurement block labels and distinguished between batch and immediate measurement structures.

The parser handled all expression patterns used in the validation set, including `range(1, 6)`, `arange(0, 100+include_last, 10)`, literal lists such as `[10, 20, 30]`, and arithmetic expressions such as `arange(0, 50×2, 10)`.

6.3 Standard Factorial Designs

The block-based interface fits standard factorial experiments where factors are defined upfront, the same fabrication procedure is applied across factor combinations, and measurements are collected for each run. These experiments match the assumptions of the three-section scaffold: Define Factors describes the design space, Fabricate describes artifact production, and Measure describes the data to be collected.

The Print Shrinkage experiment is a representative example of this pattern. It crosses three infill patterns with five repetitions, producing 15 planned runs with the same fabrication and measurement protocol for each run. Figure 8 shows the experiment implemented in Blockly. The iteration block contains two nested factors: `infill_pattern` with three categorical levels (honeycomb, triangles, grid) and `repetition` using `range(5)` to generate five trials. The Fabricate section uses `printer_fab` with the `infill_pattern` variable to control the print configuration, and the Measure section collects three dimensional measurements for each test cube.

Figure 9 shows the resulting design and measurement tables. The design table exposes the Cartesian product of factor levels as 15 rows, while the measurement table shows the expected data schema. This makes the planned run structure visible before execution instead of requiring it to be inferred from nested Python loops.

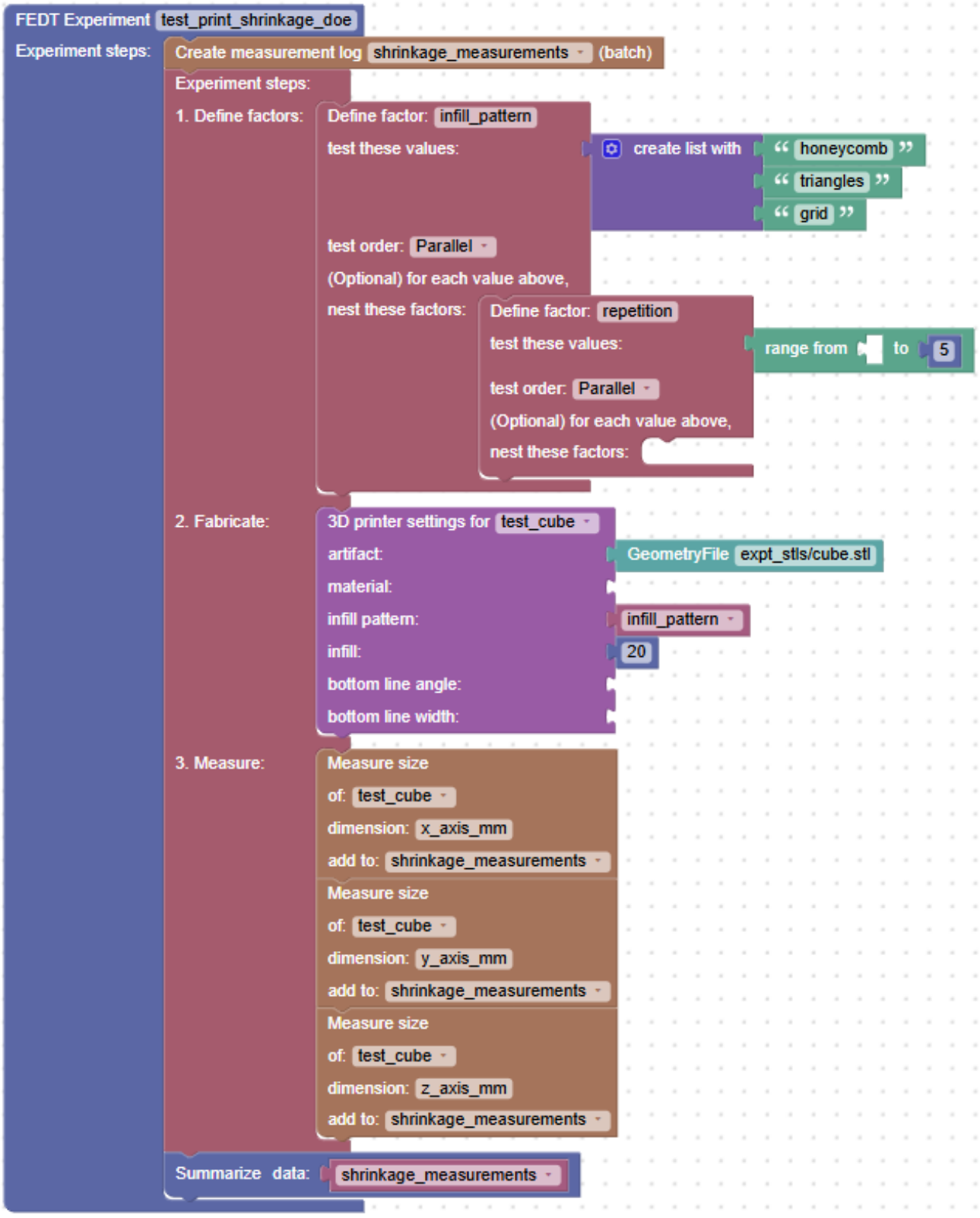


Figure 8: Print Shrinkage experiment in Blockly demonstrating standard 2-factor nested design. The three-section scaffold (Define Factors, Fabricate, Measure) makes the experimental structure explicit: 3 infill patterns × 5 repetitions = 15 planned runs.

Experiment Design:

Label	infill_pattern	repetition	infill_density
0	honeycomb	0	20
1	honeycomb	1	20
...			
6	triangle	1	20
7	triangle	2	20
...			
13	grid	3	20
14	grid	4	20

Measurements:

Label	x_axis (mm)	y_axis (mm)	z_axis (mm)
0	-	-	-
1	-	-	-
...			
6	-	-	-
7	-	-	-
...			
13	-	-	-
14	-	-	-

Figure 9: Design table and measurement table extracted from the Print Shrinkage experiment in Evaluate mode. The design table (left) shows the 15 planned factor combinations, while the measurement template (right) shows the expected data schema before values are collected.

Parametric geometry also fits the scaffold when geometry parameters are derived directly from declared factors. The Force at Break experiment varies rectangle length (50, 75, 100 mm) and material (wood, acrylic), producing six combinations. The factor variables are used to control geometry generation and fabrication settings.

This case also reveals a structural difference between the block interface and handwritten FEDT code. In the original FEDT implementation, the SVG geometry is created inside the `rect_length` loop but before the `material` loop, allowing one geometry object to be reused across both material values. In the Blockly-generated implementation, geometry definitions appear in the Fabricate section and are therefore generated inside all active factor loops. This does not change the generated factor combinations or measurement schema, but it changes the execution structure and may duplicate geometry creation compared to the handwritten FEDT version. Figure 10 shows this difference in code form, while Figure 11 shows how the same difference appears in the generated flowcharts.

The corresponding flowchart comparison is shown in Figure 11.

The AirLogic [17] OR Bend Radius experiment demonstrates how `Parallel` and `Series` iteration can be combined within the same factorial structure. The experiment crosses five bend-radius values with four input mass flow levels, producing 20 measurement contexts. The outer factor, `bend_radius`, uses `Parallel`, while the nested factor, `input_massflow`, uses `Series`, indicating that mass flow settings should be tested sequentially for each bend-radius condition. Figure 12 shows the corresponding block representation.

These examples show where the scaffold is most useful. It separates factor definition, fabrication, and measurement as conceptual phases, even though the generated FEDT program executes them through nested control flow. Compared with handwritten FEDT code, this trades some freedom over code placement for a regular structure in which factors appear in Define Factors, artifact production in Fabricate, and data collection in Measure.

Force at Break: Blockly-generated code

```
def test_force_at_break_doe():
    breakage_points = BatchMeasurements.empty()
    # 1. Define Factors and Structure
    for rect_length in Parallel(arange(50, 100+
        include_last, 25)):
        for material in Parallel(["wood", "
            acrylic"]):
            # 2. Fabricate
            test_piece = Laser.fab(SvgEditor.
                build_geometry(SvgEditor.
                draw_rectangle, CAD_vars={'width'
                : rect_length, 'height': 20}),
                material=material, power=100,
                speed=50)
            # 3. Measure
            instruction("Place object with 1cm
                overlapping shelf at each end,
                remainder suspended")
            instruction("Place weights on object
                until it breaks")
            breakage_points += Scale.
                measure_weight(test_piece, "
                total_weight_at_break")
    summarize(breakage_points.get_all_data())
```

Force at Break: Original handwritten FEDT code

```
def test_force_at_break():
    breakage_points = BatchMeasurements.empty()
    svgs = []
    fabbed_objects = []
    for rect_length in Parallel(arange(50,100+
        include_last,25)):
        svg = SvgEditor.build_geometry(draw_rect,
            CAD_vars={'rect_length':rect_length
            })
        svgs.append(svg)
        for material in Parallel(['wood','acrylic
            ']):
            fabbed_object = Laser.fab(svg,
                material=material)
            fabbed_objects.append(fabbed_object)
            instruction("place the object with 1
                cm overlapping a shelf at each
                end and the remainder suspended")
            instruction("place weights on the
                object until it breaks")
            breakage_points += Scale.
                measure_weight(fabbed_object,"
                total weight placed at break")

    summarize(breakage_points.get_all_data())
```

Figure 10: Code comparison for the Force at Break experiment. **Left:** The Blockly-generated implementation creates geometry inside both active factor loops. **Right:** The original FEDT implementation creates the SVG once per rectangle length and reuses it across both material values.

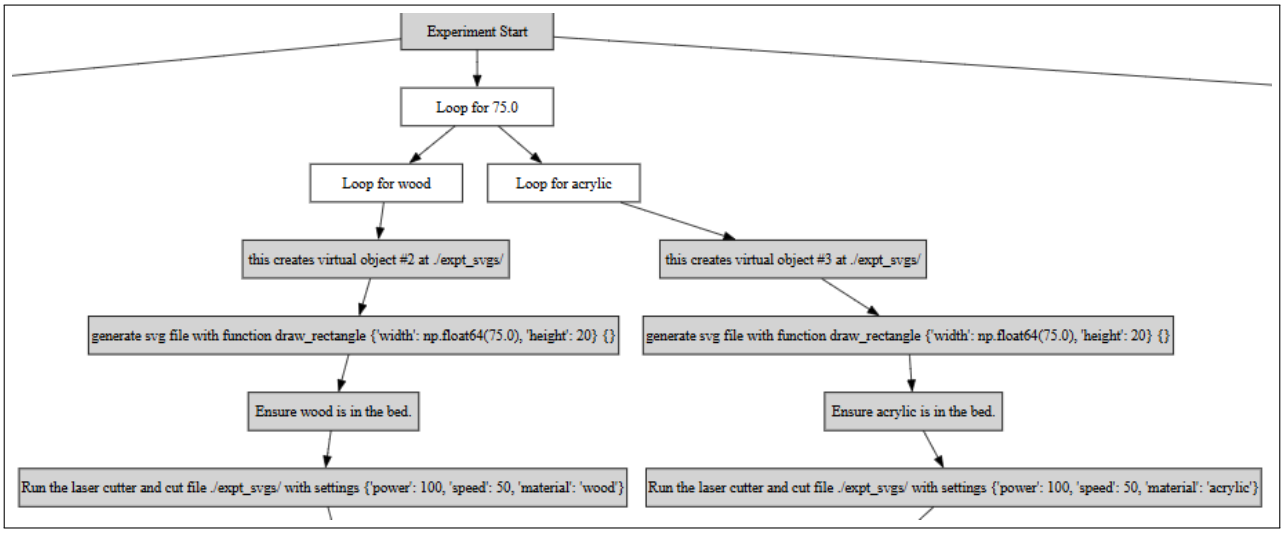
6.3.1 Complex Multi-Factor Experiments

The G-ID experiment was included to test whether the interface and AST analyzer could handle a large triple-nested factorial design. The experiment studies 3D-printed identification markers through three factors: filament color (8 levels), configuration ID (6 levels), and light intensity (51 levels via `arange(0, 500+include_last, 10)`), producing $8 \times 6 \times 51 = 2,448$ planned combinations. Each combination requires setting the lighting condition before capturing a photo with `Camera.take_picture`.

Figure 13 shows the triple-nested factor structure in Blockly. The generated Python code in Figure 14 preserves this nested loop structure and produces the expected run count. The design table is not shown in full because it contains 2,448 rows, but the row count is correctly computed by the AST analyzer.

The G-ID case also shows the difference between preserving factorial structure and preserving every parameter dependency in the original FEDT implementation. The original code uses `config_id` to index into pre-computed arrays for bottom-line angles and widths. The current Blockly implementation preserves the factors, loop structure, run count, lighting prompt, and measurement protocol but omits these indexed fabrication parameters because `lists_getIndex` is not yet translated into Python indexing expressions. This case therefore produced two findings: the AST analyzer can now handle `arange()` expressions with arithmetic terms at this scale, while list-indexed fabrication parameters remain outside the current generator coverage. This limitation is discussed again in Section 6.5.3.

Force at Break: Blockly Flowchart (rect_length = 75.0)



Force at Break: Original FEDT Flowchart (rect_length = 75.0)

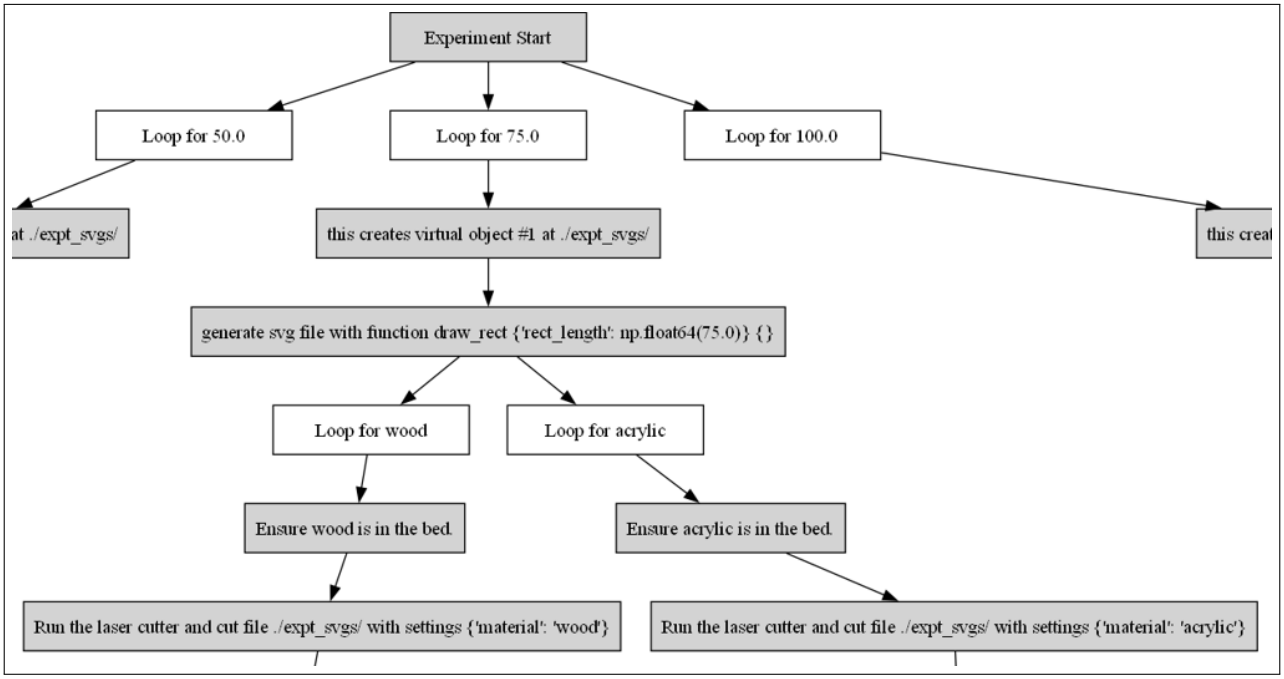


Figure 11: Flowchart comparison for the Force at Break experiment. The Blockly-generated version places SVG creation inside both nested loops, while the original FEDT version places SVG creation between the rectangle-length and material loops. This reflects the scaffold's fixed placement of geometry definitions in the Fabricate section.

6.4 Measurement Abstractions and Device Extensions

This subsection examines whether the block vocabulary can express the measurement operations, manual interventions, and device-specific fabrication abstractions used in the implemented experiments.

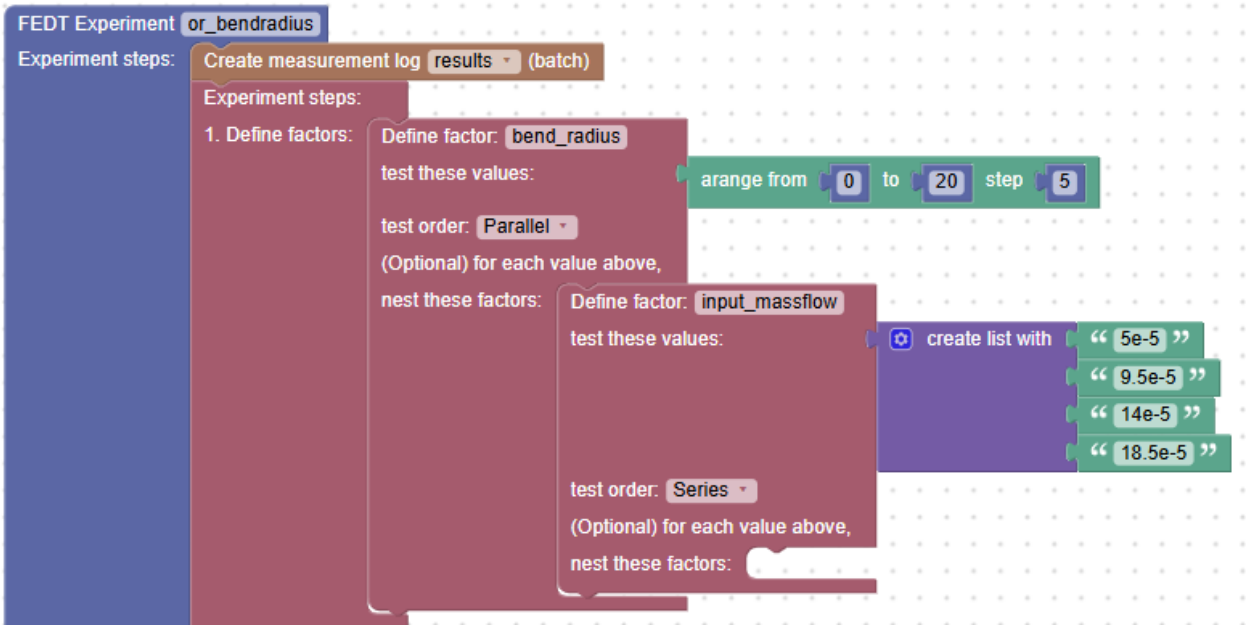


Figure 12: AirLogic OR Bend Radius experiment demonstrating mixed `Parallel`/`Series` nesting in Blockly. The outer factor (`bend_radius`) is marked as `Parallel`, while the nested factor (`input_massflow`) uses `Series`, producing 20 planned measurement contexts.

6.4.1 Standard Measurement Devices

The implemented experiments used measurement blocks for resistance, size, time, pressure, photos, and generic outcome observations. The CircWood [8] experiments use the `measure_resistance` block to assess laser-engraved conductive traces on wooden substrates, mapping this operation to FEDT's Multimeter abstraction. The implemented cases include a single-factor grain-direction experiment, a material/coating experiment with 20 combinations, and a focal-height experiment with six numeric levels. Figure 15 shows the material/coating case.

The FabHydro [24] cases show that multiple measurement modalities can be represented within the same scaffold. The pneumatic-versus-hydraulic experiment uses `measure_size` to quantify displacement, while the resin-types experiment combines `measure_size` and `measure_time` in the same Measure section. The minimum-wall-thickness experiment uses `measure_pressure` and also tests floating-point factor values through an `arange()` sequence. Figure 16 shows the resin-types case, where one factor produces three runs and two measurement operations are collected for each run.

For outcomes that do not map directly to a standard device block, the interface uses more generic measurement abstractions. The Aline [22] and Metamaterials [20] implementations use `observe_outcome` to record quantities such as actuation angle or height at rest. The Trilaterate [18] implementation uses a human-entered measurement prompt to record capacitance values across participant and electrode combinations. These cases use the Measure section for both device-specific measurements and manually entered outcomes.

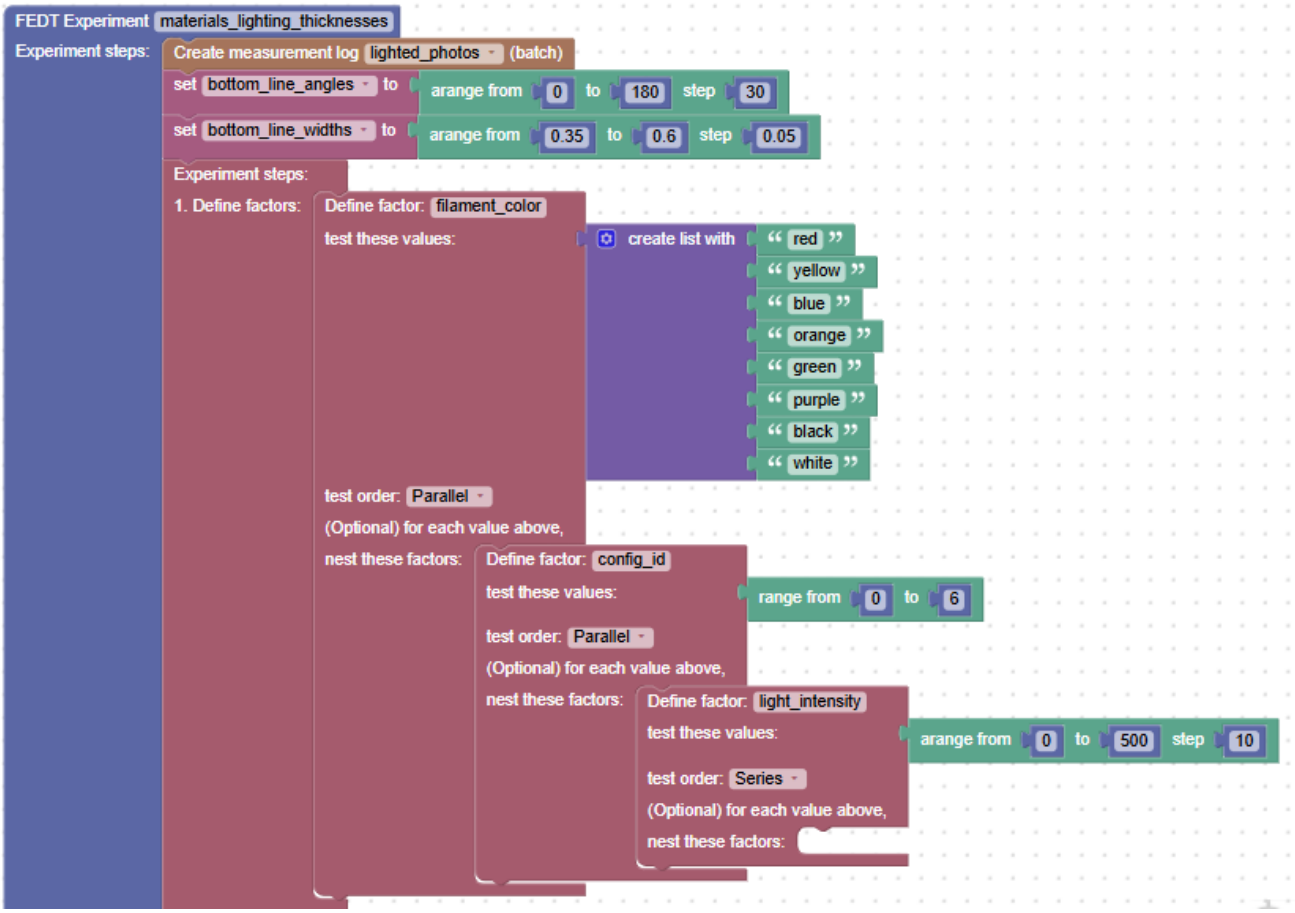


Figure 13: G-ID experiment demonstrating a triple-nested factorial design with 2,448 planned combinations ($8 \times 6 \times 51$). The visual nesting of `define_factor` blocks matches the conceptual hierarchy of filament, configuration ID, and light intensity.

6.4.2 Manual Intervention and Dynamic Files

Several experiments required manual actions that could not be fully automated through available device blocks. In these cases, instruction and human-post-processing blocks served as a fallback mechanism: they preserved the experimental procedure as part of the generated workflow, even when the physical action remained manual.

The AirLogic [17] experiments combine fabrication and measurement blocks with manual compressor setup. The OR Bend Radius experiment varies bend radius and input massflow, while the Gate Types experiment varies pneumatic widget type and massflow level. In both cases, instruction blocks prompt the user to configure the air compressor or connect the object before airflow is measured. This shows that measurement blocks can be combined with manual setup prompts without requiring a specialized block for every laboratory device.

Figure 17 shows a simpler manual-intervention pattern from BlowFab [23]. The Incision Number experiment varies the number of cuts in an inflatable structure. The Fabricate section uses an instruction block with dynamic text concatenation to embed the current factor value in the prompt. This keeps the

G-ID: Blockly-generated code

```
@fedt_experiment
def materials_lighting_thicknesses():
    lighted_photos = BatchMeasurements.empty()
    bottom_line_angles = arange(0, 180+include_last, 30)
    bottom_line_widths = arange(0.35, 0.6+include_last, 0.05)
    # 1. Define Factors and Structure
    for filament_color in Parallel(["red", "yellow", "blue", "orange", "green", "purple", "black", "
        white"]):
        for config_id in Parallel(range(0, 6)):
            for light_intensity in Series(arange(0, 500+include_last, 10)):
                # 2. Fabricate
                fabbed_object = Printer.fab(GeometryFile("uploaded/keycover.stl"), material=
                    filament_color, infill_pattern="grid", infill_density=20)
                # 3. Measure
                instruction((str("Set light brightness to ") + str(light_intensity)))
                lit_object = fabbed_object
                lighted_photos += Camera.take_picture(lit_object, "bottom")
    summarize(lighted_photos.get_all_data())
```

G-ID: Original handwritten FEDT code

```
@fedt_experiment
def materials_lighting_thicknesses():
    model = GeometryFile("keycover.stl")
    filament_colors = ['red', 'yellow', 'blue', 'orange', 'green', 'purple', 'black', 'white']
    bottom_line_angles = list(arange(0,180+include_last,(180-0)/6))
    bottom_line_widths = list(arange(0.35,0.6+include_last,(.6-.35)/6))
    light_intensities = arange(0,500+include_last,10)
    lighted_photos = BatchMeasurements.empty()
    for color in Parallel(filament_colors):
        for config_id in Parallel(range(len(bottom_line_angles))):
            fabbed_object = Printer.fab(model,
                                        filament_color=color,
                                        bottom_line_angle=bottom_line_angles[config_id],
                                        bottom_line_width=bottom_line_widths[config_id])
            for light_intensity in Series(light_intensities):
                fabbed_object = Human.post_process(fabbed_object,
                                    f"light brightness of {light_intensity}")
                lighted_photos += Camera.take_picture(fabbed_object, "bottom")
    summarize(lighted_photos.get_all_data())
```

Figure 14: Comparison of G-ID experiment implementations showing the same triple-nested factor structure and planned run count (2,448 combinations: 8×6×51). **Top:** Blockly-generated code preserving the filament, configuration, and light-intensity loops, but omitting indexed fabrication parameters. **Bottom:** Original FEDT code using pre-computed arrays (`bottom_line_angles`, `bottom_line_widths`) indexed by `config_id`, together with dynamic instruction text. The comparison shows that the current Blockly implementation preserves the validated factorial structure and measurement protocol, while list-indexed configuration parameters remain outside the current generator coverage.

manual cutting and inflation procedure explicit in the experiment representation.

The same pattern appears in ElectriPop [5], where the system guides manual inflation setup before measuring inflation time, and in the BlowFab Interplate Distance experiment, where geometry adjustment is represented as a manual parameter change rather than as programmatic geometry mutation. These cases are supported as guided procedures, but not as full feature-parity implementations when the original workflows depend on geometry mutation or custom device control.

The DesignMouldGrow [6] experiment adds a different kind of extension: dynamic file selection.

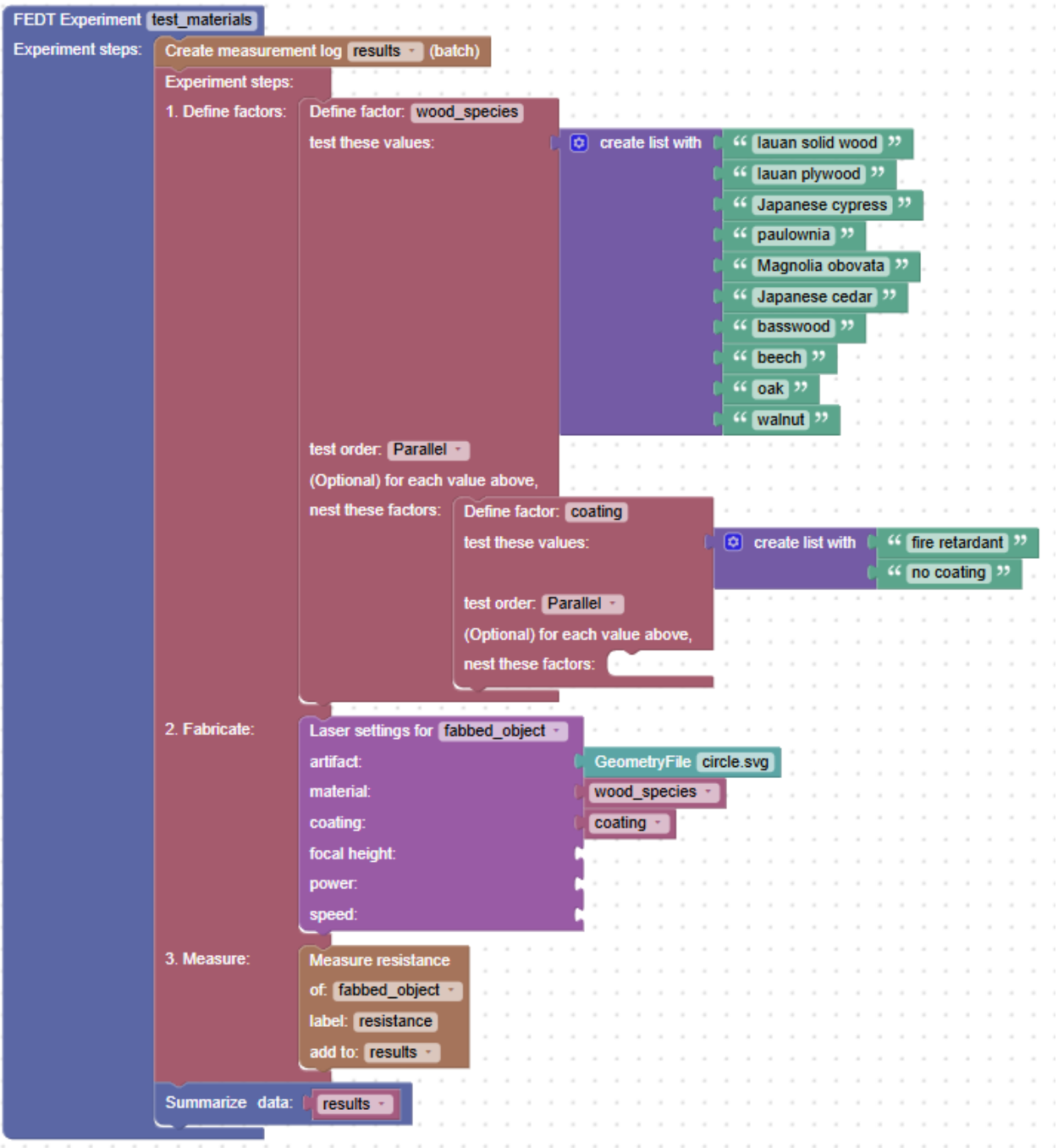


Figure 15: CircWood Test Materials experiment demonstrating the `measure_resistance` block for laser-engraved traces. The experiment crosses 10 wood species with 2 coating types, producing 20 planned runs, and uses the Measure section to record resistance of the laser-engraved traces.

The experiment uses a `geometry_file_dynamic` block so that a factor value can determine which STL file is loaded for a given run. Temporal delays such as growth and drying periods are represented as instruction blocks rather than through FEDT's `Environment.wait_up_to_time()` API. This is sufficient for planning and documentation, but it remains a simplified representation of the original time-based protocol.

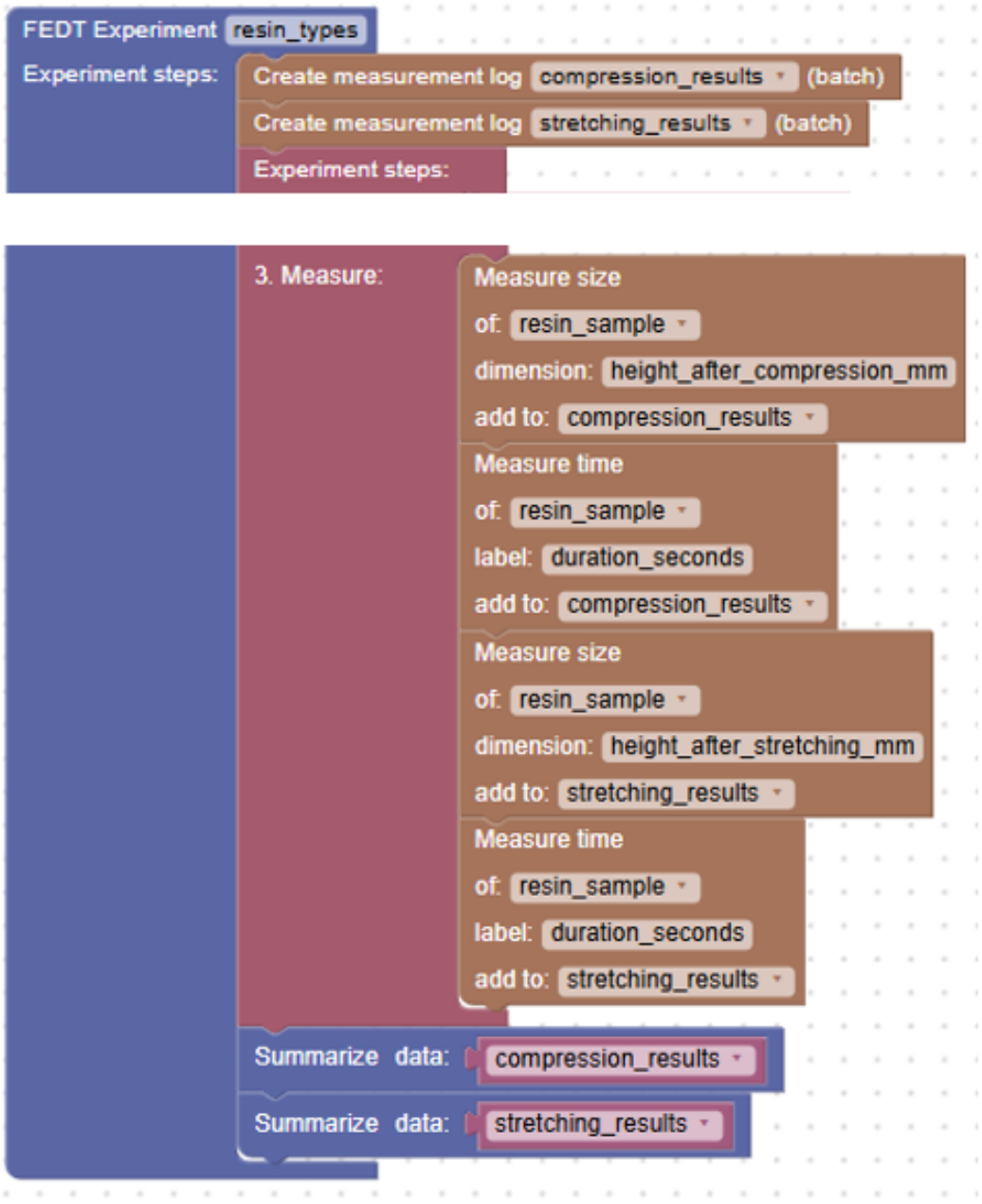


Figure 16: FabHydro Resin Types experiment demonstrating multiple measurement types in a single experiment. The Measure section contains both `measure_size` and `measure_time`, showing that different measurement modalities can coexist within the scaffold.

6.4.3 Textile Fabrication and Class-Based Iteration

The textile experiments required device-specific fabrication blocks and exposed a limit in how the interface represents factor values.

KnitPicking [7] required a new `knitting_machine_knit` block, implemented by analogy with existing fabrication blocks such as `printer_fab` and `laser_fab`. The block accepts a knitting pattern file and represents the fabrication of a textile artifact. Measurements use `observe_outcome` blocks to

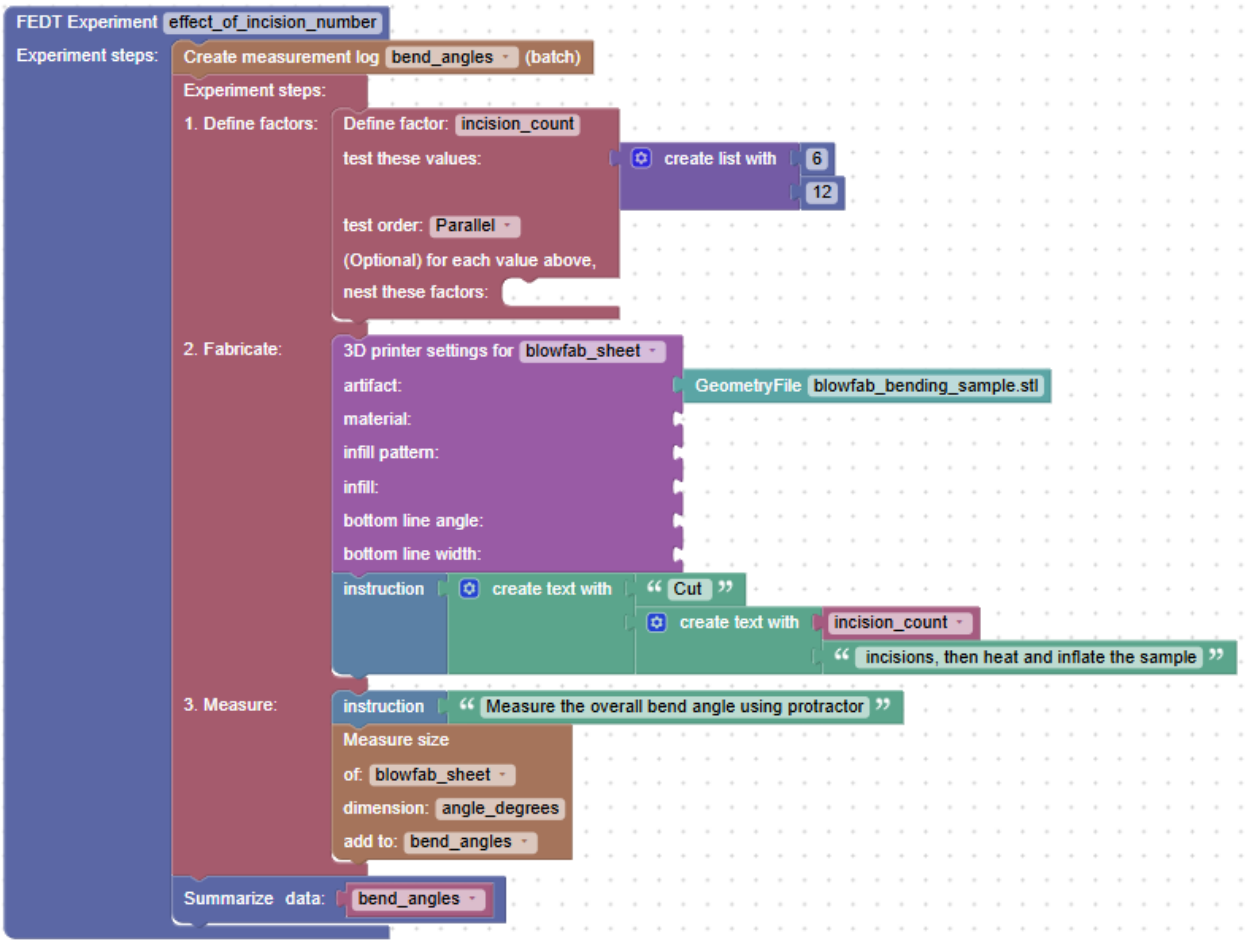


Figure 17: BlowFab Incision Number experiment demonstrating manual intervention through instruction blocks. The `text_join` block embeds the current factor value into dynamic instructions, producing prompts such as ”Cut 6 incisions” or ”Cut 12 incisions”.

record fabric weight and dimensions before and after stretching. Figure 19 shows the complete experiment structure. This case shows that a new fabrication device can be added by defining a block type and extending the code generator, without changing the three-section scaffold.

SPEERLoom [21] was more challenging because the original FEDT implementation iterates over loom classes rather than ordinary data values. The original structure uses a loop such as `for loom in Parallel([SPEERLoom, Jacq3GLoom, AlbaughLoom, AshfordLoom, TC2Loom])`, where each factor level is a class object and fabrication calls `loom.fab(pattern)`. This does not fit the usual block representation of factor levels as strings or numbers.

The implemented solution introduces two specialized blocks: `loom_type_selector`, which outputs a class reference rather than a string, and `loom_fab`, which calls `.fab()` on the selected loom type. The code generator was extended so that loom selector blocks emit Python class names without quotation marks, and so that simplified loom definitions are included only when loom blocks are present. Figure 20 shows this class-based iteration pattern.

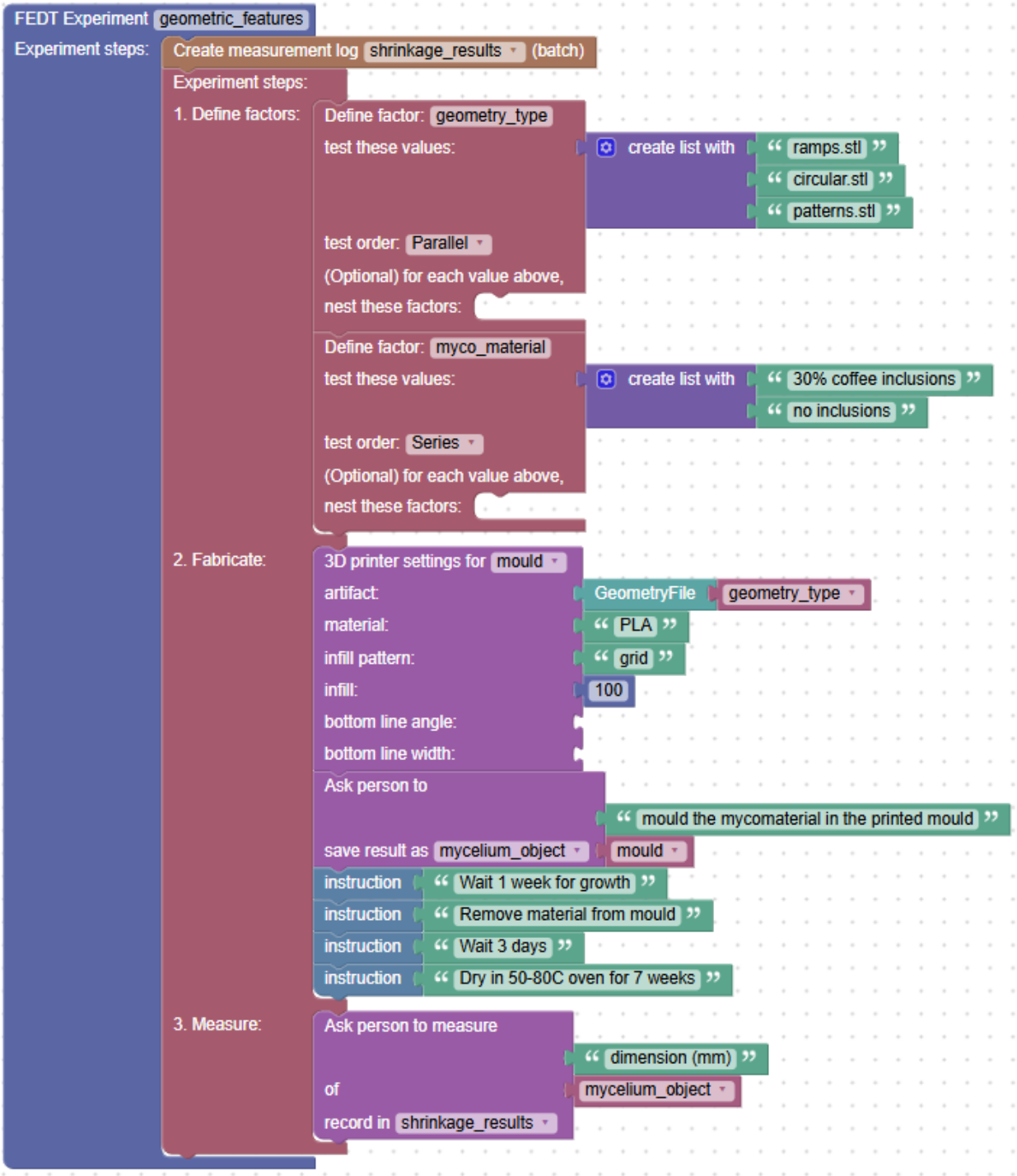


Figure 18: DesignMouldGrow experiment showing `geometry_file_dynamic` for variable filename selection. The `geometry_type` factor determines which STL file is loaded in the Fabricate section, while instruction blocks document long temporal waits such as growth and drying periods.

This solution works for the implemented representation, but it also marks a boundary of the block language. Most factor values in the interface are ordinary data values, while this experiment treats device classes as values. Supporting this pattern required specialized blocks and simplified wrappers rather than a general solution for arbitrary class-based or algorithm-based iteration.

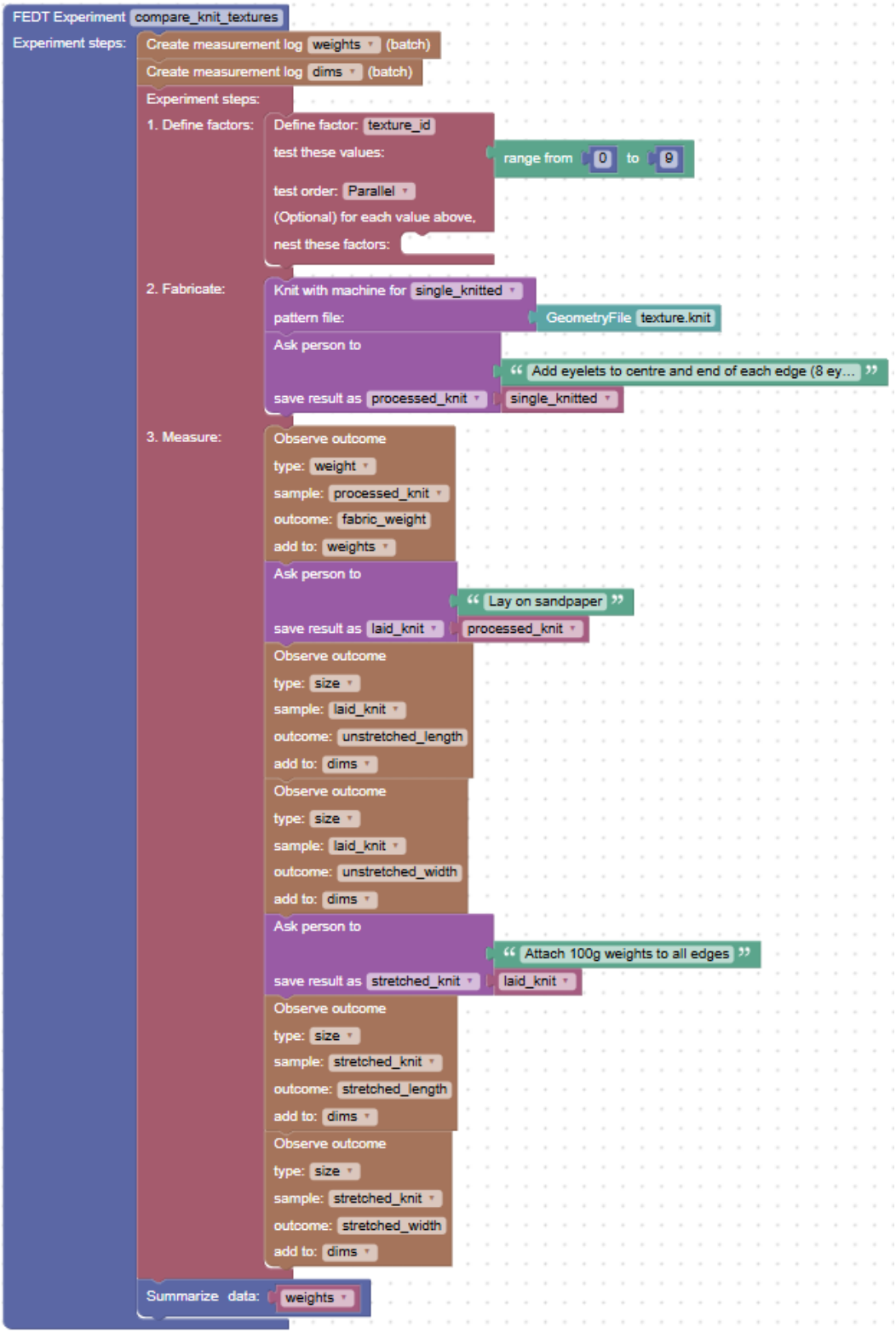


Figure 19: KnitPicking Texture Comparison experiment showing the custom `knitting_machine_knit` fabrication block. The experiment tests 10 knitting pattern variants and records fabric properties using `observe_outcome` blocks.

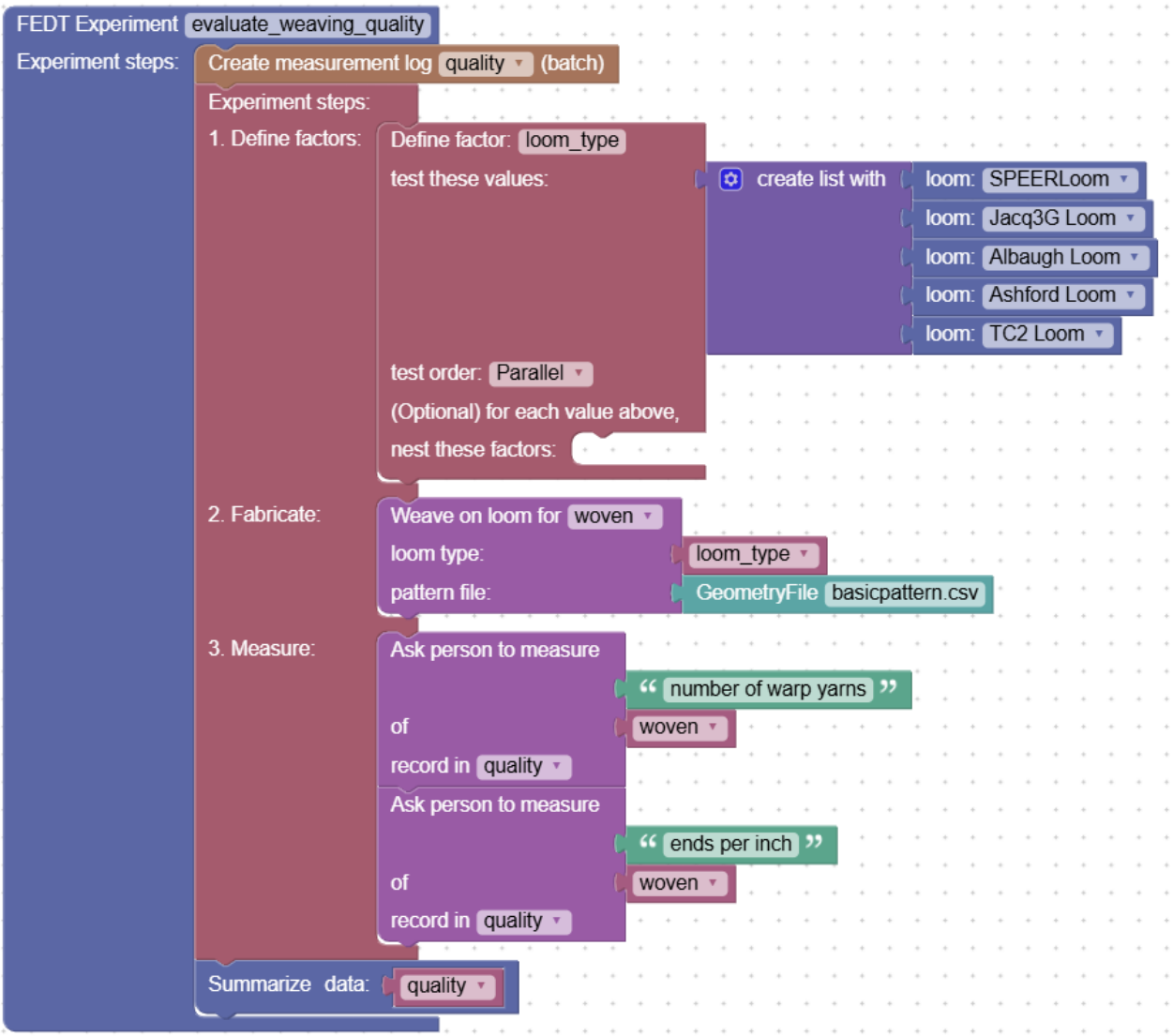


Figure 20: SPEERLoom Weaving Quality experiment demonstrating class-based iteration. The factor definition uses `loom_type_selector` blocks to create a list of loom class references, and the Fabricate section uses `loom_fab` to call `.fab()` on the selected class.

These cases show that the interface can be extended beyond generic fabrication blocks and can document manual laboratory actions when automation is unavailable. They also show that support often depends on simplifying device APIs or adding specialized blocks. The next subsection examines the cases where such extensions were not sufficient and where the scaffold itself became the limiting factor.

6.5 Discovered Limitations

The validation revealed four main boundaries: static-analysis coverage, visual scalability, code-generation coverage, and structural limits in the DOE-based scaffold.

6.5.1 Static Analysis and Preview Boundaries

Design table previews rely on AST parsing to extract factor levels and predict run counts before execution. This works for the expression patterns used in the validation set, including literal lists, `range()`, `arange()`, variable references to predefined lists, and simple arithmetic expressions. However, dynamic factor levels loaded from files, database queries, or user-defined functions cannot be previewed without executing the corresponding code. Conditional design structures would also require runtime evaluation to determine which branches contribute to the actual experiment.

The G-ID experiment exposed this boundary during implementation. Initially, the AST parser could not evaluate `arange()` expressions with arithmetic terms, such as `arange(0, 500+include_last, 10)`, and therefore produced only 48 rows instead of the correct 2,448. Extending the expression evaluator to process `ast.BinOp` nodes resolved this case, allowing the parser to compute 51 light-intensity values from 0 to 500. This issue was fixed, but it illustrates a general constraint: static previews depend on the parser recognizing the expression patterns used in generated experiments.

6.5.2 Large Design Spaces

The G-ID experiment also revealed a visual scalability issue. The system correctly generated the 2,448-row design table and the corresponding flowchart structure, but these representations are difficult to inspect manually at that scale. This does not affect code generation or run-count accuracy, but it limits the usefulness of the preview as an inspection tool. Complete design tables are useful for small and medium-sized experiments, while large factorial designs require filtering, grouping, summarization, or search.

6.5.3 List Indexing in Generated Code

The G-ID experiment exposed a code-generation boundary around list indexing. The original FEDT implementation uses `config_id` to index into pre-computed arrays, such as `bottom_line_angles[config_id]` and `bottom_line_widths[config_id]`, so that each configuration ID maps to derived fabrication parameters. The Blockly workspace can visually express this pattern with list-indexing blocks, but the current generator does not translate `lists_getIndex` blocks into Python indexing expressions. As a result, these indexed parameters are omitted from the generated `Printer.fab()` call in the G-ID implementation.

This is an API Translation limitation rather than a Visual Representation limitation: the pattern can be represented in the workspace, but not yet faithfully emitted as Python code.

6.5.4 Adaptive Termination and Runtime-Determined Sample Counts

The Paint Layers experiment exposes a structural mismatch between adaptive termination and the DOE-based scaffold. The original FEDT implementation fabricates one wooden flower, then repeatedly applies paint coats to the same object until a human judges the coverage reasonable. The number of iterations is not known before execution, which conflicts with the scaffold's assumption that the run structure can be previewed as a fixed design table.

The block-based implementation approximated this pattern by predefining a maximum of 10 iterations and using conditional early termination. This preserved the idea of repeated checking, but it did not preserve the semantics of the original procedure. Two problems emerged. First, fabrication occurs inside the iteration loop, creating separate flower objects rather than repeatedly modifying one persistent object. Second, the nested loop reissues instructions for all previous coats rather than applying only the next coat.

Figure 21 compares the original FEDT implementation with the Blockly attempt. The FEDT version fabricates once and then iteratively refines the same object. The Blockly version creates new objects on each iteration and re-applies coats cumulatively.

Paint Layers: Original FEDT (while loop)

```
flower = Laser.fab(
    GeometryFile('flower.svg'),
    material='wood')
photos = ImmediateMeasurements()
is_reasonable = False
coats = 0
while not is_reasonable:
    flower = Human.is_reasonable(
        flower)
    photos += Camera.take_picture(
        flower)
    is_reasonable = flower.metadata[
        "human reasonableness check"]
    if not is_reasonable:
        coats = coats + 1
        flower = Human.post_process(
            flower,
            f"add coat #{coats}")
summarize(photos.dump_to_csv())
```

Paint Layers: Blockly attempt

```
photos = ImmediateMeasurements()
is_reasonable = False
for paint_layer in Series(
        range(0, 10)):
    if not is_reasonable:
        flower = Laser.fab(
            GeometryFile('flower.svg'),
            material='wood')
        coat_counter = 0
        while coat_counter<paint_layer:
            coat_counter = coat_counter+1
            instruction(
                f"add coat #{coat_counter}")
            flower = flower
    if not is_reasonable:
        photos += Camera.take_picture(
            flower, "photo")
        is_reasonable = (
            Human.do_and_respond(
            "user confirmation",
            "coverage reasonable?"
            ) == 'y')
summarize(photos.dump_to_csv())
```

Figure 21: Comparison of Paint Layers adaptive termination. **Left:** FEDT's while-loop implementation and **right:** the incomplete Blockly representation. The FEDT version fabricates once then iteratively applies paint coats to the same object. The Blockly version creates a new flower object on each iteration and re-applies all previous coats cumulatively, demonstrating the difficulty of representing adaptive termination within the DOE-based scaffold.

Figure 22 shows the Blockly implementation structure. The outer factor pre-defines 10 maximum iterations, with conditional if-blocks enabling early termination when `is_reasonable` becomes true.

A correct representation would require artifact state to persist across adaptive iterations, as well as a design-table model that distinguishes planned maximum iterations from actually executed iterations.

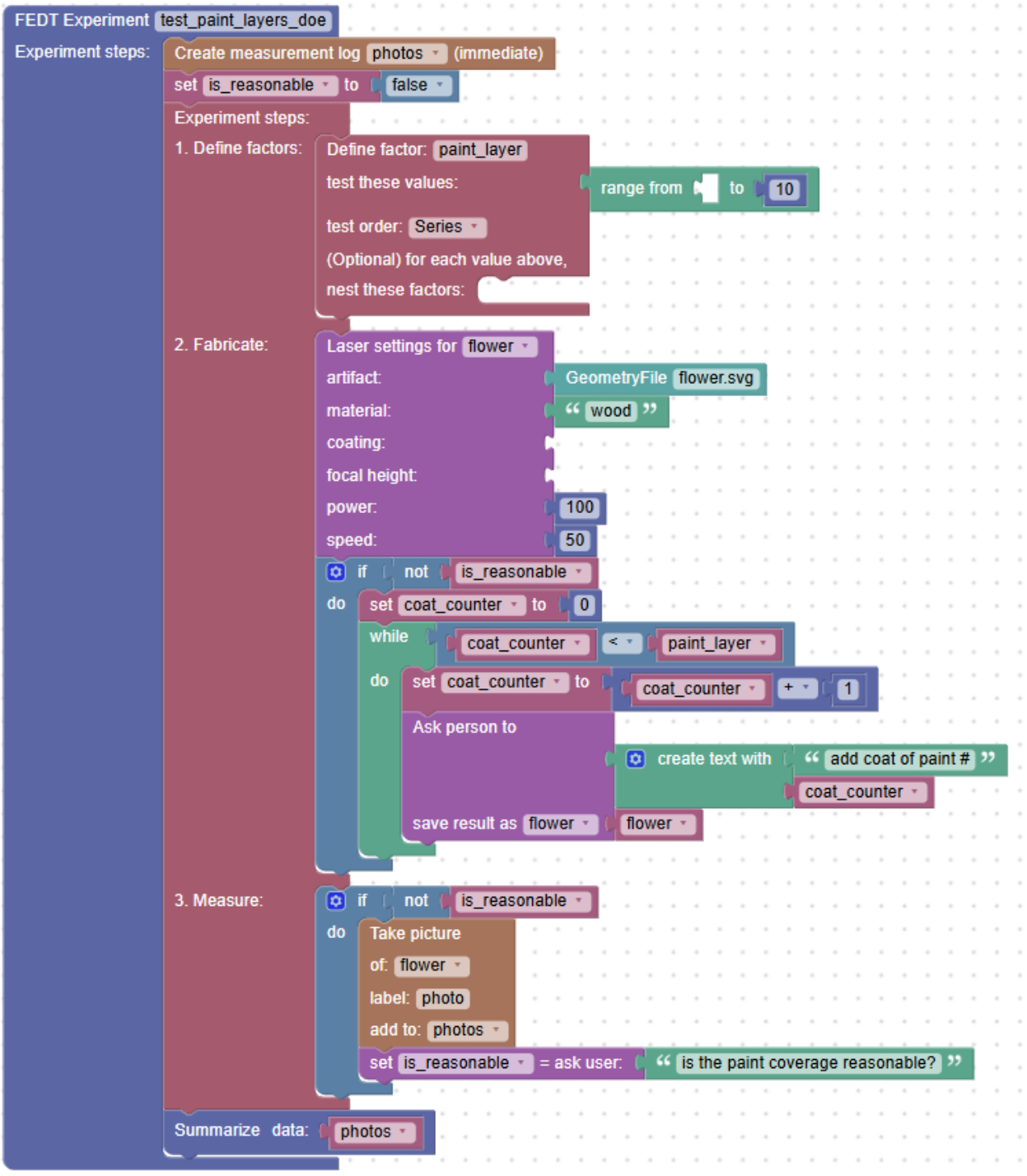


Figure 22: Paint Layers experiment in Blockly showing incomplete adaptive termination attempt. The factor pre-defines 10 maximum iterations with conditional early termination, but structural constraints prevent correct implementation.

This is not only a missing block issue; it is a mismatch between adaptive object refinement and the current scaffold, which assumes that fabrication is applied uniformly across declared factor combinations. The current implementation should therefore be treated as an incomplete representation rather than a supported feature.

6.5.5 Runtime Assignment and Conditional Branching

The User Assembly Time experiment exposes another mismatch: treatment assignment happens at runtime, and the presentation order is chosen with conditional logic. The original FEDT implementation assigns four participants to treatment groups using `shuffle(["simple_first"] * 2 + ["complex_first"] * 2)`, ensuring exactly two participants per condition. Conditional logic then determines whether each participant assembles the simple kit before the complex kit or the reverse.

The block-based implementation can generate correct imperative Python for this protocol using blocks for shuffle, list operations, enumeration, variable assignment, and conditionals. However, the pattern does not fit the three-section scaffold because treatment assignment occurs before iteration, participant/treatment pairs are unpacked at runtime, and the fabrication order is selected dynamically inside an `if`/`else` branch.

Figures 23 and 24 show the implemented solution using raw FEDT blocks outside the scaffold. The generated code runs correctly, but the experiment cannot be represented within the Define Factors / Fabricate / Measure scaffold.

An important distinction: randomization itself does not violate DOE principles. Classical DOE uses randomization extensively (randomizing run order, balancing covariates). The design table can enumerate all factor combinations while leaving execution order unspecified. What conflicts with the DOE-based scaffold here is the *combination* of shuffle before iteration (requiring pre-loop declarations), conditional branching determining fabrication sequence dynamically, and runtime list construction.

A DOE-compliant alternative could use a `treatment_order` factor (”simple_first”, ”complex_first”) and a `participant` factor (0-3), producing an 8-row design table. However, this loses the counterbalancing guarantee (exactly 2 participants per group) and requires duplicating fabrication logic in both conditional branches. The raw-block version preserves the intended counterbalancing better than a forced scaffold version. This suggests that some protocols need imperative structure rather than the current Define Factors / Fabricate / Measure scaffold.

6.5.6 Systematic Counterbalancing

The Trilaterate experiment demonstrates a more specific ordering limitation. The block-based implementation captures the core factorial structure: a single pyramid object is fabricated in the Fabricate section, and nested factors for 12 participants and 7 electrodes produce 84 measurement combinations. The implementation also supports dynamic instruction text through `text_join` and a warmup phase through `random_choice`. Figure 25 shows this DOE scaffold structure with the three-section organization including the warmup instruction.

However, one experimental-design feature remains outside the current block vocabulary. The original FEDT code uses Latin-square counterbalancing to vary the electrode touch order for each participant

User Assembly: Blockly-generated code

```
@fedt_experiment
def test_user_assembly_time():
    timings = ImmediateMeasurements.empty()
    treatments = shuffle(["simple_first"] * 2 + ["complex_first"] * 2)
    for user_treatment in Parallel(enumerate(treatments)):
        user = user_treatment[0]
        treatment = user_treatment[1]
        simple_assembly = Printer.fab(GeometryFile("uploaded/simple_assembly.stl"), material="PLA")
        complex_assembly = Printer.fab(GeometryFile("uploaded/complex_assembly.stl"), material="PLA")
        order = []
        if treatment == "simple_first":
            order = [simple_assembly, complex_assembly]
        else:
            order = [complex_assembly, simple_assembly]
        for assembly in Series(order):
            instruction("solve the assembly")
            timings += Stopwatch.measure_time(assembly, "time_to_solve_assembly")
    summarize(timings.dump_to_csv())
```

Figure 23: Generated Python code for User Assembly Time experiment showing successful shuffle-based counterbalancing with conditional presentation order. The generated code preserves the same counterbalancing and presentation-order structure as the handwritten FEDT version, demonstrating that the block system can generate correct imperative code even when it does not fit the DOE-based scaffold.

during the measured trials. This controls for order effects by ensuring that electrode position and sequence position are not confounded. The block interface does not currently provide a counterbalancing block, so the measured electrode sequence follows the same `Series` order for each participant. Supporting the original Latin-square structure would require a dedicated block or helper abstraction for balanced sequence generation.

The limitation is therefore not random selection or dynamic instruction text, both of which are supported through `random_choice` and `text_join`. The limitation is systematic counterbalanced ordering, such as Latin-square sequence generation. In the current implementation, the warmup electrode can be chosen randomly, but the measured electrode order follows the same `Series` sequence for each participant.

These limitations show that the interface is strongest when experiments can be expressed as declared factors, uniform fabrication steps, and systematic measurements. They also show that additional blocks alone are not always sufficient: adaptive refinement, runtime assignment, and counterbalanced ordering require different structural abstractions, not just more device-specific wrappers.

6.6 Validation Summary

The validation results can be summarized through the two perspectives introduced in the Abstraction Model: API translation and visual representation.

API Translation Perspective: All 22 implemented experiments generated Python code that ran in the FEDT environment without manual edits. The generated code preserved the main FEDT-specific

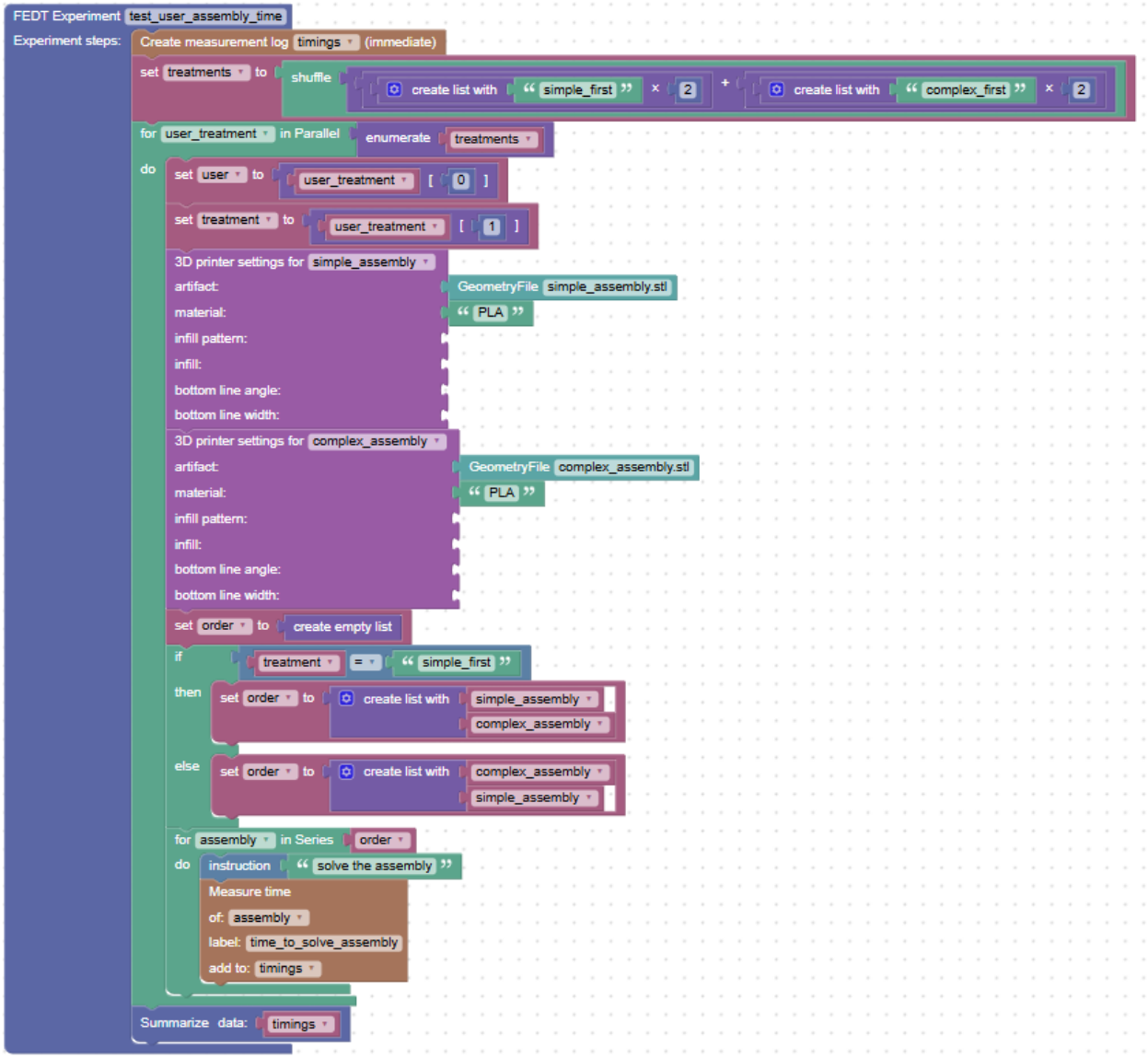


Figure 24: User Assembly Time experiment implemented using raw FEDT blocks outside the three-section scaffold. The structure includes shuffle operations, enumerate-based unpacking, conditional if/else logic, and dynamic list construction. While the code generation works correctly, the experiment cannot be represented within the Define Factors / Fabricate / Measure scaffold because treatment assignment happens at runtime.

programming patterns used in the validation set, including `include_last`, `Parallel`/`Series`, and `BatchMeasurements`. CSV output verification confirmed that the generated experiments preserved the expected design spaces and measurement schemas for the implemented representations. In several cases, this required documented simplifications or workarounds rather than complete feature parity with the original FEDT code.

Visual Representation Perspective: Design table previews showed the planned experiment structure before execution, and flowcharts made the generated execution structure inspectable. The three-section scaffold (Define Factors, Fabricate, Measure) provided a consistent representation of experiment logic, while the code panel made the block-to-code translation visible.

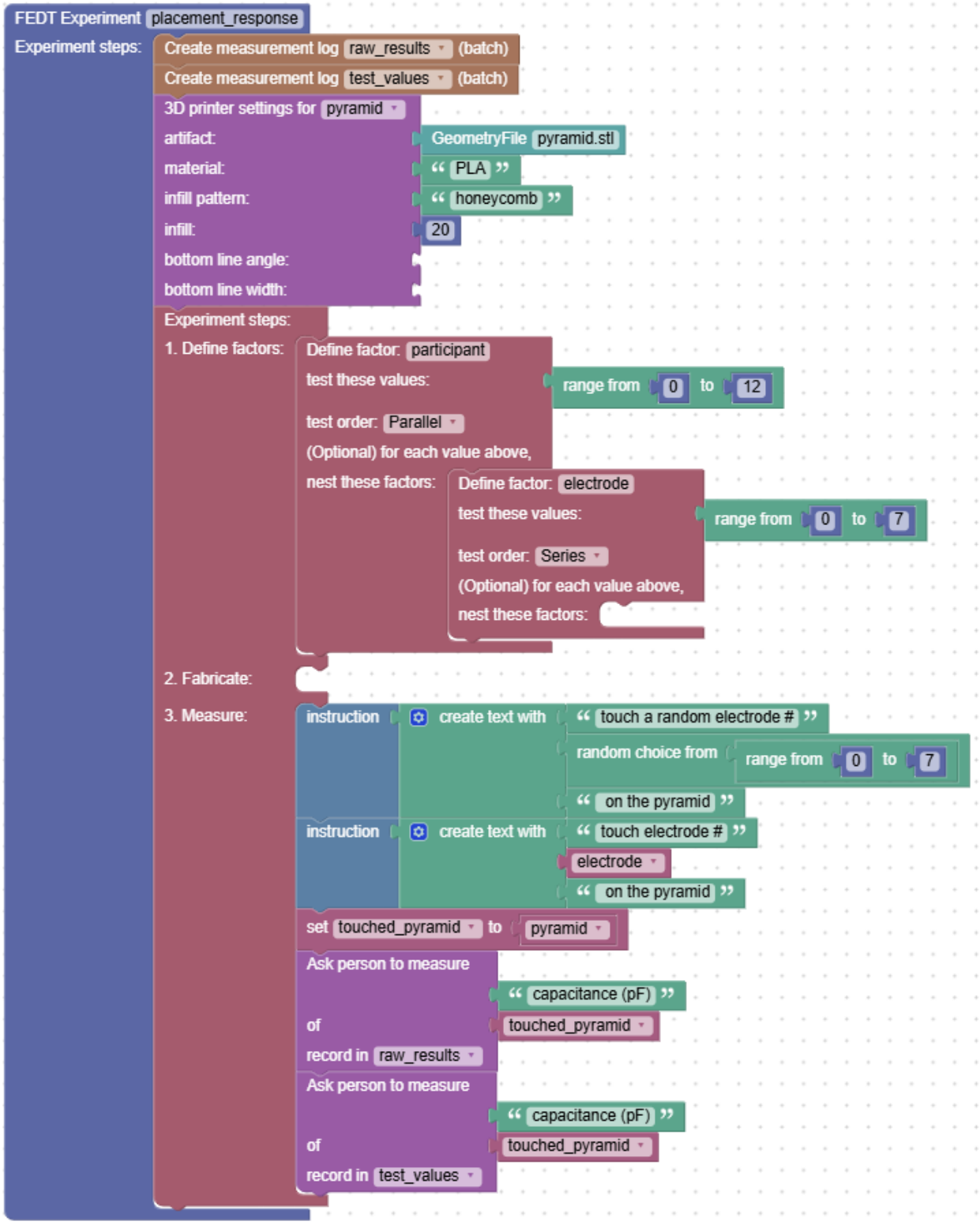


Figure 25: Trilaterate Placement Response experiment demonstrating nested factorial design with single object fabrication, warmup phase randomization, and dynamic instruction text. The pyramid is fabricated in the Fabricate section of the scaffold, with a warmup instruction using `random_choice` to select a random electrode, then 12 participants (Parallel) each touch all 7 electrodes (Series nested), producing 84 measurements.

Across the experiment set, the implementation covered approximately 2,700 planned run contexts, ranging from small single-factor experiments such as CircWood grain direction to the 2,448-combination G-ID experiment. Most experiments fit the scaffold directly or with limited extensions such as instruction blocks, dynamic geometry files, random-choice blocks, or device-specific blocks. A smaller set exposed the boundaries of the approach: Paint Layers depends on adaptive termination, User Assembly Time depends on runtime assignment and conditional ordering outside the scaffold, and Trilaterate requires Latin-square counterbalancing that is not currently supported.

Overall, the validation shows that the system can translate the tested Blockly experiment structures into executable FEDT Python code. The supported cases are mainly declarative factorial experiments with uniform fabrication and measurement steps. The main boundaries are static-analysis coverage, large design tables, incomplete value-expression translation, adaptive termination, runtime-dependent branching, counterbalancing, and highly device-specific protocols.

7 Discussion

This section interprets the validation results and reflects on the design trade-offs that emerged from implementing the experiment set. It revisits the two perspectives introduced in the Abstraction Model: visual representation and API translation. It also discusses limitations of the evaluation and outlines future work.

7.1 Interpretation of Validation Results

The validation shows that the block-based interface can represent the implemented versions of the 22 selected FEDT experiments, either directly or through documented simplifications, wrappers, or workarounds. For these cases, the generated code ran in the FEDT environment without manual edits, preserved important FEDT-specific patterns, and produced flowcharts and CSV outputs that corresponded to the implemented structures. This indicates that the interface is not only a visual editor but also a working translation layer from Blockly configurations to executable FEDT Python code. At the same time, the results should not be read as complete feature parity with every handwritten FEDT implementation.

The flowchart comparison focused on structural correspondence rather than perfect replication. The relevant question was whether Header nodes, fabrication Instruction nodes, and measurement Instruction nodes appeared in the expected order and nesting structure. This level of comparison is appropriate because the interface may organize generated code differently from handwritten FEDT while still preserving the intended experiment structure for supported cases.

For visual representation, the clearest result is the design table preview. By using AST-based static analysis, the system can show the planned design space before fabrication begins, including large cases

such as the G-ID experiment with 2,448 runs. This preview makes run counts, factor combinations, and measurement schemas visible before execution. However, the same mechanism also defines a boundary of the approach: it works for predictable factor definitions such as `range()`, `arange()`, and literal lists, but not for factor levels loaded dynamically from files, databases, or arbitrary user-defined functions. This is separate from code generation itself. Even if the workspace can visually express an experiment, the generator still needs explicit rules for translating each block pattern into Python code.

7.2 Dual Contribution Revisited

The validation shows that the visual interface and code generation depend on each other. Design tables, measurement templates, flowcharts, and block structure make the experiment structure directly inspectable, but these representations are only useful if they correspond to executable FEDT behavior. Conversely, executable code generation alone would offer limited benefit if the interface simply reproduced the complexity of FEDT's Python API in another form.

The three-section scaffold connects the visual structure to the generated code. In the interface, it separates factors, fabrication, and measurement. In the generated code, it ensures that factor variables are defined before fabrication and measurement blocks use them, which simplifies scoping and static analysis. The same constraint also explains several structural limitations: experiments that depend on adaptive termination, runtime branching, or counterbalanced assignment schemes do not fit naturally within the scaffold.

7.3 Design Implications from Implementation Experience

Implementing the 22 experiments indicated that the interface works best when it does not try to expose the entire FEDT API directly. A smaller experiment-oriented vocabulary makes factors, fabrication steps, and measurements explicit, while lower-level configuration details remain inside specialized blocks.

7.3.1 Scaffold over Feature Parity

The implementation suggests that the three-section scaffold is more practical than a direct block-for-function translation of FEDT. A block for every FEDT class, method, and device abstraction would preserve API coverage, but it would also reproduce much of the complexity of the Python workflow. By contrast, the scaffold enforces a clearer separation between Define Factors, Fabricate, and Measure. This keeps the factor, fabrication, and measurement structure visible, even when some individual steps are represented by instruction blocks or simplified wrappers.

From an implementation perspective, the scaffold also makes variable scope predictable. Because factor variables are introduced before fabrication and measurement blocks are generated, the code generator can translate later references without inferring variables from arbitrary control flow. This supports

both code generation and AST-based previews.

The limitation is that this structure favors experiments with predefined runs. As shown by the Paint Layers case, adaptive procedures that continue until a runtime condition is satisfied require a different model of iteration and object state. Supporting such workflows would require either extending the scaffold or introducing a separate mode for more imperative experiment logic. This scaffold-level limitation should be distinguished from lower-level generator coverage gaps, such as missing support for list-indexing expressions in G-ID.

7.3.2 Design Tables as Machine-Readable Specifications

During validation, the Evaluate mode design table became one of the most useful outputs. It served both as a preview for the user and as a structured record of the planned design space. By making factor combinations, run counts, and measurement schemas explicit before execution, the table helped compare implementations and detect unexpected row counts, missing factor values, or incorrect parameter bindings.

7.3.3 Instruction Blocks as Controlled Escape Hatches

Manual instruction blocks were important for representing procedures that could not be fully automated through available device blocks. Air compressor setup, manual cutting and inflation, long biological growth periods, and simplified geometry adjustments could all be documented as part of the generated workflow even when full device integration was unavailable.

This pattern is useful, but it should not be mistaken for complete feature parity. Instruction blocks preserve procedural documentation and guide the user through manual actions, but they do not provide the same guarantees as executable device integrations. They are therefore useful as limited escape hatches: they keep manual steps inside the workflow, while making clear that those steps are not automated.

7.3.4 Device Extensibility and Maintenance Cost

The KnitPicking and SPEERLoom implementations show that FEDT's device abstractions can be exposed through device-specific blocks. This is useful because device-specific blocks can expose relevant parameters directly in the interface. For example, a knitting block can expose pattern-file inputs, while a loom block can present loom-type options without requiring users to remember Python class names or API details.

The cost is maintenance. In FEDT, a new device can be made available by defining or importing the corresponding Python class. In the block interface, each new device usually also requires a block definition, palette entry, code-generation rule, and sometimes additional backend support. Device-specific blocks make supported devices more explicit in the interface, but they also increase the work needed to

support new devices. The SPEERLoom case illustrates this clearly: class-based iteration could be supported, but only through specialized class-reference blocks and simplified wrappers rather than through a general mechanism for arbitrary device classes.

7.3.5 Trade-offs Between Visual and Textual Programming

The implementation suggests that the value of the block interface depends on the user's programming background and task. For users with limited Python experience, the visual scaffold may reduce direct exposure to Python syntax and present experiment structure through factors, tables, and flowcharts. Blockly's variable menu also supports consistent reuse of factor and measurement variables without retyping names manually.

For experienced programmers, however, writing FEDT Python directly may be faster and more flexible, especially for adaptive or highly customized experiments. The block interface introduces its own vocabulary and connection rules, so users must still learn what each block means and where it can be placed. This shifts the learning burden rather than removing it.

This does not make the block interface redundant. It means the interface should be treated as an additional way into FEDT, not as a full replacement for the Python DSL. The generated code panel is important in this respect because it keeps the textual representation visible. It allows users to inspect how block configurations map to FEDT code, and it can also reveal cases where the visual workspace contains a pattern that the generator does not yet fully translate, such as list-indexed parameter lookup.

7.4 Implications for Fabrication Experiment Design

7.4.1 Design Space Visibility

Previewing the design space can reveal configuration mistakes before they consume machine time or material. For example, a factor accidentally configured as `range(0, 100)` instead of `range(0, 10)` would produce many more runs than intended. The design table makes this visible before execution and helps inspect whether the planned coverage is balanced across factor combinations.

7.4.2 Reproducibility Through Structured Representations

FEDT already supports reproducibility by capturing experiment logic in executable Python code. The block interface adds another representation of the same logic: a spatial workspace with explicit factor definitions, generated tables, and flowcharts. These representations can support discussion with collaborators who are less comfortable reading Python code, although this thesis does not empirically evaluate that use.

Sharing the Blockly workspace XML alongside the generated FEDT code could therefore support

multiple levels of inspection. The code remains the executable artifact, while the workspace, design table, and flowchart provide planning and communication artifacts.

7.4.3 Boundaries of Declarative Experiment Design

The validation also clarifies the boundary of the DOE-based approach. Declarative factorial designs assume that factors and levels are known before execution. This assumption enables static previews and clear resource estimates, but it does not hold for every experimental protocol.

Paint Layers and User Assembly Time illustrate this boundary. Paint Layers depends on adaptive termination, where the number of paint coats is determined by runtime judgment. User Assembly Time depends on counterbalanced assignment and conditional ordering. These are valid experimental patterns, but they require control-flow constructs that are less compatible with the current scaffold. This does not mean that all randomization is unsupported: simple random choices can be represented through the random-choice block. The limitation is therefore not only technical; it reflects a trade-off between declarative clarity and imperative expressiveness. The scaffold makes planned factor combinations easy to inspect, but it is less suited to protocols where the next step depends on runtime decisions.

7.5 Generalizability Beyond FEDT

Some design patterns from this work may generalize beyond FEDT. Other experiment-planning systems, such as PLanet [2] and the HCI User Studies Toolkit [19], also structure studies around conditions, assignments, procedures, and measurements. A scaffold that separates condition definition, procedure execution, and data collection could be useful in related domains.

However, this generalizability remains speculative. The implementation was designed around FEDT's execution model, device abstractions, and flowchart generation. Applying the same approach to another framework would require additional validation, especially if that framework uses different assumptions about assignment, execution, or measurement.

7.6 Limitations and Threats to Validity

The evaluation methodology has several limitations that constrain how far the findings can be generalized.

7.6.1 Self-Evaluation Approach

All 22 experiments were implemented and validated by the thesis author. This creates a risk of bias: the implementation process may have favored patterns that the system handles well, and the validation criteria may reflect the system's strengths. A stronger evaluation would involve independent users implementing experiments without prior knowledge of the interface's constraints.

The dual-contribution perspective should also be understood as a conceptual framing, not as an empirically validated model of user experience. The thesis inspects whether the interface exposes experiment structure through blocks, tables, and flowcharts, but it does not measure whether users actually understand the experiment better because of these representations.

7.6.2 Coverage-Based Rather Than User Study Evaluation

The evaluation is coverage-based. It demonstrates that the interface can represent a diverse set of experiments from the FEDT demonstration examples and published HCI fabrication papers, but it does not show whether researchers can successfully use the interface in practice. Important questions remain open: how quickly users learn the block vocabulary, what mistakes they make, whether the design table helps them catch errors, and whether they prefer the block interface over handwritten FEDT code.

Although formative paper-prototype sessions informed the block vocabulary and workflow structure, they were exploratory design activities rather than a controlled evaluation of the final system.

The original project description also considered a comparative usability evaluation against FEDT's command-line workflow, focusing on learnability, error proneness, and coverage. During the project, the scope was narrowed toward technical validation of representation, translation, preview, and execution. Questions about learnability, error reduction, and comparison with the command-line workflow are therefore treated as future work. A controlled user study could compare Blockly-based experiment creation with direct FEDT programming across users with different programming backgrounds, measuring task completion time, error rates, debugging behavior, and confidence in the resulting experiment specification.

7.6.3 Static Analysis Boundaries

The design table preview relies on static analysis to predict experiment structure without activating fabrication devices. The AST-based evaluator handled the expressions used in the experiment set, but it has known limits: dynamic factor levels from CSV files, runtime conditionals, adaptive termination, and arbitrary user-defined functions cannot always be resolved safely before execution.

These limits are inherent to the decision to preview experiments without running their full code. If factor values depend on file I/O, database queries, or complex functions, the analyzer cannot know the resulting design space from syntax alone. In these cases, the system can still generate executable code, but it may not be able to provide the same planning table preview.

A separate limitation concerns code generation. The validation checks the block patterns used by the implemented experiments, but each block pattern still needs a translation rule before it can appear correctly in the generated Python code. When such a rule is missing, the workspace may visually express an experiment that is only partly preserved in the generated code. This means that an experiment may

show the correct number of runs in the design table, while the generated code still omits some parameter values that should be computed from those factors.

7.7 Future Work Directions

The implementation and validation suggest four areas for future work: empirical evaluation, richer control-flow support, scalable previews, and deeper integration with fabrication tools.

First, empirical evaluation with real users would clarify the practical value of the interface. Comparing novice and experienced programmers could help determine whether the block-based scaffold mainly helps users with limited coding experience or whether it also benefits experienced FEDT users during planning and communication.

Second, support for adaptive termination, conditional workflows, and broader value-expression translation would increase the range of experiments that can be represented faithfully. This might involve an advanced mode that relaxes the three-section scaffold, additional visual constructs for persistent objects, runtime branching, and counterbalanced assignment, and more complete generator support for value expressions such as indexed lookup. The challenge is to support these cases without making the block interface as complex as general Python.

Third, large design spaces require better visualization. Flat tables become difficult to inspect when experiments contain hundreds or thousands of runs. Hierarchical views, filtering, grouping, or summary statistics could support the review of large factorial designs.

Finally, the interface could be integrated more deeply with external fabrication tools. Many fabrication experiments involve CAD software, slicers, machine controllers, and statistical analysis tools. A future version of the system could serve as a coordination layer that connects experiment definition, fabrication execution, and downstream analysis in one workflow.

8 Conclusion

This thesis examined whether a block-based web interface can provide an alternative entry point to FEDT while maintaining inspectable experiment structure and executable code generation for supported fabrication patterns. The answer is yes for the declarative fabrication experiments tested in this thesis, where the design space is known before execution. For these experiments, the interface connects Blockly-based authoring with executable FEDT Python code and makes the experiment structure inspectable through blocks, tables, and flowcharts. For adaptive, runtime-dependent, or highly imperative workflows, the answer is more limited because the current DOE-based scaffold cannot yet express these structures directly.

The main contribution is a web-based interface that translates Blockly configurations into FEDT pro-

grams. By using Design of Experiments (DOE) as the organizing structure, the system separates the experiment into three phases: Define Factors, Fabricate, and Measure. This scaffold exposes the intended design space before execution and limits code generation to the patterns currently supported by the system. The implementation of 22 experiments from FEDT examples and published fabrication papers demonstrates that the approach covers a broad range of declarative fabrication workflows, including factorial designs, parametric geometry, device-specific fabrication steps, manual intervention, and multiple measurement types.

The thesis evaluated the system from two perspectives: what the interface makes visible and what Python code it generates. From the visual representation perspective, the interface should make experiment structure inspectable through the workspace and its generated previews. From the API translation perspective, the same visual structure must translate into FEDT Python code that runs in the FEDT environment and preserves important FEDT-specific patterns. The validation shows that both parts matter: visual previews are useful only if they correspond to the executable experiment, and generated code is easier to verify when its structure can be inspected.

The implementation also shows where the approach breaks down. The system works best for experiments whose design space is known before execution, because this enables static previews, resource estimation, and inspection of planned factor combinations. Experiments that depend on adaptive termination, runtime assignment, conditional ordering, systematic counterbalancing, or dynamically generated factor values do not fit as naturally within the current scaffold. Some limitations are structural, such as the lack of persistent object-state support for adaptive refinement. Others concern generator coverage, such as value blocks that can be represented visually but are not yet translated into complete Python expressions.

Overall, the thesis shows that the implemented system is technically feasible for the tested experiment patterns. The system generated FEDT Python code for the implemented experiments, preserved key FEDT patterns such as `Parallel`, `Series`, `include_last`, and `BatchMeasurements`, and provided planning previews before execution. At the same time, the validation shows that representational coverage and full semantic equivalence are not identical. Several experiments required simplifications, manual instruction blocks, specialized wrappers, or incomplete value-block translation.

Future work should evaluate the interface with real users, especially across different levels of programming experience, to determine whether the visual scaffold improves learnability, error detection, and confidence in experiment specifications. Further technical work should extend support for adaptive and conditional workflows, persistent object state, counterbalanced assignment, richer value-expression translation, and better visualization of large design spaces.

This work shows one way to make structured fabrication experiments visible in a web interface while still generating executable specifications. Many fabrication experiments involve systematic variation across materials, geometry, machine settings, or post-processing conditions. The main trade-off is be-

tween structure and expressiveness: the declarative scaffold makes planned factor combinations easier to inspect, but it is less suited to workflows that depend on runtime adaptation. Future interface work should therefore explore how to support adaptive and imperative workflows without losing the clarity of the scaffold.

Acknowledgements

I would like to thank my supervisor, Valkyrie Savage, for her guidance and support throughout this thesis project. I am also grateful to Harry Goldstein and Chandrakana Nandi for their insights into FEDT's structure and for their feedback on the implementation. Lastly, I would like to thank the participants in the paper prototype sessions for their time and input.

References

[1] Miryam Barad. Design of experiments (doe)—a valuable multi-purpose methodology. *Applied Mathematics*, 5, 01 2014.

[2] London Bielicke, Anna Zhang, Shruti Tyagi, Emery D. Berger, Adam Chlipala, and Eunice Jun. PLanet: Formalizing assignment procedures in the design of experiments, 2025.

[3] Karen Brennan and Mitchel Resnick. New frameworks for studying and assessing the development of computational thinking. In *American Educational Research Association (AERA) Annual Meeting*, Vancouver, BC, Canada, 2012. Conference paper.

[4] Mustafa Doga Dogan, Faraz Faruqi, Andrew Day Churchill, Kenneth Friedman, Leon Cheng, Sriram Subramanian, and Stefanie Mueller. G-id: Identifying 3d prints using slicing parameters. In *Proceedings of the 2020 CHI Conference on Human Factors in Computing Systems*, CHI '20, page 1–13, New York, NY, USA, 2020. Association for Computing Machinery.

[5] Cathy Mengying Fang, Jianzhe Gu, Lining Yao, and Chris Harrison. Electripop: Low-cost, shape-changing displays using electrostatically inflated mylar sheets. In *Proceedings of the 2022 CHI Conference on Human Factors in Computing Systems*, CHI '22, New York, NY, USA, 2022. Association for Computing Machinery.

[6] Phillip Gough, Praneeth Bimsara Perera, Michael A. Kertesz, and Anusha Withana. Design, mould, grow!: A fabrication pipeline for growing 3d designs using myco-materials. In *Proceedings of the 2023 CHI Conference on Human Factors in Computing Systems*, CHI '23, New York, NY, USA, 2023. Association for Computing Machinery.

[7] Megan Hofmann, Lea Albaugh, Ticha Sethapakadi, Jessica Hodgins, Scott E. Hudson, James McCann, and Jennifer Mankoff. Knitpicking textures: Programming and modifying complex knitted textures for machine and hand knitting. In *Proceedings of the 32nd Annual ACM Symposium on User Interface Software and Technology*, UIST '19, page 5–16, New York, NY, USA, 2019. Association for Computing Machinery.

[8] Ayaka Ishii, Kunihiro Kato, Kaori Ikematsu, Yoshihiro Kawahara, and Itiro Siio. Circwood: Laser printed circuit boards and sensors for affordable diy woodworking. In *Proceedings of the Sixteenth International Conference on Tangible, Embedded, and Embodied Interaction*, TEI '22, New York, NY, USA, 2022. Association for Computing Machinery.

[9] John Maloney, Mitchel Resnick, Natalie Rusk, Brian Silverman, and Evelyn Eastmond. The Scratch programming language and environment. *ACM Transactions on Computing Education (TOCE)*, 10(4), November 2010.

[10] Judyta Niemiro-Maźniak. Application of design of experiment (doe) methods in materials engineering and construction. *Zeszyty Naukowe Politechniki Częstochowskiej. Budownictwo*, 31 (181):22–28, 2025.

[11] Jasper Tran O'Leary, Gabrielle Benabdallah, and Nadya Peek. Imprimer: Computational notebooks for CNC milling. In *Proceedings of the 2023 CHI Conference on Human Factors in Computing Systems (CHI '23)*, Hamburg, Germany, April 2023. ACM.

[12] Jasper Tran O'Leary, Thrisha Ramesh, Octi Zhang, and Nadya Peek. Tandem: Reproducible digital fabrication workflows as multimodal programs. In *Proceedings of the CHI Conference on Human Factors in Computing Systems (CHI '24)*, Honolulu, HI, USA, May 2024. ACM.

[13] Erik Pasternak, Rachel Fenichel, and Andrew N. Marshall. Tips for creating a block language with blockly. In *2017 IEEE Blocks and Beyond Workshop (B&B)*, pages 21–24, 2017.

[14] Arvind Satyanarayan, Dominik Moritz, Kanit Wongsuphasawat, and Jeffrey Heer. Vega-lite: A grammar of interactive graphics. *IEEE Transactions on Visualization and Computer Graphics (Proc. IEEE InfoVis)*, 2017.

[15] Arvind Satyanarayan, Ryan Russell, Jane Hoffswell, and Jeffrey Heer. Reactive vega: A streaming dataflow architecture for declarative interactive visualization. *IEEE Transactions on Visualization and Computer Graphics (Proc. IEEE InfoVis)*, 2016.

[16] Valkyrie Savage, Harrison Goldstein, Nóra Püsök, Jia Yi Ren, Bhaskar Dutt, Chandrakana Nandi, and Lora Oehlberg. FEDT: Supporting experiment design and execution in HCI fabrication research. In *The 38th Annual ACM Symposium on User Interface Software and Technology (UIST '25)*, Busan, Republic of Korea, September 2025. ACM.

[17] Valkyrie Savage, Carlos Tejada, Mengyu Zhong, Raf Ramakers, Daniel Ashbrook, and Hyunyoung Kim. Airlogic: Embedding pneumatic computation and i/o in 3d models to fabricate electronics-free interactive objects. In *Proceedings of the 35th Annual ACM Symposium on User Interface Software and Technology*, UIST '22, New York, NY, USA, 2022. Association for Computing Machinery.

[18] Martin Schmitz, Martin Stitz, Florian Müller, Markus Funk, and Max Mühlhäuser. ./trilaterate: A fabrication pipeline to design and 3d print hover-, touch-, and force-sensitive objects. In *Proceedings of the 2019 CHI Conference on Human Factors in Computing Systems*, CHI '19, page 1–13, New York, NY, USA, 2019. Association for Computing Machinery.

[19] Valentin Schwind, Stefan Resch, and Jessica Sehrt. The HCI user studies toolkit: Supporting study designing and planning for undergraduates and novice researchers in human-computer interaction. In *Extended Abstracts of the 2023 CHI Conference on Human Factors in Computing Systems (CHI EA '23)*, Hamburg, Germany, April 2023. ACM.

[20] Madlaina Signer, Alexandra Ion, and Olga Sorkine-Hornung. Developable metamaterials: Mass-fabricable metamaterials by laser-cutting elastic structures. In *Proceedings of the 2021 CHI Conference on Human Factors in Computing Systems*, CHI '21, New York, NY, USA, 2021. Association for Computing Machinery.

[21] Samantha Speer, Ana P Garcia-Alonzo, Joey Huang, Nickolina Yankova, Carolyn Rosé, Kylie A Peppler, James Mccann, and Melisa Orta Martinez. Speerloom: An open-source loom kit for interdisciplinary engagement in math, engineering, and textiles. In *Proceedings of the 36th Annual ACM Symposium on User Interface Software and Technology*, UIST '23, New York, NY, USA, 2023. Association for Computing Machinery.

[22] Guanyun Wang, Ye Tao, Ozguc Bertug Capunaman, Humphrey Yang, and Lining Yao. A-line: 4d printing morphing linear composite structures. In *Proceedings of the 2019 CHI Conference on Human Factors in Computing Systems*, CHI '19, page 1–12, New York, NY, USA, 2019. Association for Computing Machinery.

[23] Junichi Yamaoka, Ryuma Niiyama, and Yasuaki Kakehi. Blowfab: Rapid prototyping for rigid and reusable objects using inflation of laser-cut surfaces. In *Proceedings of the 30th Annual ACM Symposium on User Interface Software and Technology*, UIST '17, page 461–469, New York, NY, USA, 2017. Association for Computing Machinery.

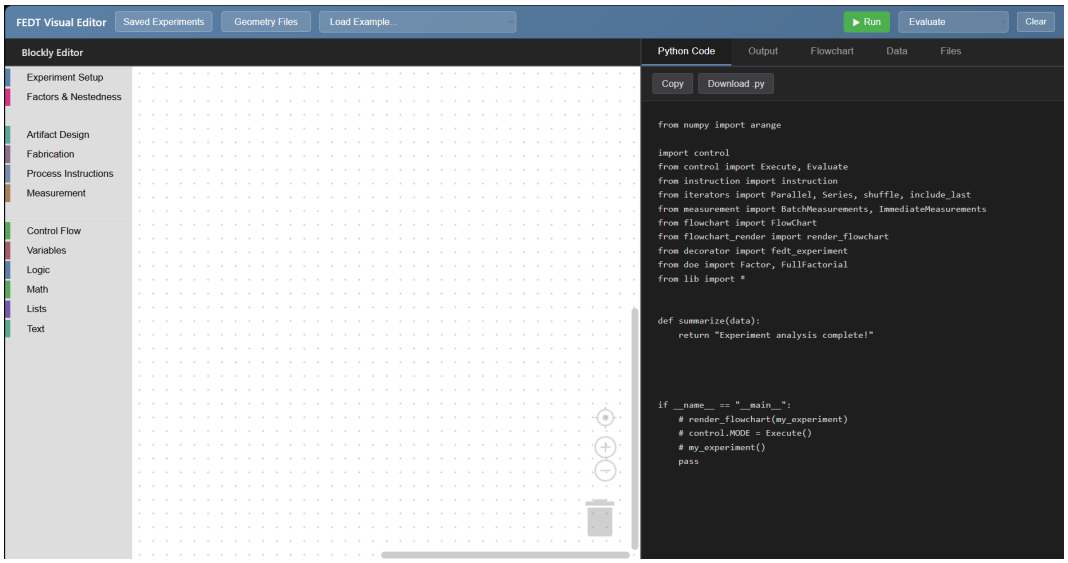
[24] Zeyu Yan and Huaishu Peng. Fabhydro: Printing interactive hydraulic devices with an affordable sla 3d printer. In *The 34th Annual ACM Symposium on User Interface Software and Technology*, UIST '21, page 298–311, New York, NY, USA, 2021. Association for Computing Machinery.

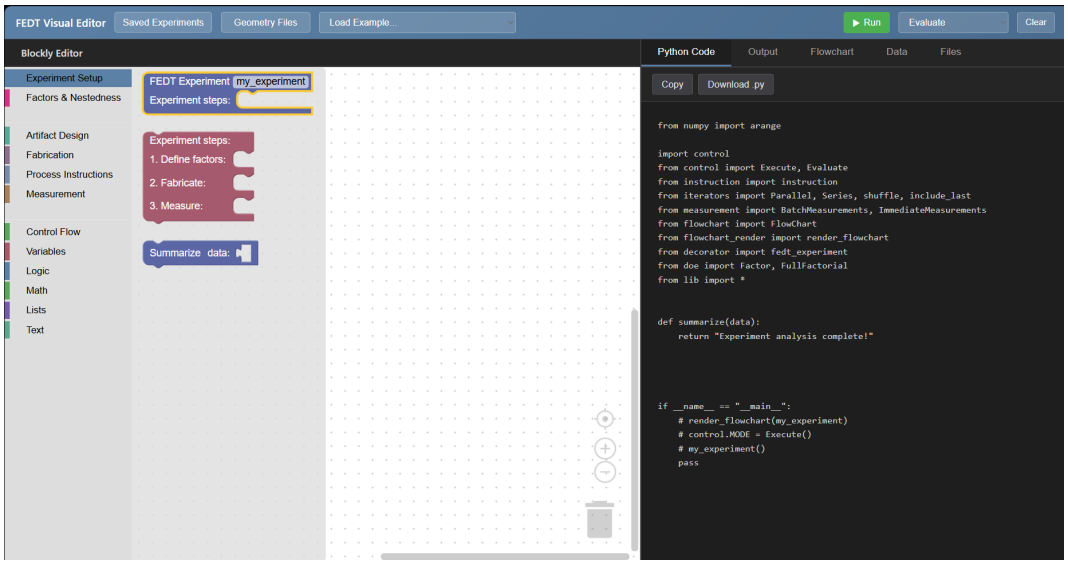
A User Walkthrough: Creating a Laser Cutting Experiment

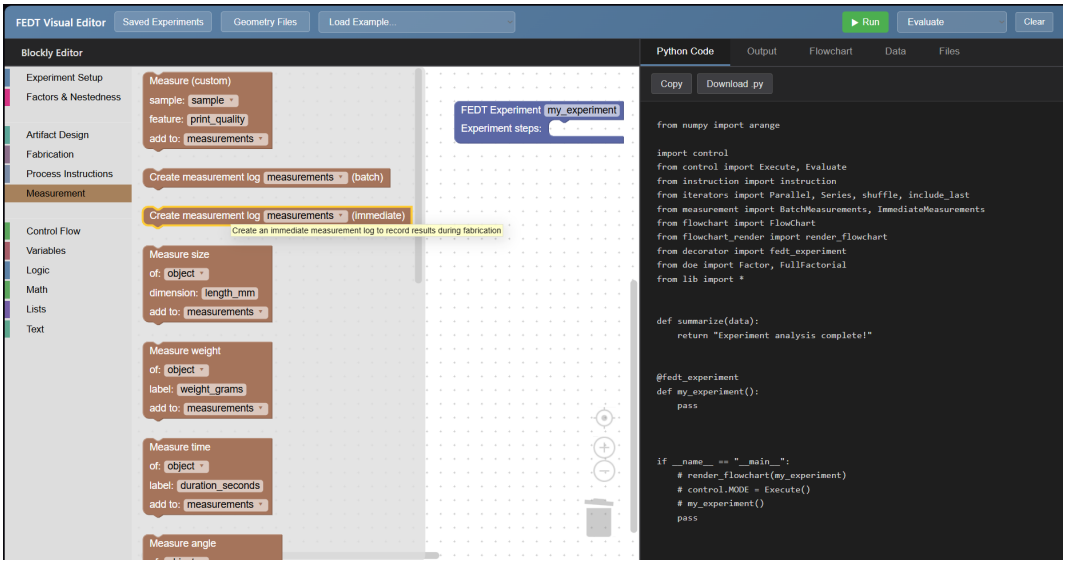
This appendix demonstrates the complete workflow for creating and executing a fabrication experiment using the FEDT Visual Editor web interface. The example experiment tests two materials (acrylic and wood) by laser cutting rectangles and measuring the thickness of the resulting pieces.

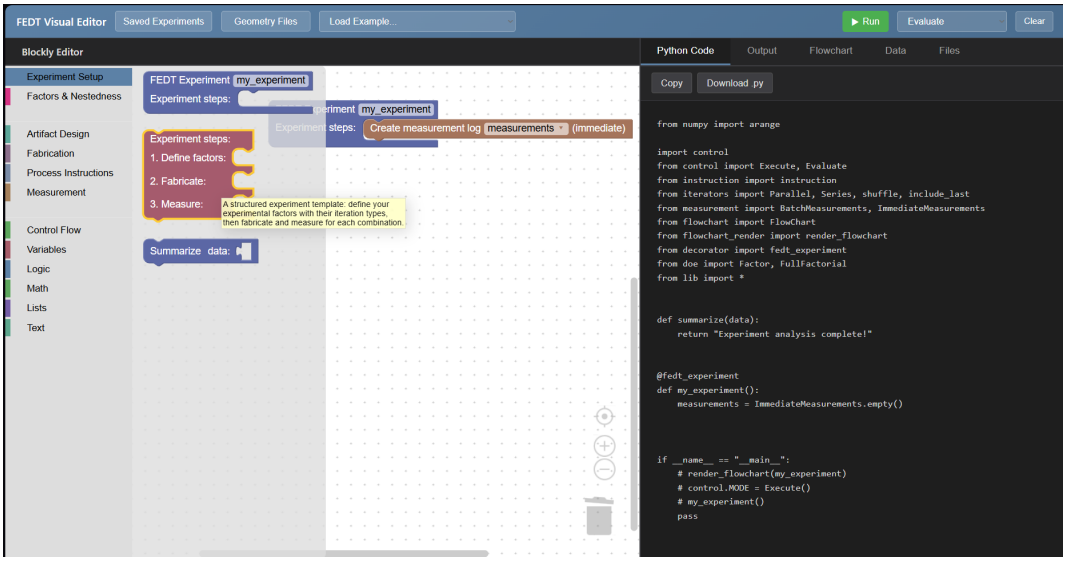
A.1 Building the Experiment

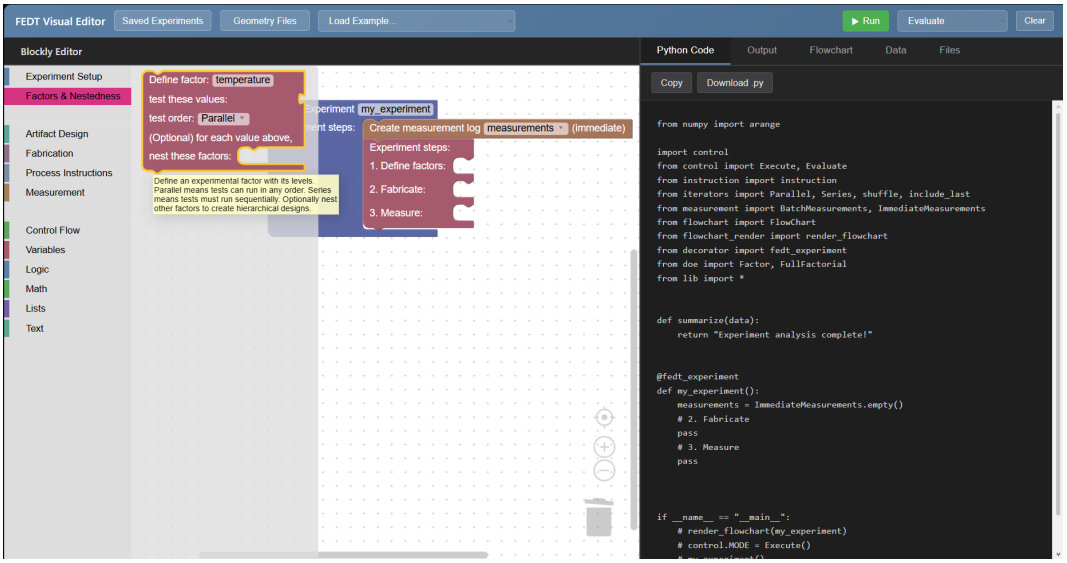
Below are the screenshots of building an experiment in the FEDT Visual Editor using the block interface. The sequence shows starting with an empty workspace, adding the experiment template, selecting the factor definition, building the complete experiment with material values (acrylic and wood), laser cutting fabrication, and thickness measurement, viewing the automatically generated Python code, flowchart, and data tables.

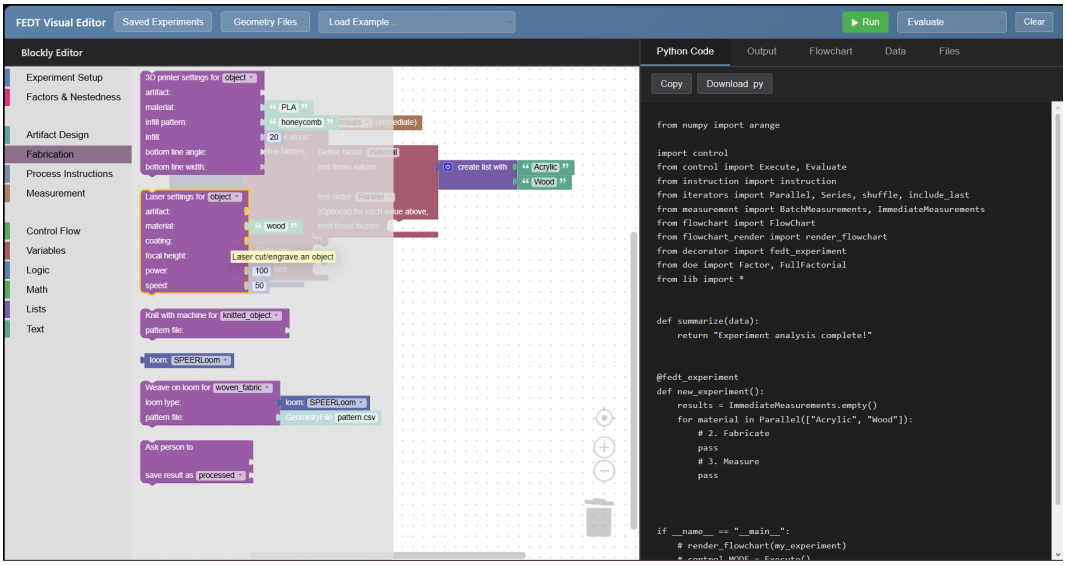


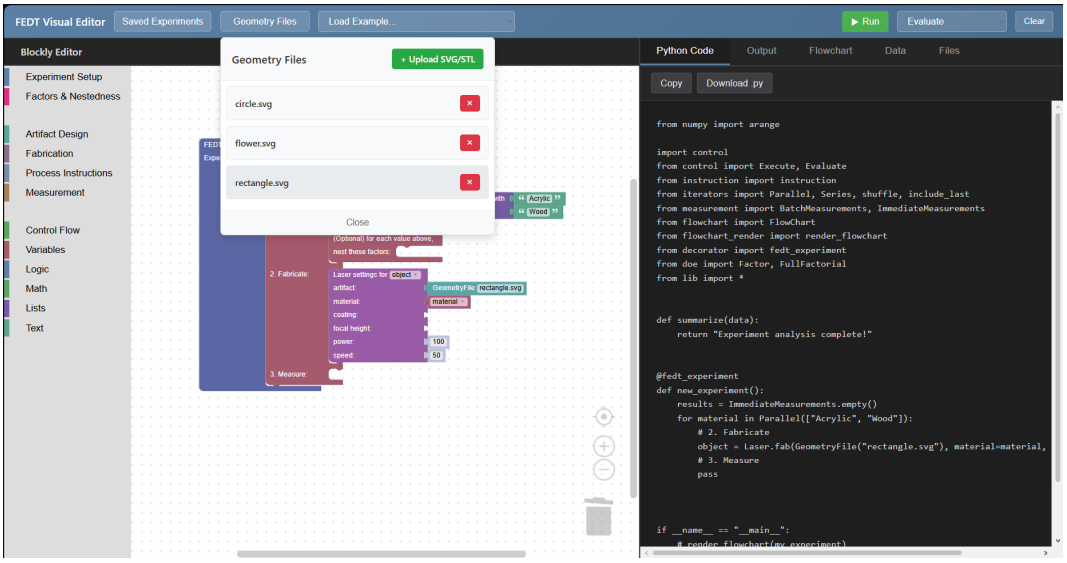


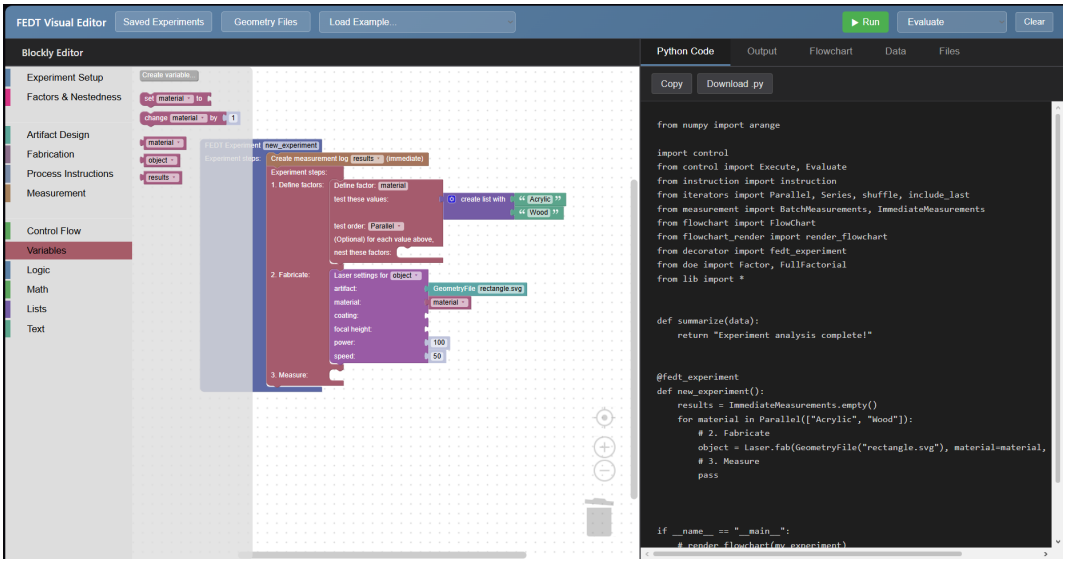

FEDT Visual Editor
Saved Experiments
Geometry Files
Load Example...
Run
Evaluate
Clear
Blockly Editor
Experiment Setup
Factors & Nestedness
Artifact Design
Fabrication
Process Instructions
Measurement
Control Flow
Variables
Logic
Math
Lists
Text
Measure (custom)
sample: sample
feature: print_quality
add to: measurements
Create measurement log measurements (batch)
Create measurement log measurements (immediate)
Create an immediate measurement log to record results during fabrication
Measure size
of: object
dimension: length_mm
add to: measurements
Measure weight
of: object
label: weight_grams
add to: measurements
Measure time
of: object
label: duration_seconds
add to: measurements
Measure angle
FEDT Experiment my_experiment
Experiment steps:
Python Code
Output
Flowchart
Data
Files
Copy
Download .py
from numpy import arange
import control
from control import Execute, Evaluate
from instruction import instruction
from iterators import Parallel, Series, shuffle, include_last
from measurement import BatchMeasurements, ImmediateMeasurements
from flowchart import FlowChart
from flowchart_render import render_flowchart
from decorator import fedt_experiment
from doe import Factor, FullFactorial
from lib import *
def summarize(data):
return "Experiment analysis complete!"
@fedt_experiment
def my_experiment():
pass
if __name__ == "__main__":
render_flowchart(my_experiment)
control.MODE = Execute()
my_experiment()
pass

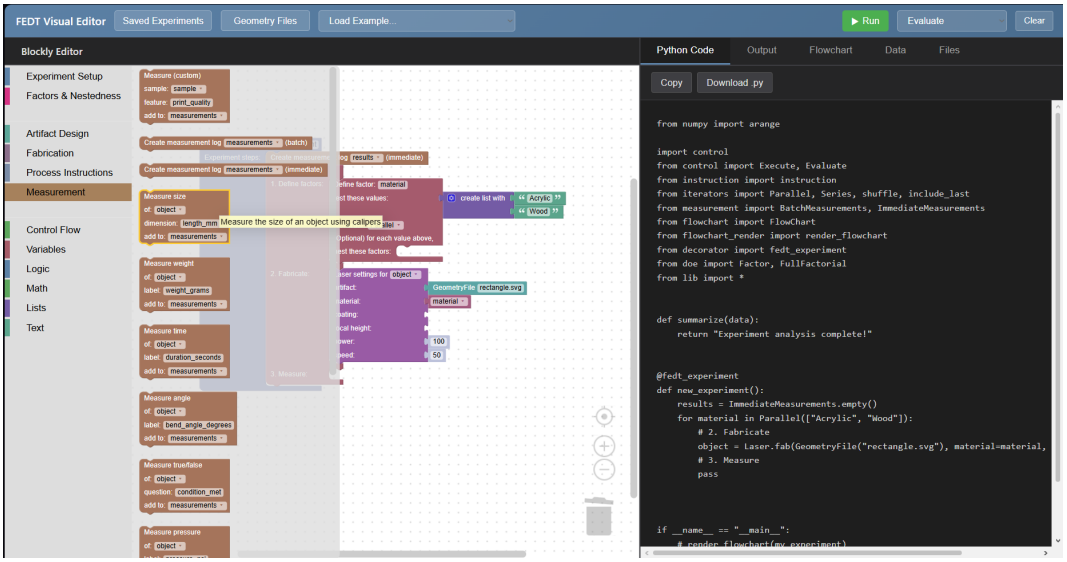

FEDT Visual Editor
Saved Experiments
Geometry Files
Load Example...
Run
Evaluate
Clear
Blockly Editor
Experiment Setup
Factors & Nestedness
Artifact Design
Fabrication
Process Instructions
Measurement
Control Flow
Variables
Logic
Math
Lists
Text
FEDT Experiment my_experiment
Experiment steps:
Create measurement log measurements (immediate)
Experiment steps:
1. Define factors:
2. Fabricate:
3. Measure:
A structured experiment template: define your experimental factors with their iteration types, then fabricate and measure for each combination.
Summarize data:
Python Code
Output
Flowchart
Data
Files
Copy
Download .py
from numpy import arange
import control
from control import Execute, Evaluate
from instruction import instruction
from iterators import Parallel, Series, shuffle, include_last
from measurement import BatchMeasurements, ImmediateMeasurements
from flowchart import FlowChart
from flowchart_render import render_flowchart
from decorator import fedt_experiment
from doe import Factor, FullFactorial
from lib import *
def summarize(data):
return "Experiment analysis complete!"
@fedt_experiment
def my_experiment():
measurements = ImmediateMeasurements.empty()
if __name__ == "__main__":
render_flowchart(my_experiment)
control.MODE = Execute()
my_experiment()
pass

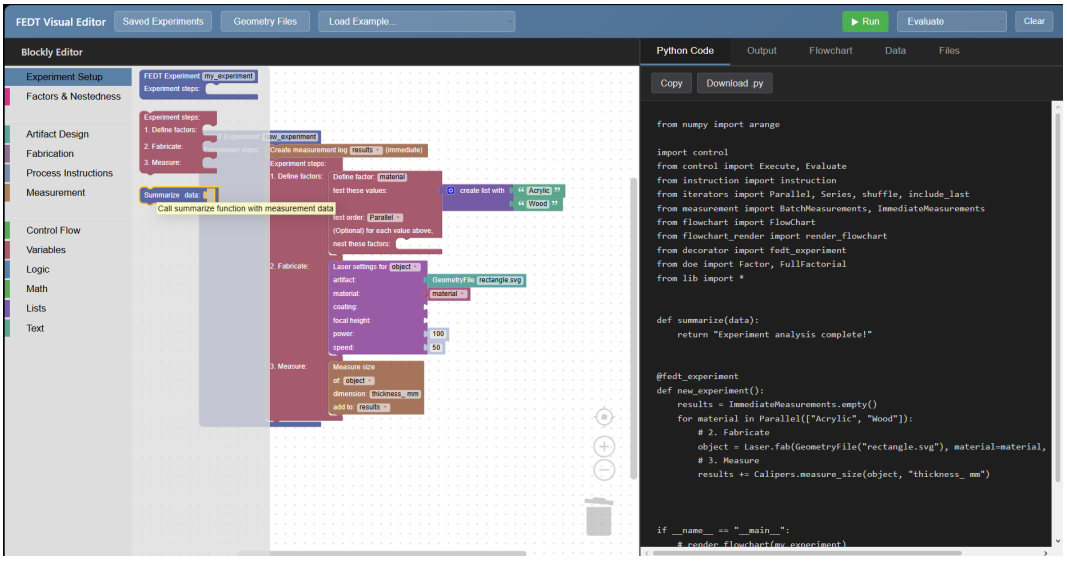

FEDT Visual Editor
Saved Experiments
Geometry Files
Load Example...
Run
Evaluate
Clear
Blockly Editor
Experiment Setup
Factors & Nestedness
Artifact Design
Fabrication
Process Instructions
Measurement
Control Flow
Variables
Logic
Math
Lists
Text
Define factor: temperature
test these values:
test order: Parallel
(Optional) for each value above,
nest these factors:
Define an experimental factor with its levels. Parallel means tests can run in any order. Series means tests must run sequentially. Optionally nest other factors to create hierarchical designs.
Create measurement log measurements (immediate)
Experiment steps:
1. Define factors:
2. Fabricate:
3. Measure:
Python Code
Output
Flowchart
Data
Files
Copy
Download .py
from numpy import arange
import control
from control import Execute, Evaluate
from instruction import instruction
from iterators import Parallel, Series, shuffle, include_last
from measurement import BatchMeasurements, ImmediateMeasurements
from flowchart import FlowChart
from flowchart_render import render_flowchart
from decorator import fedt_experiment
from doe import Factor, FullFactorial
from lib import *
def summarize(data):
return "Experiment analysis complete!"
@fedt_experiment
def my_experiment():
measurements = ImmediateMeasurements.empty()
2. Fabricate
pass
3. Measure
pass
if __name__ == "__main__":
render_flowchart(my_experiment)
control.MODE = Execute()

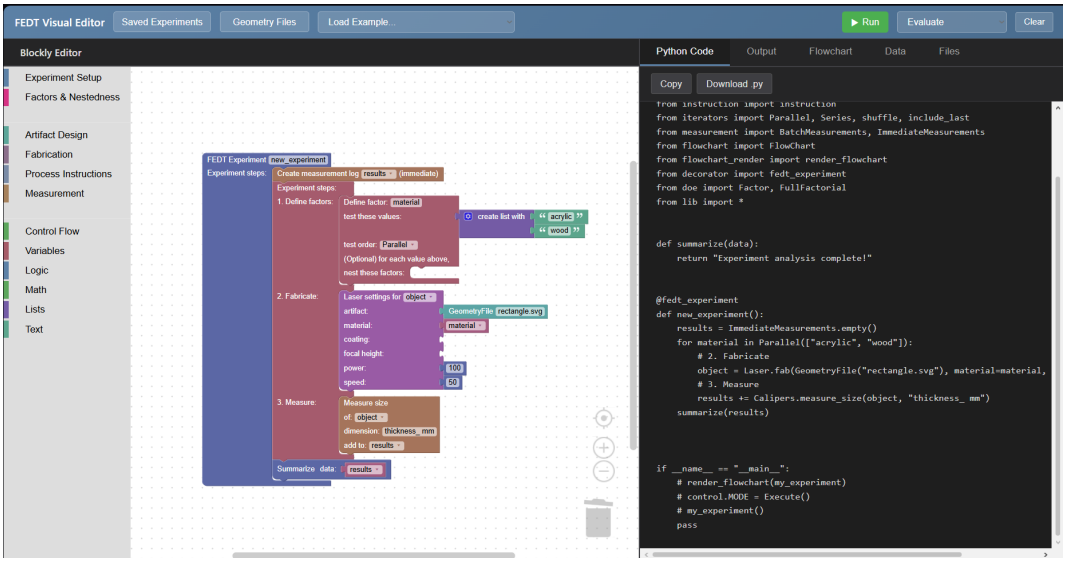

FEDT Visual Editor
Saved Experiments
Geometry Files
Load Example...
Run
Evaluate
Clear
Blockly Editor
Experiment Setup
Factors & Nestedness
Artifact Design
Fabrication
Process Instructions
Measurement
Control Flow
Variables
Logic
Math
Lists
Text
Python Code
Output
Flowchart
Data
Files
Copy
Download .py
from numpy import arange
import control
from control import Execute, Evaluate
from instruction import instruction
from iterators import Parallel, Series, shuffle, include_last
from measurement import BatchMeasurements, ImmediateMeasurements
from flowchart import FlowChart
from flowchart_render import render_flowchart
from decorator import fedt_experiment
from doe import Factor, FullFactorial
from lib import *
def summarize(data):
return "Experiment analysis complete!"
@fedt_experiment
def new_experiment():
results = ImmediateMeasurements.empty()
for material in Parallel(["Acrylic", "Wood"]):
2. Fabricate
pass
3. Measure
pass
if __name__ == "__main__":
render_flowchart(my_experiment)

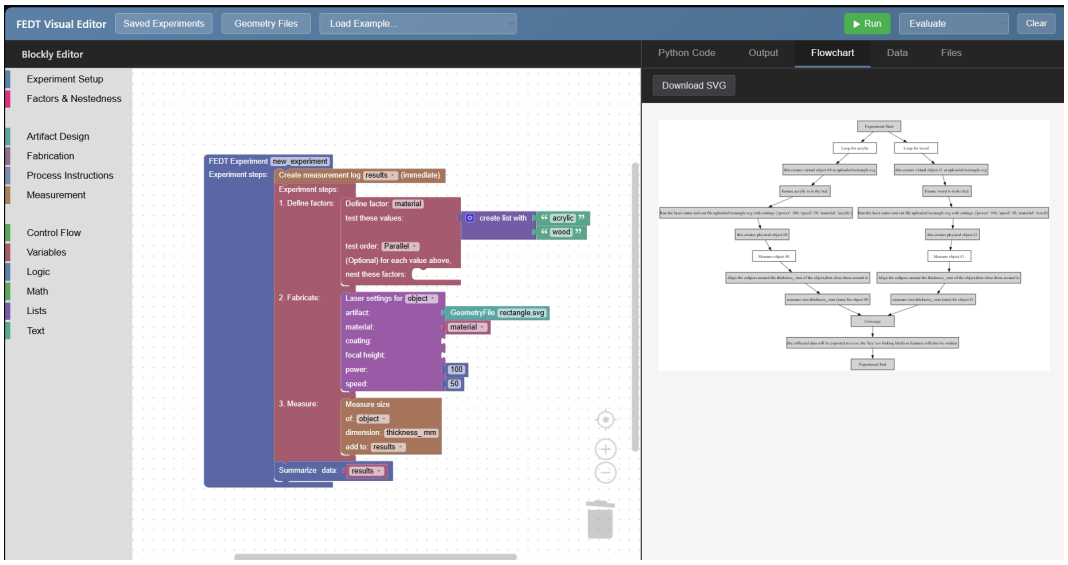

FEDT Visual Editor
Saved Experiments
Geometry Files
Load Example...
Run
Evaluate
Clear
Blockly Editor
Geometry Files
+ Upload SVG/STL
circle.svg
flower.svg
rectangle.svg
Close
Experiment Setup
Factors & Nestedness
Artifact Design
Fabrication
Process Instructions
Measurement
Control Flow
Variables
Logic
Math
Lists
Text
Python Code
Output
Flowchart
Data
Files
Copy
Download .py
from numpy import arange
import control
from control import Execute, Evaluate
from instruction import instruction
from iterators import Parallel, Series, shuffle, include_last
from measurement import BatchMeasurements, ImmediateMeasurements
from flowchart import FlowChart
from flowchart_render import render_flowchart
from decorator import fedt_experiment
from doe import Factor, FullFactorial
from lib import *
def summarize(data):
return "Experiment analysis complete!"
@fedt_experiment
def new_experiment():
results = ImmediateMeasurements.empty()
for material in Parallel(["Acrylic", "Wood"]):
2. Fabricate
object = Laser.fab(GeometryFile("rectangle.svg"), material=material,
3. Measure
pass
if __name__ == "__main__":

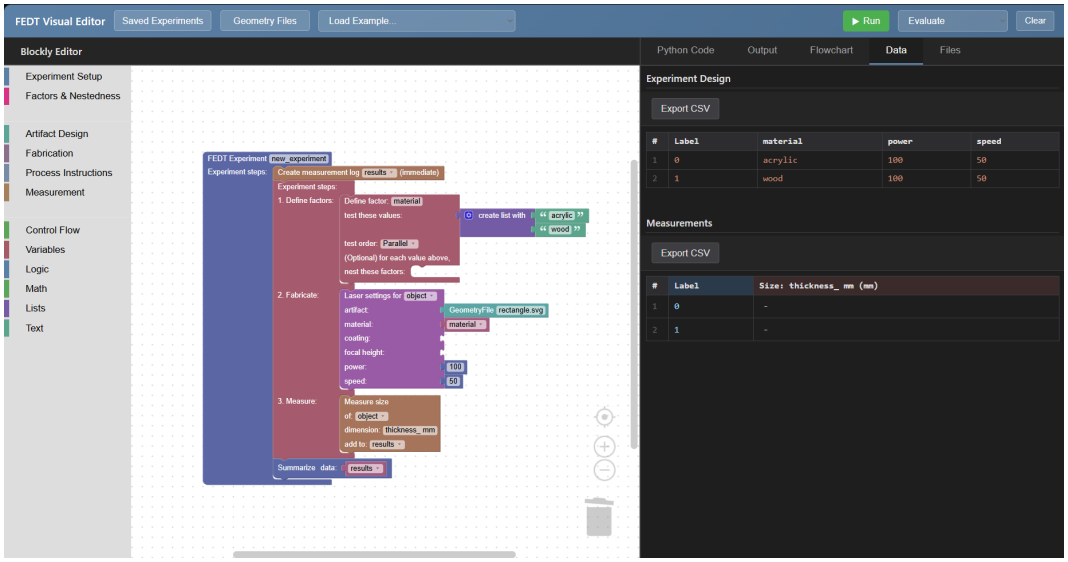

FEDT Visual Editor
Saved Experiments
Geometry Files
Load Example...
Run
Evaluate
Clear
Blockly Editor
Experiment Setup
Factors & Nestedness
Artifact Design
Fabrication
Process Instructions
Measurement
Control Flow
Variables
Logic
Math
Lists
Text
Python Code
Output
Flowchart
Data
Files
Copy
Download .py
from numpy import arange
import control
from control import Execute, Evaluate
from instruction import instruction
from iterators import Parallel, Series, shuffle, include_last
from measurement import BatchMeasurements, ImmediateMeasurements
from flowchart import FlowChart
from flowchart_render import render_flowchart
from decorator import fedt_experiment
from doe import Factor, FullFactorial
from lib import *
def summarize(data):
return "Experiment analysis complete!"
@fedt_experiment
def new_experiment():
results = ImmediateMeasurements.empty()
for material in Parallel(["Acrylic", "Wood"]):
2. Fabricate
object = Laser.fab(GeometryFile("rectangle.svg"), material=material,
3. Measure
pass
if __name__ == "__main__":


FEDT Visual Editor
Saved Experiments
Geometry Files
Load Example...
Run
Evaluate
Clear
Blockly Editor
Python Code
Output
Flowchart
Data
Files
Copy
Download .py
Experiment Setup
Factors & Nestedness
Artifact Design
Fabrication
Process Instructions
Measurement
Control Flow
Variables
Logic
Math
Lists
Text
Measure the size of an object using calipers
from numpy import arange
import control
from control import Execute, Evaluate
from instruction import instruction
from iterators import Parallel, Series, shuffle, include_last
from measurement import BatchMeasurements, ImmediateMeasurements
from flowchart import FlowChart
from flowchart_render import render_flowchart
from decorator import fedt_experiment
from doe import Factor, FullFactorial
from lib import *
def summarize(data):
return "Experiment analysis complete!"
@fedt_experiment
def new_experiment():
results = ImmediateMeasurements.empty()
for material in Parallel(["Acrylic", "Wood"]):
2. Fabricate
object = Laser.fab(GeometryFile("rectangle.svg"), material=material,
3. Measure
pass
if __name__ == "__main__":
render_flowchart(my_experiment)


FEDT Visual Editor
Saved Experiments
Geometry Files
Load Example...
Run
Evaluate
Clear
Blockly Editor
Python Code
Output
Flowchart
Data
Files
Copy
Download .py
Experiment Setup
Factors & Nestedness
Artifact Design
Fabrication
Process Instructions
Measurement
Control Flow
Variables
Logic
Math
Lists
Text
Call summarize function with measurement data
from numpy import arange
import control
from control import Execute, Evaluate
from instruction import instruction
from iterators import Parallel, Series, shuffle, include_last
from measurement import BatchMeasurements, ImmediateMeasurements
from flowchart import FlowChart
from flowchart_render import render_flowchart
from decorator import fedt_experiment
from doe import Factor, FullFactorial
from lib import *
def summarize(data):
return "Experiment analysis complete!"
@fedt_experiment
def new_experiment():
results = ImmediateMeasurements.empty()
for material in Parallel(["Acrylic", "Wood"]):
2. Fabricate
object = Laser.fab(GeometryFile("rectangle.svg"), material=material,
3. Measure
results += Calipers.measure_size(object, "thickness_ mm")
if __name__ == "__main__":
render_flowchart(my_experiment)

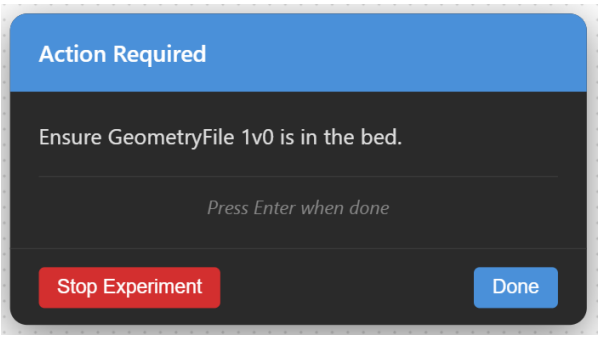

FEDT Visual Editor
Saved Experiments
Geometry Files
Load Example...
Run
Evaluate
Clear
Blockly Editor
Python Code
Output
Flowchart
Data
Files
Copy
Download .py
Experiment Setup
Factors & Nestedness
Artifact Design
Fabrication
Process Instructions
Measurement
Control Flow
Variables
Logic
Math
Lists
Text
from instruction import instruction
from iterators import Parallel, Series, shuffle, include_last
from measurement import BatchMeasurements, ImmediateMeasurements
from flowchart import FlowChart
from flowchart_render import render_flowchart
from decorator import fedt_experiment
from doe import Factor, FullFactorial
from lib import *
def summarize(data):
return "Experiment analysis complete!"
@fedt_experiment
def new_experiment():
results = ImmediateMeasurements.empty()
for material in Parallel(["acrylic", "wood"]):
2. Fabricate
object = Laser.fab(GeometryFile("rectangle.svg"), material=material,
3. Measure
results += Calipers.measure_size(object, "thickness_ mm")
summarize(results)
if __name__ == "__main__":
render_flowchart(my_experiment)
control.MODE = Execute()
my_experiment()
pass

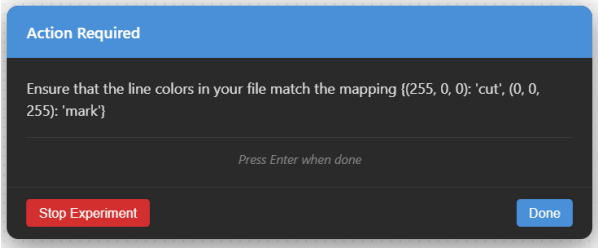

FEDT Visual Editor
Saved Experiments
Geometry Files
Load Example...
Run
Evaluate
Clear
Blockly Editor
Python Code
Output
Flowchart
Data
Files
Download SVG
Experiment Setup
Factors & Nestedness
Artifact Design
Fabrication
Process Instructions
Measurement
Control Flow
Variables
Logic
Math
Lists
Text

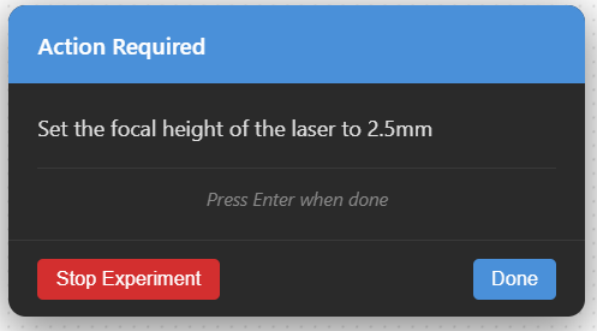

FEDT Visual Editor
Saved Experiments
Geometry Files
Load Example...
Run
Evaluate
Clear
Blockly Editor
Python Code
Output
Flowchart
Data
Files
Experiment Design
Export CSV
Measurements
Export CSV
Experiment Setup
Factors & Nestedness
Artifact Design
Fabrication
Process Instructions
Measurement
Control Flow
Variables
Logic
Math
Lists
Text

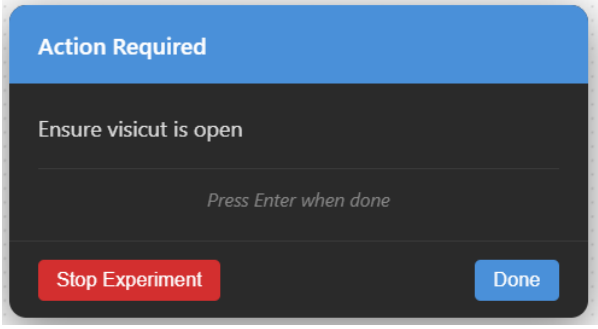
A.2 Execution and Results

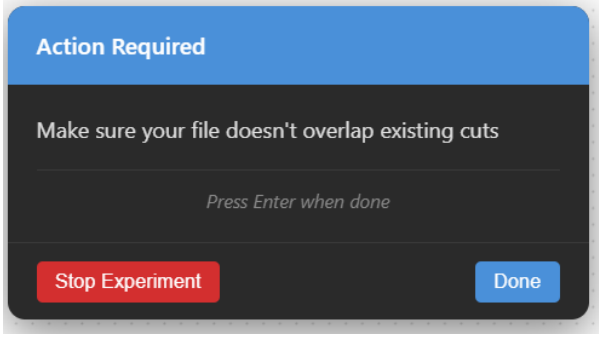
Below are the screenshots of the execution mode workflow. The interface guides users through each required action with modal dialogs: ensuring VisiCut laser software is open, setting the focal height, verifying line color mappings, placing the geometry file in the laser bed, avoiding overlaps, executing the laser cut, additional action prompts for the second material iteration, and the final measurement table showing recorded thickness values (5mm for acrylic, 7mm for wood) with Export CSV capability.

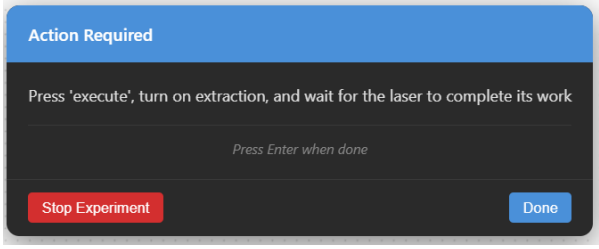




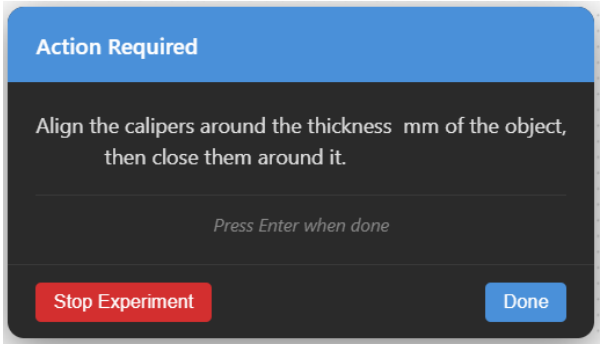


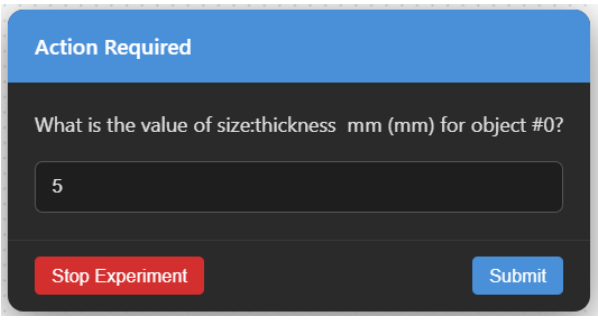


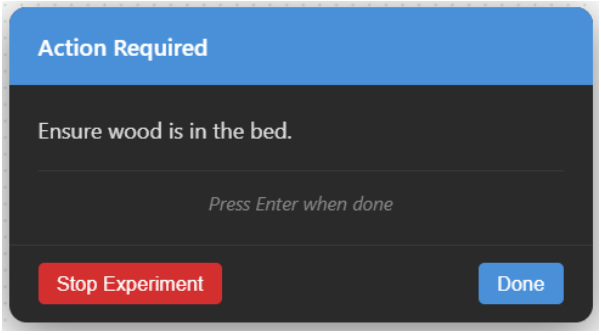


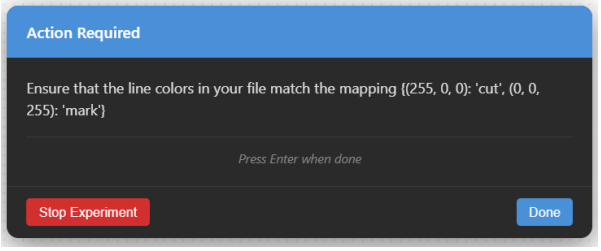


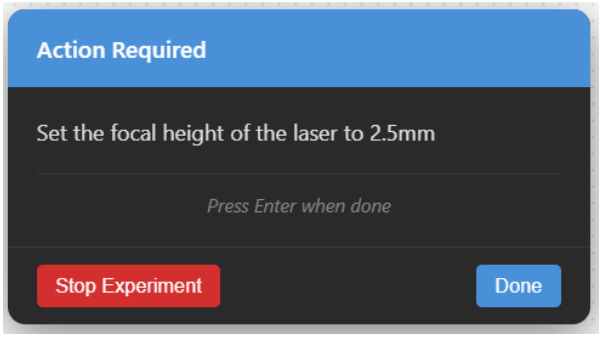
This walkthrough demonstrates how the visual interface supports the complete experimental workflow from initial design through execution to results collection, while maintaining connection to the underlying FEDT representation.

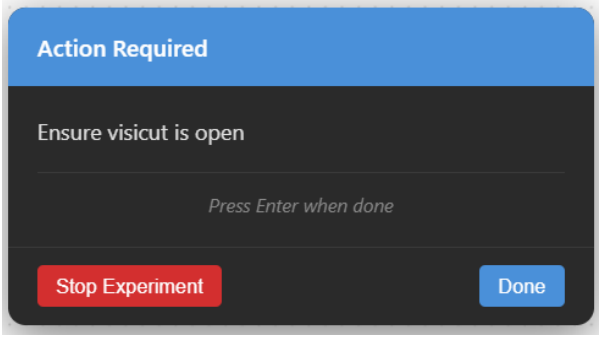

Action Required
Align the calipers around the thickness mm of the object, then close them around it.
Press Enter when done
Stop Experiment
Done

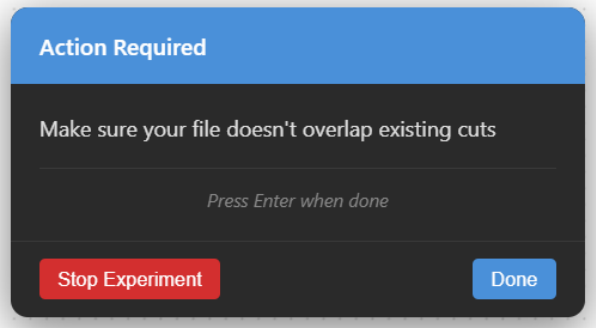

Action Required
What is the value of size:thickness mm (mm) for object #0?
5
Stop Experiment
Submit

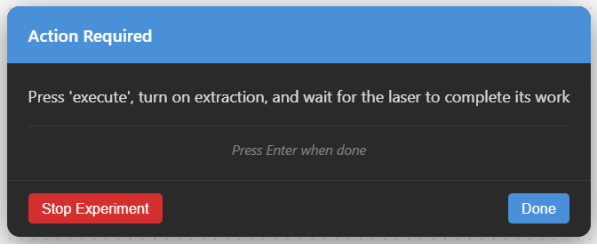

Action Required
Ensure wood is in the bed.
Press Enter when done
Stop Experiment
Done

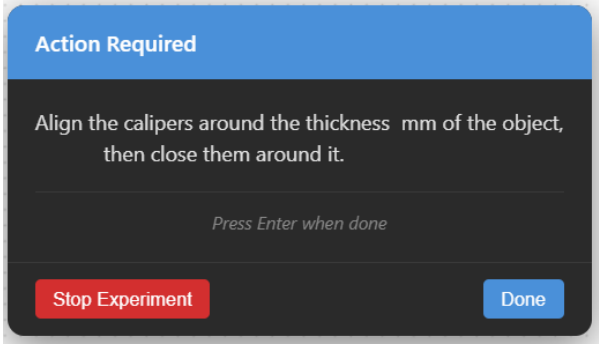

Action Required
Ensure that the line colors in your file match the mapping {(255, 0, 0): 'cut', (0, 0, 255): 'mark'}
Press Enter when done
Stop Experiment
Done

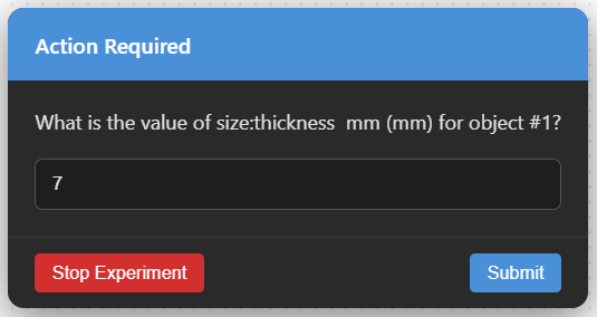

Action Required
Set the focal height of the laser to 2.5mm
Press Enter when done
Stop Experiment
Done

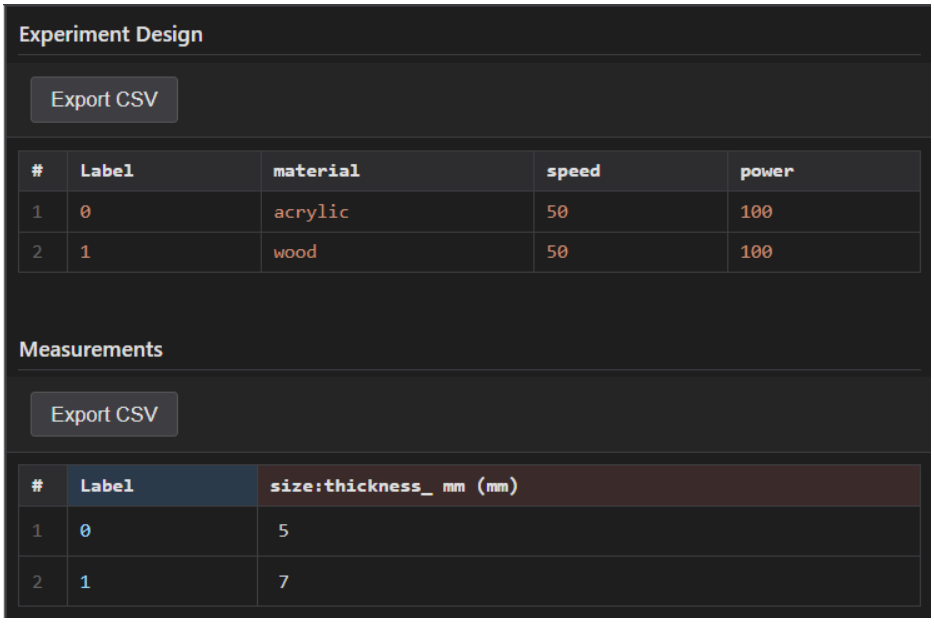

Action Required
Ensure visicut is open
Press Enter when done
Stop Experiment
Done


Action Required
Make sure your file doesn't overlap existing cuts
Press Enter when done
Stop Experiment
Done


Action Required
Press 'execute', turn on extraction, and wait for the laser to complete its work
Press Enter when done
Stop Experiment
Done


Action Required
Align the calipers around the thickness mm of the object, then close them around it.
Press Enter when done
Stop Experiment
Done


Action Required
What is the value of size:thickness mm (mm) for object #1?
7
Stop Experiment
Submit



Experiment Design

Export CSV

#	Label	material	speed	power
1	0	acrylic	50	100
2	1	wood	50	100

Measurements

Export CSV

#	Label	size:thickness_ mm (mm)
1	0	5
2	1	7

A.3 Formative Paper Prototype and Early Layout Sketches

This appendix shows low-fidelity design artifacts used during the formative exploration before the final implementation. The paper prototype task asked participants to arrange blocks for a material breakage experiment involving geometry loading, material iteration, laser cutting, manual instructions, and measurement collection. The early layout sketches show how this block-based workflow was translated into a browser-based interface with a block palette, workspace, run controls, and generated outputs.

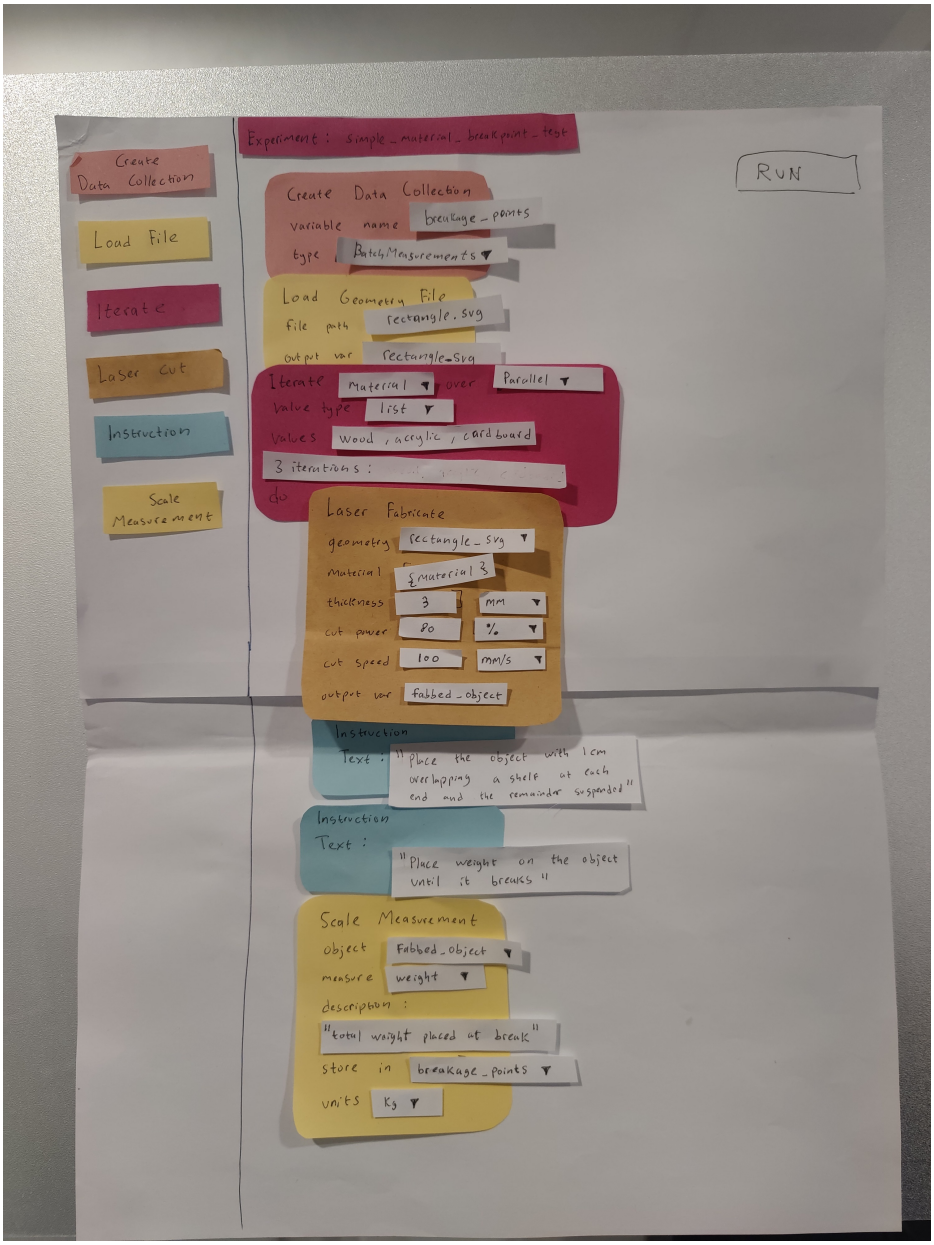


Figure 26: Low-fidelity paper prototype used during formative think-aloud exploration.

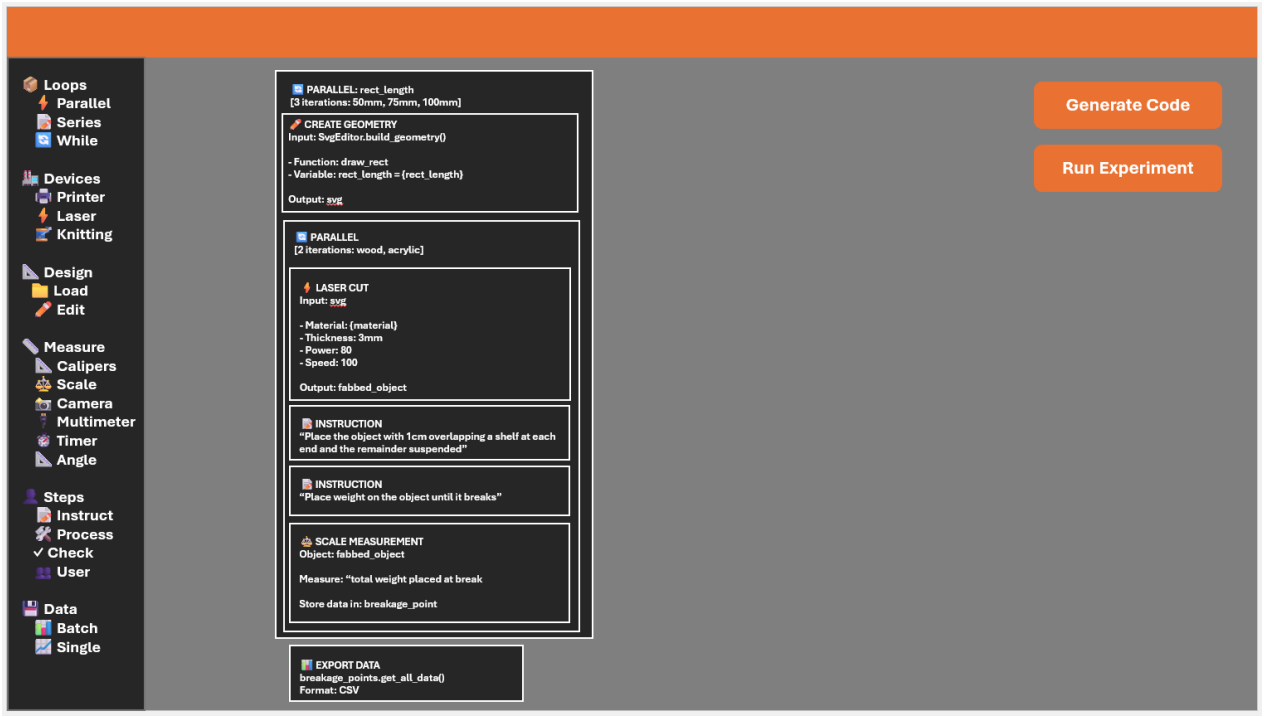


Figure 27: Early web-interface layout sketch showing a block palette, workflow canvas, and run controls.

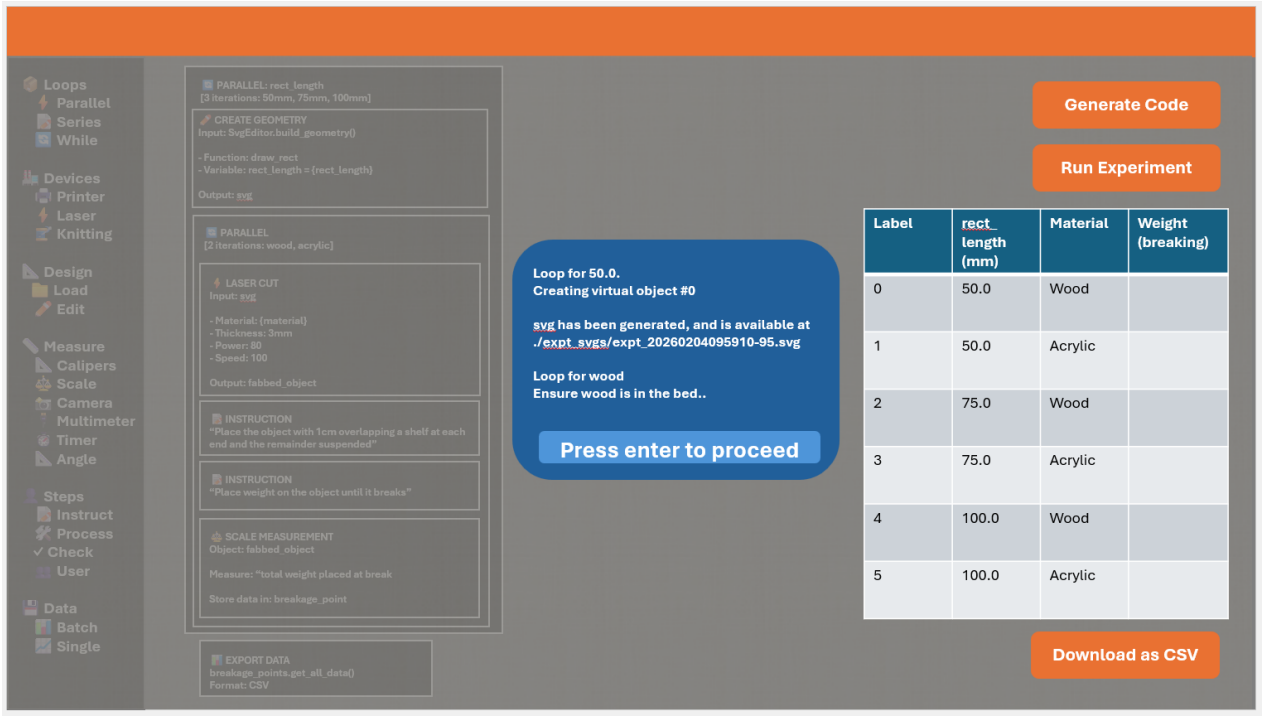


Figure 28: Early web-interface layout sketch exploring how generated flowcharts, execution controls, and experiment outputs could be presented alongside the block workspace.

Declaration of using generative AI tools (for students)

Declaration of using generative AI tools (for students)
☒ **I/we have used generative AI as an aid/tool** *(please tick)* ☐ **I/we have NOT used generative AI as an aid/tool** *(please tick)* *If generative AI is permitted in the exam, but you haven't used it in your exam paper, you just need to tick the box stating that you have not used GAI. You don't have to fill in the rest.*
List which GAI tools you have used and include the link to the platform (if possible): *GitHub Copilot (connected via my GitHub account linked to UCPH email):* *https://github.com/copilot* *LanguageTool (Grammar):* *https://languagetool.org* *Good Tape (AI transcription tool):* *https://goodtape.io/*
Describe how generative AI has been used in the exam paper: I primarily used generative AI tools for grammar correction, coding inspiration, and transcription support. For correcting sentence wording and grammar, I used GitHub Copilot VS Code and LanguageTool. I used it mainly during the writing phase to polish the text before supervisor meetings and before final submission. When reviewing suggestions from Copilot, I manually edited them to ensure they fit well with the overall flow and tone of the thesis. For coding inspiration, I also used GitHub Copilot in VS Code during the development phase of the web app. Copilot provided code auto completions while I was working on the frontend and backend code. I reviewed, modified, and tested the suggested code before including it in the project. For transcription, I used Good Tape to transcribe audio to text for preliminary research. I corrected the automated transcripts, when necessary, since automated transcription can contain errors, especially with technical terminology or non-English words.